# Nemo.jl

November 2, 2026

# Contents


Contents ii

I Getting Started 1
1 Installation 3
2 Quick start 4
3 Building dependencies from source 6
4 Experimental threading support for flint 7

II About Nemo 8
5 Why Julia? 10

III Types in Nemo 11

IV Constructing mathematical objects in Nemo 14
6 Constructing objects in Julia 15
7 How we construct objects in Nemo 16
8 Constructing parent objects 17
9 List of parent object constructors 18

V Rings 19
10 Integers 20
10.1 Integer functionality . . . 20
11 Complex Integers 43
12 Univariate polynomials 44
12.1 Introduction . . . 44
12.2 Polynomial functionality . . . 44
13 Multivariate polynomials 57
13.1 Introduction . . . 57
13.2 Polynomial functionality . . . 57
14 Power series and Laurent series 58
14.1 Capped relative power series . . . 59
14.2 Capped absolute power series . . . 60
14.3 Power series functionality . . . 60
15 Puiseux series 62
15.1 Puiseux power series . . . 62
15.2 Puiseux series functionality . . . 63
16 Residue rings 65
16.1 Residue functionality . . . 65

**VI Fields** **67**
**17 Fraction fields** **68**
17.1 Fraction functionality . . . 68
17.2 Rational enumeration . . . 71
**18 Rationals** **76**
**19 Algebraic closure of the rational numbers** **77**
19.1 Constructing the field of algebraic numbers . . . 77
19.2 Algebraic number functionality . . . 77
**20 Important note on performance** **93**
**21 Exact real and complex numbers** **94**
21.1 Constructor options . . . 95
21.2 Basic examples . . . 95
21.3 Conversions and numerical evaluation . . . 96
21.4 Comparisons and properties . . . 97
21.5 Infinities and special values . . . 100
21.6 Complex parts . . . 102
21.7 Elementary and special functions . . . 104
21.8 Rewriting and simplification . . . 109
**22 Arbitrary precision complex balls** **110**
22.1 Complex ball functionality . . . 110
22.2 Basic functionality . . . 111
**23 Arbitrary precision real balls** **131**
23.1 Real ball functionality . . . 131
**24 Fixed precision real balls** **155**
24.1 Real ball functionality . . . 155
**25 Fixed precision complex balls** **178**
25.1 Complex ball functionality . . . 178
25.2 Basic functionality . . . 179
**26 Galois fields** **199**
26.1 Galois field functionality . . . 199
26.2 Basic manipulation . . . 200
**27 Finite fields** **201**
27.1 Finite field functionality . . . 201
27.2 Constructors . . . 201
27.3 Field functionality . . . 202
27.4 Element functionality . . . 204
**28 Finite field embeddings** **206**
28.1 Introduction . . . 206
28.2 Embedding functionality . . . 206
**29 Number field arithmetic** **209**
29.1 Number field functionality . . . 209
**30 Padics** **216**
30.1 P-adic functionality . . . 216
**31 Qadics** **222**
31.1 P-adic functionality . . . 222

**VII Nemo matrices** **229**
**32 Matrix functionality** **231**
32.1 Comparison operators . . . 231
32.2 Scaling . . . 232
32.3 Determinant . . . 233

32.4 Pseudo inverse . . . 234
32.5 Nullspace . . . 234
32.6 Modular reduction . . . 234
32.7 Lifting . . . 235
32.8 Special matrices . . . 236
32.9 Hermite Normal Form . . . 236
32.10 Lattice basis reduction . . . 238
32.11 Smith Normal Form . . . 241
32.12 Strong Echelon Form . . . 242
32.13 Howell Form . . . 242
32.14 Gram-Schmidt Orthogonalisation . . . 243
32.15 Exponential . . . 243
32.16 Norm . . . 244
32.17 Shifting . . . 244
32.18 Predicates . . . 245
32.19 Conversion to Julia matrices . . . 245
32.20 Eigenvalues and Eigenvectors (experimental) . . . 245

**VIII Factorisation 248**
32.21 Basic functionality . . . 249

**IX Miscellaneous 251**
**33 Global variables and precompilation 252**

**X Developer 253**
**34 Introduction to Nemo development 254**
34.1 Relationship to AbstractAlgebra.jl . . . 254
34.2 Layout of files . . . 254
34.3 Git, GitHub and project workflows . . . 255
34.4 Development list . . . 257
34.5 Reporting bugs . . . 257
34.6 Development roadmap . . . 257
34.7 Binaries . . . 257
34.8 Relationship to Oscar . . . 257
**35 Conventions 258**
35.1 Code conventions . . . 258
**36 The type system 261**
36.1 Use of Julia types in Nemo . . . 261
36.2 Type hierarchy diagram . . . 264
**37 Parent objects 267**
37.1 The use of parent objects in Nemo . . . 267
**38 Interfaces 270**
38.1 Functionality for Generic and Abstract Types . . . 270
38.2 Generic interfaces . . . 271
38.3 Julia interfaces we support . . . 272
38.4 Column major vs row major matrices . . . 273
**39 Specific topics 274**
39.1 Julia arithmetic . . . 274
39.2 The Generic submodule . . . 277
39.3 Unsafe operations and aliasing . . . 277
39.4 Aliasing rules and mutation . . . 278

39.5 The SparsePoly module . . . . . 280
39.6 Parent object caching . . . . . 280
39.7 Throw/nothrow for `check_parent` . . . . . 280
39.8 Delayed reduction . . . . . 281

# Part I

# Getting Started

Nemo is a computer algebra package for the Julia programming language, maintained by William Hart, Tommy Hofmann, Claus Fieker, Fredrik Johansson with additional code by Oleksandr Motsak, Marek Kaluba and other contributors.

- https://github.com/Nemocas/Nemo.jl (Source code)
- https://nemocas.github.io/Nemo.jl/stable/ (Online documentation)

The features of Nemo so far include:

- Multiprecision integers and rationals
- Integers modulo n
- p-adic numbers
- Finite fields (prime and non-prime order)
- Number field arithmetic
- Algebraic numbers
- Exact real and complex numbers
- Arbitrary precision real and complex balls
- Univariate and multivariate polynomials and matrices over the above

Nemo depends on AbstractAlgebra.jl which provides Nemo with generic routines for:

- Univariate and multivariate polynomials
- Absolute and relative power series
- Laurent series
- Fraction fields
- Residue rings
- Matrices and linear algebra
- Young Tableaux
- Permutation groups
- Characters

# Chapter 1

# Installation

To use Nemo we require Julia 1.10 or higher. Please see https://julialang.org/downloads/ for instructions on how to obtain julia for your system.

At the Julia prompt simply type

```
julia> using Pkg; Pkg.add("Nemo")
```

# Chapter 2

# Quick start

Here are some examples of using Nemo.

This example computes recursive univariate polynomials.

```
julia> using Nemo

julia> R, x = polynomial_ring(ZZ, "x")
(Univariate polynomial ring in x over ZZ, x)

julia> S, y = polynomial_ring(R, "y")
(Univariate polynomial ring in y over univariate polynomial ring, y)

julia> T, z = polynomial_ring(S, "z")
(Univariate polynomial ring in z over univariate polynomial ring, z)

julia> f = x + y + z + 1
z + y + x + 1

julia> p = f^30; # semicolon suppresses output

julia> @time q = p*(p+1);
  0.161733 seconds (79.42 k allocations: 2.409 MiB)
```

Here is an example using generic recursive ring constructions.

```
julia> using Nemo

julia> R, x = finite_field(7, 11, "x")
(Finite field of degree 11 and characteristic 7, x)

julia> S, y = polynomial_ring(R, "y")
(Univariate polynomial ring in y over GF(7, 11), y)

julia> T, _ = residue_ring(S, y^3 + 3x*y + 1)
(Residue ring of univariate polynomial ring modulo y^3 + 3*x*y + 1, Map: univariate polynomial ring
↪  -> residue ring)

julia> U, z = polynomial_ring(T, "z")
(Univariate polynomial ring in z over residue ring, z)
```

```
julia> f = (3y^2 + y + x)*z^2 + ((x + 2)*y^2 + x + 1)*z + 4x*y + 3;

julia> g = (7y^2 - y + 2x + 7)*z^2 + (3y^2 + 4x + 1)*z + (2x + 1)*y + 1;

julia> s = f^12;

julia> t = (s + g)^12;

julia> @time resultant(s, t)
  0.059095 seconds (391.89 k allocations: 54.851 MiB, 5.22% gc time)
(x^10 + 4*x^8 + 6*x^7 + 3*x^6 + 4*x^5 + x^4 + 6*x^3 + 5*x^2 + x)*y^2 + (5*x^10 + x^8 + 4*x^7 +
↪  3*x^5 + 5*x^4 + 3*x^3 + x^2 + x + 6)*y + 2*x^10 + 6*x^9 + 5*x^8 + 5*x^7 + x^6 + 6*x^5 + 5*x^4 +
↪  4*x^3 + x + 3
```

Here is an example using matrices.

```
julia> using Nemo

julia> R, x = polynomial_ring(ZZ, "x")
(Univariate polynomial ring in x over ZZ, x)

julia> S = matrix_space(R, 40, 40)
Matrix space of 40 rows and 40 columns
  over univariate polynomial ring in x over ZZ

julia> M = rand(S, 2:2, -20:20);

julia> @time det(M);
  0.080976 seconds (132.28 k allocations: 23.341 MiB, 4.11% gc time)
```

And here is an example with power series.

```
julia> using Nemo

julia> R, x = QQ["x"]
(Univariate polynomial ring in x over QQ, x)

julia> S, t = power_series_ring(R, 100, "t")
(Univariate power series ring over univariate polynomial ring, t + O(t^101))

julia> u = t + O(t^100)
t + O(t^100)

julia> @time divexact((u*exp(x*u)), (exp(u)-1));
  0.412813 seconds (667.49 k allocations: 33.966 MiB, 90.26% compilation time)
```

## Chapter 3

# Building dependencies from source

Nemo depends on the FLINT C library which is installed using binaries by default. With Julia version >= 1.3, the use of this binary can be overridden by putting the following into the file `~/.julia/artifacts/Overrides.toml`:

```
[e134572f-a0d5-539d-bddf-3cad8db41a82]
FLINT = "/prefix/for/libflint"
```

## Chapter 4

# Experimental threading support for flint

Enabling a threaded version of flint can be done by setting the environment variable `NEMO_THREADED=1`. To set the actual number of threads, use `Nemo.flint_set_num_threads($numberofthreads)`.

# Part II

# About Nemo

Nemo is a library for fast basic arithmetic in various commonly used rings, for the Julia programming language. Our aim is to provide a highly performant package covering

- Commutative Algebra
- Number Theory
- Group Theory

Nemo consists of wrappers of the specialised C library FLINT http://flintlib.org/.

Nemo also uses AbstractAlgebra.jl to provide generic constructions over the basic rings provided by the above packages.

# Chapter 5

# Why Julia?

Julia is a sophisticated, modern programming language which is designed to be both performant and flexible. It was written by mathematicians, for mathematicians.

The benefits of Julia include

- Familiar imperative syntax
- JIT compilation (provides near native performance, even for highly generic code)
- REPL console (cuts down on development time)
- Parametric types (allows for fast generic constructions over other data types)
- Powerful metaprogramming facilities
- Operator overloading
- Multiple dispatch (dispatch on every argument of a function)
- Efficient native C interface (little or no wrapper overhead)
- Experimental C++ interface
- Dynamic type inference
- Built-in bignums
- Able to be embedded in C programs
- High performance collection types (dictionaries, iterators, arrays, etc.)
- Jupyter support (for web based notebooks)

The main benefits for Nemo are the parametric type system and JIT compilation. The former allows us to model many mathematical types, e.g. generic polynomial rings over an arbitrary base ring. The latter speeds up the runtime performance, even of highly generic mathematical procedures.

# Part III

# Types in Nemo

Nemo is fully compatible with AbstractAlgebra.jl, but specialises implementations of various commonly used rings with a highly optimised C implementation, provided by the C libraries wrapped by Nemo.

Below, we give a list of all of the specialised types available in Nemo that implement rings using a specialised C library. The types of elements of the respective rings and other mathematical structures are given, and in parentheses we list the types of the parent objects of the given rings and structures.

- `ZZRingElem` (`ZZRing`)
- `QQFieldElem` (`QQField`)
- `zzModRingElem` (`zzModRing`)
- `ZZModRingElem` (`ZZModRing`)
- `fqPolyRepFieldElem` (`fqPolyRepField`)
- `fpFieldElem` (`fpField`)
- `FpFieldElem` (`FpField`)
- `FqPolyRepFieldElem` (`FqPolyRepField`)
- `PadicFieldElem` (`PadicField`)
- `QadicFieldElem` (`QadicField`)
- `ZZPolyRingElem` (`ZZPolyRing`)
- `QQPolyRingElem` (`QQPolyRing`)
- `zzModPolyRingElem` (`zzModPolyRing`)
- `ZZModPolyRingElem` (`ZZModPolyRing`)
- `FqPolyRepPolyRingElem` (`FqPolyRepPolyRing`)
- `fqPolyRepPolyRingElem` (`fqPolyRepPolyRing`)
- `ZZMPolyRingElem` (`ZZMPolyRing`)
- `QQMPolyRingElem` (`QQMPolyRing`)
- `zzModMPolyRingElem` (`zzModMPolyRing`)
- `fqPolyRepMPolyRingElem` (fqPolyRepMPolyRing‘)
- `fpPolyRingElem` (`fpPolyRing`)
- `FpPolyRingElem` (`FpPolyRing`)
- `ZZRelPowerSeriesRingElem` (`ZZRelPowerSeriesRing`)
- `ZZAbsPowerSeriesRingElem` (`ZZAbsPowerSeriesRing`)
- `QQRelPowerSeriesRingElem` (`QQRelPowerSeriesRing`)
- `QQAbsPowerSeriesRingElem` (`QQAbsPowerSeriesRing`)
- `ZZModRelPowerSeriesRingElem` (`ZZModRelPowerSeriesRing`)

- ZZModAbsPowerSeriesRingElem (ZZModAbsPowerSeriesRing)
- zzModRelPowerSeriesRingElem (zzModRelPowerSeriesRing)
- zzModAbsPowerSeriesRingElem (zzModAbsPowerSeriesRing)
- fpRelPowerSeriesRingElem (fpRelPowerSeriesRing)
- fpAbsPowerSeriesRingElem (fpAbsPowerSeriesRing)
- FpRelPowerSeriesRingElem (FpRelPowerSeriesRing)
- FpAbsPowerSeriesRingElem (FpAbsPowerSeriesRing)
- fqPolyRepRelPowerSeriesRingElem (fqPolyRepRelPowerSeriesRing)
- fqPolyRepAbsPowerSeriesRingElem (fqPolyRepAbsPowerSeriesRing)
- FqPolyRepRelPowerSeriesRingElem (FqPolyRepRelPowerSeriesRing)
- FqPolyRepAbsPowerSeriesRingElem (FqPolyRepAbsPowerSeriesRing)
- ZZMatrix (ZZMatrixSpace)
- QQMatrix (QQMatrixSpace)
- zzModMatrix (zzModMatrixSpace)
- ZZModMatrix (ZZModMatrixSpace')
- fqPolyRepMatrix (fqPolyRepMatrixSpace)
- FqPolyRepMatrix (FqPolyRepMatrixSpace)
- fpMatrix (fpMatrixSpace)
- perm (SymmetricGroup)
- AbsSimpleNumFieldElem (AbsSimpleNumField)
- ArbFieldElem (ArbField)
- AcbFieldElem (AcbField)
- ArbPolyRingElem (ArbPolyRing)
- AcbPolyRingElem (AcbPolyRing)
- ArbMatrix (ArbMatrixSpace)
- AcbMatrix (AcbMatrixSpace)
- QQBarFieldElem (QQBarField)
- CalciumFieldElem (CalciumField)

# Part IV

# Constructing mathematical objects in Nemo

# Chapter 6

# Constructing objects in Julia

In Julia, one constructs objects of a given type by calling a type constructor. This is simply a function with the same name as the type itself. For example, to construct a `BigInt` object in Julia, we simply call the `BigInt` constructor:

```
julia> BigInt(12345678987654345678987654345678765434567876545678 90)
12345678987654345678987654345678765434567876545678 90
```

Julia also uses constructors to convert between types. For example, to convert an `Int` to a `BigInt`:

```
julia> m = BigInt(123)
123
```

# Chapter 7

# How we construct objects in Nemo

Julia types don't contain enough information to properly model groups, rings and fields, especially if they are parameterised by values. For example, the ring of integers modulo $n$ for a multiprecision modulus $n$ cannot be modeled using types alone.

Instead of using types to construct objects in Nemo, we use special objects that we refer to as parent objects. They behave a lot like Julia types.

Consider the following simple example, to create a FLINT multiprecision integer:

```
julia> n = ZZ(12345678765456787654567890987654567898765678909876567890)
12345678765456787654567890987654567898765678909876567890
```

Here ZZ is not a Julia type, but a callable object. However, for most purposes one can think of such a parent object ZZ as though it were a type.

# Chapter 8

# Constructing parent objects

For more complicated groups, rings, fields, etc., one first needs to construct the parent object before one can use it to construct element objects.

Nemo provides a set of functions for constructing such parent objects. For example, to create a parent object for polynomials over the integers, we use the `polynomial_ring` parent object constructor.

```
julia> R, x = polynomial_ring(ZZ, "x")
(Univariate polynomial ring in x over ZZ, x)

julia> f = x^3 + 3x + 1
x^3 + 3*x + 1

julia> g = R(12)
12
```

In this example, $R$ is the parent object and we use it to convert the `Int` value $12$ to an element of the polynomial ring $\mathbb{Z}[x]$.

# Chapter 9

# List of parent object constructors

For convenience, we provide a list of all the parent object constructors in Nemo and explain what domains they represent.

| Mathematics | Nemo constructor |
|---:|---:|
| $R = \mathbb{Z}$ | `R = ZZ` |
| $R = \mathbb{Q}$ | `R = QQ` |
| $R = \mathbb{F}_{p^n}$ | `R, a = finite_field(p, n, "a")` |
| $R = \mathbb{Z}/n\mathbb{Z}$ | `R, = residue_ring(ZZ, n)` |
| $S = R[x]$ | `S, x = polynomial_ring(R, "x")` |
| $S = R[x, y]$ | `S, (x, y) = polynomial_ring(R, ["x", "y"])` |
| $S = R[[x]]$ (to precision $n$) | `S, x = power_series_ring(R, n, "x")` |
| $S = R((x))$ (to precision $n$) | `S, x = laurent_series_ring(R, n, "x")` |
| $S = \mathrm{Frac}_R$ | `S = fraction_field(R)` |
| $S = R/(f)$ | `S, = residue_ring(R, f)` |
| $S = \mathrm{Mat}_{m\times n}(R)$ | `S = matrix_space(R, m, n)` |
| $S = \mathbb{Q}[x]/(f)$ | `S, a = number_field(f, "a")` |
| $S = \mathbb{Q}_p$ (to precision $N$) | `S = PadicField(p, n)` |
| $S = \mathbb{R}$ | `S = real_field()` |
| $S = \mathbb{C}$ | `S = complex_field()` |

# Part V

# Rings

# Chapter 10

# Integers

The default integer type in Nemo is provided by FLINT. The associated ring of integers is represented by the constant parent object called ZZ. Note that this is the name of a specific parent object, not the name of its type.

The types of the integer ring parent objects and elements of the associated rings of integers are given in the following table according to the library providing them.

| Library | Element type | Parent type |
|---|---|---|
| FLINT | `ZZRingElem` | `ZZRing` |

All integer element types belong directly to the abstract type `RingElem` and all the integer ring parent object types belong to the abstract type `Ring`.

A lot of code will want to accept both `ZZRingElem` integers and Julia integers, that is, subtypes of `Base.Integer`. Thus for convenience we define

```
IntegerUnion = Union{Integer,ZZRingElem}
```

## 10.1 Integer functionality

Nemo integers provide all of the ring and Euclidean ring functionality of AbstractAlgebra.jl.

https://nemocas.github.io/AbstractAlgebra.jl/stable/ring

https://nemocas.github.io/AbstractAlgebra.jl/stable/euclidean_interface

Below, we describe the functionality that is specific to the Nemo/FLINT integer ring.

### Constructors

```
ZZ(n::Integer)
```

Coerce a Julia integer value into the integer ring.

```
ZZ(n::String)
```

Parse the given string as an integer.

```
ZZ(n::Float64)
ZZ(n::Float32)
ZZ(n::Float16)
ZZ(n::BigFloat)
```

Coerce the given floating point number into the integer ring, assuming that it can be exactly represented as an integer.

### Basic manipulation

Base.sign – Method.

```
sign(a::ZZRingElem)
```

Return the sign of $a$, i.e. $+1$, $0$ or $-1$.

source

```
sign(g::Perm)
```

Return the sign of a permutation.

sign returns $1$ if g is even and $-1$ if g is odd. sign represents the homomorphism from the permutation group to the unit group of $\mathbb{Z}$ whose kernel is the alternating group.

**Examples**

```
julia> g = Perm([3,4,1,2,5])
(1,3)(2,4)

julia> sign(g)
1

julia> g = Perm([3,4,5,2,1,6])
(1,3,5)(2,4)

julia> sign(g)
-1
```

source

Base.size – Method.

```
size(a::ZZRingElem)
```

Return the number of limbs required to store the absolute value of $a$.

source

`Nemo.fits` – Method.

```
fits(::Type{UInt}, a::ZZRingElem)
```

Return `true` if $a$ fits into a `UInt`, otherwise return `false`.

source

`Nemo.fits` – Method.

```
fits(::Type{Int}, a::ZZRingElem)
```

Return `true` if $a$ fits into an `Int`, otherwise return `false`.

source

`Base.denominator` – Method.

```
denominator(a::ZZRingElem)
```

Return the denominator of $a$ thought of as a rational. Always returns $1$.

source

`Base.numerator` – Method.

```
numerator(a::ZZRingElem)
```

Return the numerator of $a$ thought of as a rational. Always returns $a$.

source

`Nemo.set!` – Method.

```
set!(z::TypeOrPtr{ZZRingElem}, a::TypeOrPtr{ZZRingElem})
set!(z::TypeOrPtr{ZZRingElem}, a::Integer)
```

Change `z` to be equal to `a` and return `z`.

The command `set!(z, a)` has essentially the same effect as `z = ZZ(a)`, except that in the former case, the existing object `z` references is modified while in the latter `z` is changed to point to a new object.

A benefit of this that it generally is faster and avoids allocations, at least if `a` is small enough to fit in the already allocated storage of `z`.

However, this can also have unintended consequences if for example some other variable references the same object as z.

**Examples**

```
julia> a = ZZ(304); z = ZZ(936); x = z
936

julia> set!(z, a)
304

julia> add!(z, 123)
427

julia> (a, x, z)
(304, 427, 427)

julia> x === z
true

julia> A = zeros(ZZRingElem, 2, 2) # All entries of `A` point to the same object.
2×2 Matrix{ZZRingElem}:
 0  0
 0  0

julia> A[1,1] = ZZ(5) # Create a new object and let `A[1,1]` point to it.
5

julia> set!(A[1,1], 8) # So, changing `A[1,1]` using `set!` does not change the other entries of
↪  `A`.
8

julia> set!(A[1,2], 3) # The other three matrix entries are still pointing to the same object.
↪  Changing one of them using `set!` changes all of them.
3

julia> A
2×2 Matrix{ZZRingElem}:
 8  3
 3  3
```

source

**Examples**

```
julia> a = ZZ(12)
12

julia> is_unit(a)
false

julia> sign(a)
1

julia> s = size(a)
```

```
1

julia> fits(Int, a)
true

julia> n = numerator(a)
12

julia> d = denominator(a)
1
```

### Euclidean division

Nemo also provides a large number of Euclidean division operations. Recall that for a dividend $a$ and divisor $b$, we can write $a = bq + r$ with $0 \leq |r| < |b|$. We call $q$ the quotient and $r$ the remainder.

We distinguish three cases. If $q$ is rounded towards zero, $r$ will have the same sign as $a$. If $q$ is rounded towards plus infinity, $r$ will have the opposite sign to $b$. Finally, if $q$ is rounded towards minus infinity, $r$ will have the same sign as $b$.

In the following table we list the division functions and their rounding behaviour. We also give the return value of the function, with $q$ representing return of the quotient and $r$ representing return of the remainder.

| Function | Return | Rounding of the quotient |
|---:|---:|---:|
| `mod` | r | towards minus infinity |
| `rem` | r | towards zero |
| `div` | q | towards minus infinity |
| `divrem(a::ZZRingElem, b::ZZRingElem)` | q, r | towards minus infinity |
| `tdivrem(a::ZZRingElem, b::ZZRingElem)` | q, r | towards zero |
| `fdivrem(a::ZZRingElem, b::ZZRingElem)` | q, r | towards minus infinity |
| `cdivrem(a::ZZRingElem, b::ZZRingElem)` | q, r | towards plus infinity |
| `ntdivrem(a::ZZRingElem, b::ZZRingElem)` | q, r | nearest integer, ties toward zero |
| `nfdivrem(a::ZZRingElem, b::ZZRingElem)` | q, r | nearest integer, ties toward minus infinity |
| `ncdivrem(a::ZZRingElem, b::ZZRingElem)` | q, r | nearest integer, ties toward plus infinity |

N.B: the internal definition of `Nemo.div` and `Nemo.divrem` are the same as `fdiv` and `fdivrem`. The definitions in the table are of `Base.div` and `Base.divrem` which agree with Julia's definitions of `div` and `divrem`.

Nemo also offers the following ad hoc division operators. The notation and description is as for the other Euclidean division functions.

| Function | Return | Rounding |
|---:|---:|---:|
| `mod(a::ZZRingElem, b::Int)` | r | towards minus infinity |
| `rem(a::ZZRingElem, b::Int)` | r | towards zero |
| `div(a::ZZRingElem, b::Int)` | q | towards zero |
| `tdiv(a::ZZRingElem, b::Int)` | q | towards zero |
| `fdiv(a::ZZRingElem, b::Int)` | q | towards minus infinity |
| `cdiv(a::ZZRingElem, b::Int)` | q | towards plus infinity |

N.B: the internal definition of `Nemo.div` is the same as `fdiv`. The definition in the table is `Base.div` which agrees with Julia's definition of `div`.

The following functions are also available, for the case where one is dividing by a power of $2$. In other words, for Euclidean division of the form $a = b2^d + r$. These are useful for bit twiddling.

| Function | Return | Rounding |
|---|---|---|
| `tdivpow2(a::ZZRingElem, d::Int)` | q | towards zero |
| `fdivpow2(a::ZZRingElem, d::Int)` | q | towards minus infinity |
| `fmodpow2(a::ZZRingElem, d::Int)` | r | towards minus infinity |
| `cdivpow2(a::ZZRingElem, d::Int)` | q | towards plus infinity |

**Examples**

```
julia> a = ZZ(12)
12

julia> b = ZZ(5)
5

julia> q, r = divrem(a, b)
(2, 2)

julia> c = cdiv(a, b)
3

julia> d = fdiv(a, b)
2

julia> f = tdivpow2(a, 2)
3

julia> g = fmodpow2(a, 3)
4
```

### Comparison

Instead of `isless` we implement a function `cmp(a, b)` which returns a positive value if $a > b$, zero if $a == b$ and a negative value if $a < b$. We then implement all the other operators, including == in terms of `cmp`.

For convenience we also implement a `cmpabs(a, b)` function which returns a positive value if $|a| > |b|$, zero if $|a| == |b|$ and a negative value if $|a| < |b|$. This can be slightly faster than a call to `cmp` or one of the comparison operators when comparing non-negative values for example.

Here is a list of the comparison functions implemented, with the understanding that `cmp` provides all of the comparison operators listed above.

| Function |
|---|
| `cmp(a::ZZRingElem, b::ZZRingElem)` |
| `cmpabs(a::ZZRingElem, b::ZZRingElem)` |

We also provide the following ad hoc comparisons which again provide all of the comparison operators mentioned above.

**Examples**

| Function |
|---|
| cmp(a::ZZRingElem, b::Int) |
| cmp(a::Int, b::ZZRingElem) |
| cmp(a::ZZRingElem, b::UInt) |
| cmp(a::UInt, b::ZZRingElem) |

```
julia> a = ZZ(12)
12

julia> b = ZZ(3)
3

julia> a < b
false

julia> a != b
true

julia> a > 4
true

julia> 5 <= b
false

julia> cmpabs(a, b)
1
```

### Shifting

Base.:<< – Method.

```
<<(x::ZZRingElem, c::Int)
```

Return $2^c x$ where $c \geq 0$.

source

Base.:>> – Method.

```
>>(x::ZZRingElem, c::Int)
```

Return $x/2^c$, discarding any remainder, where $c \geq 0$.

source

### Examples

```
julia> a = ZZ(12)
12

julia> a << 3
96

julia> a >> 5
0
```

### Modular arithmetic

Nemo.sqrtmod – Method.

```
sqrtmod(x::ZZRingElem, m::ZZRingElem)
```

Return a square root of $x( \mod m)$ if one exists. The remainder will be in the range $[0, m)$. We require that $m$ is prime, otherwise the algorithm may not terminate.

**Examples**

```
julia> sqrtmod(ZZ(12), ZZ(13))
5
```

source

Nemo.sqrtmod_all – Method.

```
sqrtmod_all(a::IntegerUnion, n::IntegerUnion) -> Vector{ZZRingElem}
```

Return every $x$ with $0 \leq x < n$ and $x^2 \equiv a \pmod n$, in increasing order, and none if $a$ is not a square modulo $n$. The modulus must be positive and fit in a UInt; $a$ is any integer, not necessarily a unit modulo $n$.

There can be many roots: $2^k$ for $a = 0$ and $n = 4^k$. For one square root modulo a prime, see sqrtmod.

**Examples**

```
julia> sqrtmod_all(4, 15)
4-element Vector{ZZRingElem}:
  2
  7
  8
 13

julia> sqrtmod_all(-7, 1024)
4-element Vector{ZZRingElem}:
 181
 331
 693
```

```
 843

julia> sqrtmod_all(2, 5)
ZZRingElem[]
```

source

AbstractAlgebra.crt – Function.

```
crt(r1::ZZRingElem, m1::ZZRingElem, r2::ZZRingElem, m2::ZZRingElem, signed=false;
↪  check::Bool=true)
crt(r1::ZZRingElem, m1::ZZRingElem, r2::Union{Int, UInt}, m2::Union{Int, UInt}, signed=false;
↪  check::Bool=true)
crt(r::Vector{ZZRingElem}, m::Vector{ZZRingElem}, signed=false; check::Bool=true)
crt_with_lcm(r1::ZZRingElem, m1::ZZRingElem, r2::ZZRingElem, m2::ZZRingElem, signed=false;
↪  check::Bool=true)
crt_with_lcm(r1::ZZRingElem, m1::ZZRingElem, r2::Union{Int, UInt}, m2::Union{Int, UInt},
↪  signed=false; check::Bool=true)
crt_with_lcm(r::Vector{ZZRingElem}, m::Vector{ZZRingElem}, signed=false; check::Bool=true)
```

As per the AbstractAlgebra `crt` interface, with the following option. If `signed = true`, the solution is the range $(-m/2, m/2]$, otherwise it is in the range $[0, m)$, where $m$ is the least common multiple of the moduli.

**Examples**

```
julia> crt(ZZ(5), ZZ(13), ZZ(7), ZZ(37), true)
44

julia> crt(ZZ(5), ZZ(13), 7, 37, true)
44
```

source

### Integer logarithm

Nemo.flog – Method.

```
flog(x::ZZRingElem, c::ZZRingElem)
flog(x::ZZRingElem, c::Int)
```

Return the floor of the logarithm of $x$ to base $c$.

**Examples**

```
julia> flog(ZZ(12), ZZ(2))
3

julia> flog(ZZ(12), 3)
2
```

source

Nemo.clog – Method.

```
clog(x::ZZRingElem, c::ZZRingElem)
clog(x::ZZRingElem, c::Int)
```

Return the ceiling of the logarithm of $x$ to base $c$.

**Examples**

```
julia> clog(ZZ(12), ZZ(2))
4

julia> clog(ZZ(12), 3)
3
```

source

### Integer roots

Base.isqrt – Method.

```
isqrt(x::ZZRingElem)
```

Return the floor of the square root of $x$.

**Examples**

```
julia> isqrt(ZZ(13))
3
```

source

Nemo.isqrtrem – Method.

```
isqrtrem(x::ZZRingElem)
```

Return a tuple $s, r$ consisting of the floor $s$ of the square root of $x$ and the remainder $r$, i.e. such that $x = s^2 + r$. We require $x \geq 0$.

**Examples**

```
julia> isqrtrem(ZZ(13))
(3, 4)
```

source

`AbstractAlgebra.root` – Method.

```
root(x::ZZRingElem, n::Int; check::Bool=true)
```

Return the $n$-the root of $x$. We require $n > 0$ and that $x \geq 0$ if $n$ is even. By default the function tests whether the input was a perfect $n$-th power and if not raises an exception. If `check=false` this check is omitted.

**Examples**

```
julia> root(ZZ(27), 3; check=true)
3
```

source

`AbstractAlgebra.iroot` – Method.

```
iroot(x::ZZRingElem, n::Int)
```

Return the integer truncation of the $n$-the root of $x$ (round towards zero). We require $n > 0$ and that $x \geq 0$ if $n$ is even.

**Examples**

```
julia> iroot(ZZ(13), 3)
2
```

source

### Number theoretic functionality

`AbstractAlgebra.is_divisible_by` – Method.

```
is_divisible_by(x::TypeOrPtr{ZZRingElem}, y::Int)
```

Return `true` if $x$ is divisible by $y$, otherwise return `false`.

source

`AbstractAlgebra.is_divisible_by` – Method.

```
is_divisible_by(x::T, y::T) where T <: RingElem
```

Check if x is divisible by y, i.e. if $x = zy$ for some $z$.

source

```
is_divisible_by(x::TypeOrPtr{ZZRingElem}, y::TypeOrPtr{ZZRingElem})
```

Return `true` if $x$ is divisible by $y$, otherwise return `false`.

source

`AbstractAlgebra.is_square` – Method.

```
is_square(a::NCRingElement)
```

Return `true` iff a is the square of a value in its own ring. See also `is_square(A::MatElem)` which tests whether a matrix has square shape.

source

`Nemo.is_prime` – Method.

```
is_prime(x::ZZRingElem)
is_prime(x::Int)
```

Return `true` if $x$ is a prime number, otherwise return `false`.

**Examples**

```
julia> is_prime(ZZ(13))
true
```

source

`AbstractAlgebra.is_probable_prime` – Method.

```
is_probable_prime(x::ZZRingElem)
```

Return `true` if $x$ is very probably a prime number, otherwise return `false`. No counterexamples are known to this test, but it is conjectured that infinitely many exist.

source

`AbstractAlgebra.factor` – Method.

```
factor(a::T) where T <: RingElement -> Fac{T}
```

Return a factorization of $a$ into irreducible elements, as a `Fac{T}`. The irreducible elements in the factorization are pairwise coprime.

source

`Nemo.divisor_lenstra` – Method.

```
divisor_lenstra(n::ZZRingElem, r::ZZRingElem, m::ZZRingElem)
```

If $n$ has a factor which lies in the residue class $r(\mod m)$ for $0 < r < m < n$, this function returns such a factor. Otherwise it returns $0$. This is only efficient if $m$ is at least the cube root of $n$. We require $\gcd(r, m) = 1$ and this condition is not checked.

source

`Base.factorial` – Method.

```
factorial(x::ZZRingElem)
```

Return the factorial of $x$, i.e. $x! = 1.2.3 \ldots x$. We require $x \geq 0$.

**Examples**

```
julia> factorial(ZZ(100))
933262154439441526816992388562667004907159682643816214685929638952175999932299156089414639761565182862536979208272
```

source

`AbstractAlgebra.Generic.rising_factorial` – Method.

```
rising_factorial(x::ZZRingElem, n::ZZRingElem)
```

Return the rising factorial of $x$, i.e. $x(x+1)(x+2)\cdots(x+n-1)$. If $n < 0$ we throw a `DomainError()`.

source

`AbstractAlgebra.Generic.rising_factorial` – Method.

```
rising_factorial(x::ZZRingElem, n::Int)
```

Return the rising factorial of $x$, i.e. $x(x+1)(x+2)\ldots(x+n-1)$. If $n < 0$ we throw a `DomainError()`.

source

`AbstractAlgebra.Generic.rising_factorial` – Method.

```
rising_factorial(x::RingElement, n::Integer)
```

Return the rising factorial of $x$, i.e. $x(x+1)(x+2)\cdots(x+n-1)$. If $n < 0$ we throw a `DomainError()`.

**Examples**

```
julia> R, x = ZZ[:x];

julia> rising_factorial(x, 1)
x

julia> rising_factorial(x, 2)
x^2 + x

julia> rising_factorial(4, 2)
20
```

source

Nemo.primorial – Method.

```
primorial(x::ZZRingElem)
```

Return the primorial of $x$, i.e. the product of all primes less than or equal to $x$. If $x < 0$ we throw a DomainError().

source

Nemo.primorial – Method.

```
primorial(x::Int)
```

Return the primorial of $x$, i.e. the product of all primes less than or equal to $x$. If $x < 0$ we throw a DomainError().

source

Nemo.fibonacci – Method.

```
fibonacci(x::Int)
```

Return the $x$-th Fibonacci number $F_x$. We define $F_1 = 1$, $F_2 = 1$ and $F_{i+1} = F_i + F_{i-1}$ for all integers $i$.

source

Nemo.fibonacci – Method.

```
fibonacci(x::ZZRingElem)
```

Return the $x$-th Fibonacci number $F_x$. We define $F_1 = 1$, $F_2 = 1$ and $F_{i+1} = F_i + F_{i-1}$ for all integers $i$.

source

Nemo.bell – Method.

```
bell(x::ZZRingElem)
```

Return the Bell number $B_x$.

source

Nemo.bell – Method.

```
bell(x::Int)
```

Return the Bell number $B_x$.

source

Base.binomial – Method.

```
binomial(n::ZZRingElem, k::ZZRingElem)
```

Return the binomial coefficient $\frac{n(n-1)\cdots(n-k+1)}{k!}$. If $k < 0$ we return $0$, and the identity `binomial(n, k) == binomial(n - 1, k - 1) + binomial(n - 1, k)` always holds for integers n and k.

source

Base.binomial – Method.

```
binomial(n::UInt, k::UInt, ::ZZRing)
```

Return the binomial coefficient $\frac{n!}{(n-k)!k!}$ as an ZZRingElem.

source

Nemo.moebius_mu – Method.

```
moebius_mu(x::Int)
```

Return the Moebius mu function of $x$ as an Int. The value returned is either $-1$, $0$ or $1$. If $x \leq 0$ we throw a DomainError().

source

Nemo.moebius_mu – Method.

```
moebius_mu(x::ZZRingElem)
```

Return the Moebius mu function of $x$ as an Int. The value returned is either $-1$, $0$ or $1$. If $x \leq 0$ we throw a DomainError().

source

Nemo.jacobi_symbol – Method.

```
jacobi_symbol(x::Int, y::Int)
```

Return the value of the Jacobi symbol $\left(\frac{x}{y}\right)$. The modulus $y$ must be odd and positive, otherwise a DomainError is thrown.

source

Nemo.jacobi_symbol – Method.

```
jacobi_symbol(x::ZZRingElem, y::ZZRingElem)
```

Return the value of the Jacobi symbol $\left(\frac{x}{y}\right)$. The modulus $y$ must be odd and positive, otherwise a DomainError is thrown.

source

Nemo.kronecker_symbol – Method.

```
kronecker_symbol(x::ZZRingElem, y::ZZRingElem)
kronecker_symbol(x::Int, y::Int)
```

Return the value of the Kronecker symbol $\left(\frac{x}{y}\right)$. The definition is as per Henri Cohen's book, "A Course in Computational Algebraic Number Theory", Definition 1.4.8.

source

Nemo.divisor_sigma – Method.

```
divisor_sigma(x::ZZRingElem, y::Int)
divisor_sigma(x::ZZRingElem, y::ZZRingElem)
divisor_sigma(x::Int, y::Int)
```

Return the value of the sigma function, i.e. $\sum_{0<d\ |x} d^y$. If $x \leq 0$ or $y < 0$ we throw a DomainError().

**Examples**

```
julia> divisor_sigma(ZZ(32), 10)
1127000493261825

julia> divisor_sigma(ZZ(32), ZZ(10))
1127000493261825

julia> divisor_sigma(32, 10)
1127000493261825
```

source

Nemo.euler_phi – Method.

```
euler_phi(x::ZZRingElem)
euler_phi(x::Int)
```

Return the value of the Euler phi function at $x$, i.e. the number of positive integers up to $x$ (inclusive) that are coprime with $x$. An exception is raised if $x \leq 0$.

**Examples**

```
julia> euler_phi(ZZ(12480))
3072

julia> euler_phi(12480)
3072
```

source

Nemo.number_of_partitions – Method.

```
number_of_partitions(x::Int)
number_of_partitions(x::ZZRingElem)
```

Return the number of partitions of $x$.

**Examples**

```
julia> number_of_partitions(100)
190569292

julia> number_of_partitions(ZZ(1000))
24061467864032622473692149727991
```

source

Nemo.is_perfect_power – Method.

```
is_perfect_power(a::IntegerUnion)
```

Return whether $a$ is a perfect power, that is, whether $a = m^r$ for some integer $m$ and $r > 1$. Neither $m$ nor $r$ is returned.

source

Nemo.is_prime_power – Method.

```
is_prime_power(q::IntegerUnion) -> Bool
```

Returns whether $q$ is a prime power.

source

Nemo.is_prime_power_with_data – Method.

```
is_prime_power_with_data(q::T) where T <: IntegerUnion -> Bool, Int, T
```

Returns a flag indicating whether $q$ is a prime power and integers $e, p$ such that $q = p^e$. If $q$ is a prime power, than $p$ is a prime.

source

### Digits and bases

Base.bin – Method.

```
bin(n::ZZRingElem)
```

Return $n$ as a binary string.

**Examples**

```
julia> bin(ZZ(12))
"1100"
```

source

Base.oct – Method.

```
oct(n::ZZRingElem)
```

Return $n$ as a octal string.

**Examples**

```
julia> oct(ZZ(12))
"14"
```

source

Base.dec – Method.

```
dec(n::ZZRingElem)
```

Return $n$ as a decimal string.

**Examples**

```
julia> dec(ZZ(12))
"12"
```

source

Base.hex – Method.

```
hex(n::ZZRingElem) = base(n, 16)
```

Return $n$ as a hexadecimal string.

**Examples**

```
julia> hex(ZZ(12))
"c"
```

source

Nemo.base – Method.

```
base(n::ZZRingElem, b::Integer)
```

Return $n$ as a string in base $b$. We require $2 \leq b \leq 62$.

**Examples**

```
julia> base(ZZ(12), 13)
"c"
```

source

AbstractAlgebra.number_of_digits – Method.

```
number_of_digits(x::ZZRingElem, b::Integer)
```

Return the number of digits of $x$ in the base $b$ (default is $b = 10$).

**Examples**

```
julia> number_of_digits(ZZ(12), 3)
3
```

source

Nemo.nbits – Method.

```
nbits(x::ZZRingElem)
```

Return the number of binary bits of $x$. We return zero if $x = 0$.

**Examples**

```
julia> nbits(ZZ(12))
4
```

source

### Bit twiddling

Nemo.popcount – Method.

```
popcount(x::ZZRingElem)
```

Return the number of ones in the binary representation of $x$.

**Examples**

```
julia> popcount(ZZ(12))
2
```

source

Nemo.prevpow2 – Method.

```
prevpow2(x::ZZRingElem)
```

Return the previous power of $2$ up to including $x$.

source

Nemo.nextpow2 – Method.

```
nextpow2(x::ZZRingElem)
```

Return the next power of $2$ that is at least $x$.

**Examples**

```
julia> nextpow2(ZZ(12))
16
```

source

Base.trailing_zeros – Method.

```
trailing_zeros(x::ZZRingElem)
```

Return the number of trailing zeros in the binary representation of $x$.

source

Nemo.clrbit! – Method.

```
clrbit!(x::ZZRingElem, c::Int)
```

Clear bit $c$ of $x$, where the least significant bit is the $0$-th bit. Note that this function modifies its input in-place.

**Examples**

```
julia> a = ZZ(12)
12

julia> clrbit!(a, 3)

julia> a
4
```

source

Nemo.setbit! – Method.

```
setbit!(x::ZZRingElem, c::Int)
```

Set bit $c$ of $x$, where the least significant bit is the $0$-th bit. Note that this function modifies its input in-place.

**Examples**

```
julia> a = ZZ(12)
12

julia> setbit!(a, 0)

julia> a
13
```

source

Nemo.combit! – Method.

```
combit!(x::ZZRingElem, c::Int)
```

Complement bit $c$ of $x$, where the least significant bit is the $0$-th bit. Note that this function modifies its input in-place.

**Examples**

```
julia> a = ZZ(12)
12

julia> combit!(a, 2)

julia> a
8
```

source

Nemo.tstbit – Method.

```
tstbit(x::ZZRingElem, c::Int)
```

Return bit $i$ of x (numbered from 0) as `true` for 1 or `false` for 0.

**Examples**

```
julia> a = ZZ(12)
12

julia> tstbit(a, 0)
false

julia> tstbit(a, 2)
true
```

source

### Random generation

Nemo.rand_bits – Method.

```
rand_bits(::ZZRing, b::Int)
```

Return a random signed integer whose absolute value has $b$ bits.

source

Nemo.rand_bits_prime – Method.

```
rand_bits_prime(::ZZRing, n::Int, proved::Bool=true)
```

Return a random prime number with the given number of bits. If only a probable prime is required, one can pass `proved=false`.

source

**Examples**

```
a = rand_bits(ZZ, 23)
b = rand_bits_prime(ZZ, 7)
```

# Chapter 11

# Complex Integers

The Gaussian integer type in Nemo is provided by a pair of FLINT integers. The associated ring of integers and the fraction field can be retrieved by `Nemo.GaussianIntegers()` and `Nemo.GaussianRationals()`.

**Examples**

```
julia> ZZi = Nemo.GaussianIntegers()
Gaussian integer ring

julia> a = ZZ(5)*im
5*im

julia> b = ZZi(3, 4)
3 + 4*im

julia> is_unit(a)
false

julia> factor(a)
im * (2 - im) * (2 + im)

julia> a//b
4//5 + 3//5*im

julia> abs2(a//b)
1
```

# Chapter 12

# Univariate polynomials

## 12.1 Introduction

Nemo allow the creation of dense, univariate polynomials over any computable ring $R$. There are two different kinds of implementation: a generic one for the case where no specific implementation exists (provided by AbstractAlgebra.jl), and efficient implementations of polynomials over numerous specific rings provided by FLINT.

The following table shows each of the polynomial types available in Nemo, the base ring $R$, and the Julia/Nemo types for that kind of polynomial (the type information is mainly of concern to developers).

| Base ring | Library | Element type | Parent type |
|---:|---:|---:|---:|
| Generic ring $R$ | AbstractAlgebra.jl | `Generic.Poly{T}` | `Generic.PolyRing{T}` |
| $\mathbb{Z}$ | FLINT | `ZZPolyRingElem` | `ZZPolyRing` |
| $\mathbb{Z}/n\mathbb{Z}$ (small $n$) | FLINT | `zzModPolyRingElem` | `zzModPolyRing` |
| $\mathbb{Z}/n\mathbb{Z}$ (large $n$) | FLINT | `ZZModPolyRingElem` | `ZZModPolyRing` |
| $\mathbb{Q}$ | FLINT | `QQPolyRingElem` | `QQPolyRing` |
| $\mathbb{Z}/p\mathbb{Z}$ (small prime $p$) | FLINT | `fpPolyRingElem` | `fpPolyRing` |
| $\mathbb{Z}/p\mathbb{Z}$ (large prime $p$) | FLINT | `FpPolyRingElem` | `FpPolyRing` |
| $\mathbb{F}_{p^n}$ (small $p$) | FLINT | `fqPolyRepPolyRingElem` | `fqPolyRepPolyRing` |
| $\mathbb{F}_{p^n}$ (large $p$) | FLINT | `FqPolyRepPolyRingElem` | `FqPolyRepPolyRing` |
| $\mathbb{R}$ (arbitrary precision) | FLINT | `RealPolyRingElem` | `RealPolyRing` |
| $\mathbb{C}$ (arbitrary precision) | FLINT | `ComplexPolyRingElem` | `ComplexPolyRing` |
| $\mathbb{R}$ (fixed precision) | FLINT | `ArbPolyRingElem` | `ArbPolyRing` |
| $\mathbb{C}$ (fixed precision) | FLINT | `AcbPolyRingElem` | `AcbPolyRing` |

The string representation of the variable and the base ring $R$ of a generic polynomial is stored in its parent object.

All polynomial element types belong to the abstract type `PolyRingElem` and all of the polynomial ring types belong to the abstract type `PolyRing`. This enables one to write generic functions that can accept any Nemo univariate polynomial type.

## 12.2 Polynomial functionality

All univariate polynomial types in Nemo provide the AbstractAlgebra univariate polynomial functionality:

https://nemocas.github.io/AbstractAlgebra.jl/stable/polynomial

Generic polynomials are also available.

We describe here only functions that are in addition to that guaranteed by AbstractAlgebra.jl, for specific coefficient rings.

### Remove and valuation

Nemo.evaluate2 – Method.

```
evaluate2(x::RealPolyRingElem, y::RingElement)
```

Return a tuple $p, q$ consisting of the polynomial $x$ evaluated at $y$ and its derivative evaluated at $y$.

source

Nemo.evaluate2 – Method.

```
evaluate2(x::ComplexPolyRingElem, y::RingElement; prec::Int = precision(Balls))
```

Return a tuple $p, q$ consisting of the polynomial $x$ evaluated at $y$ and its derivative evaluated at $y$.

source

**Examples**

```
julia> RR = real_field()
Real field

julia> T, z = polynomial_ring(RR, "z")
(Univariate polynomial ring in z over RR, z)

julia> h = z^2 + 2z + 1
z^2 + 2.000000000000000000*z + 1

julia> s, t = evaluate2(h, RR("2.0 +/- 0.1"))
([9e+0 +/- 0.611], [6e+0 +/- 0.201])
```

### Signature

Nemo.signature – Method.

```
signature(f::ZZPolyRingElem)
```

Return the signature of $f$, i.e. a tuple $(r, s)$ such that $r$ is the number of real roots of $f$ and $s$ is half the number of complex roots.

**Examples**

```
julia> R, x = polynomial_ring(ZZ, "x");

julia> signature(x^3 + 3x + 1)
(1, 1)
```

source

Nemo.signature – Method.

```
signature(f::QQPolyRingElem)
```

Return the signature of $f$, i.e. a tuple $(r, s)$ such that $r$ is the number of real roots of $f$ and $s$ is half the number of complex roots.

**Examples**

```
julia> R, x = polynomial_ring(QQ, "x");

julia> signature(x^3 + 3x + 1)
(1, 1)
```

source

### Root finding

roots_with_multiplicities is documented in AbstractAlgebra, see https://nemocas.github.io/AbstractAlgebra.jl/stable/polynomial. For zzModPolyRingElem, fpPolyRingElem, ZZModPolyRingElem, FpPolyRingElem, FqPolyRingElem, FqPolyRepPolyRingElem and fqPolyRepPolyRingElem the multiplicities are those FLINT computes while finding the roots. Over Z/nZ for composite n, roots and roots_with_multiplicities throw an error when the roots are too many to list.

AbstractAlgebra.Generic.roots – Method.

```
roots(x::ComplexPolyRingElem; target=0, isolate_real=false, initial_prec=0, max_prec=0,
↪  max_iter=0)
```

Attempts to isolate the complex roots of the complex polynomial $x$ by iteratively refining balls in which they lie.

This is done by increasing the working precision, starting at initial_prec. The maximal number of iterations can be set using max_iter and the maximal precision can be set using max_prec.

If isolate_real is set and $x$ is strictly real, then the real roots will be isolated from the non-real roots. Every root will have either zero, positive or negative real part.

It is assumed that $x$ is squarefree.

source

**Examples**

```
julia> CC = complex_field()
Complex field

julia> C, y = polynomial_ring(CC, "y")
(Univariate polynomial ring in y over CC, y)

julia> m = y^2 + 2y + 3
y^2 + 2.0000000000000000000*y + 3.0000000000000000000

julia> n = m + CC("0 +/- 0.0001", "0 +/- 0.0001")
y^2 + 2.0000000000000000000*y + [3.000 +/- 1.01e-4] + [+/- 1.01e-4]*im

julia> r = roots(n);

julia> sort(r; by=x->(real(x), imag(x))) # sort roots to make printing consistent
2-element Vector{ComplexFieldElem}:
 [-1.000 +/- 2.01e-4] + [-1.414 +/- 4.14e-4]*im
 [-1.000 +/- 2.01e-4] + [1.414 +/- 4.14e-4]*im

julia> p = y^7 - 1
y^7 - 1.0000000000000000000

julia> r = roots(n, isolate_real = true);

julia> sort(r; by=x->(real(x), imag(x))) # sort roots to make printing consistent
2-element Vector{ComplexFieldElem}:
 [-1.000 +/- 2.01e-4] + [-1.414 +/- 4.14e-4]*im
 [-1.000 +/- 2.01e-4] + [1.414 +/- 4.14e-4]*im
```

### Construction from roots

Nemo.from_roots – Method.

```
from_roots(R::ArbPolyRing, b::Vector{ArbFieldElem})
```

Construct a polynomial in the given polynomial ring from a list of its roots.

source

Nemo.from_roots – Method.

```
from_roots(R::AcbPolyRing, b::Vector{AcbFieldElem})
```

Construct a polynomial in the given polynomial ring from a list of its roots.

source

Nemo.from_roots – Method.

```
from_roots(R::ZZPolyRing, rs::Vector{ZZRingElem})
from_roots(R::zzModPolyRing, rs::Vector{zzModRingElem})
from_roots(R::fpPolyRing, rs::Vector{fpFieldElem})
from_roots(R::ZZModPolyRing, rs::Vector{ZZModRingElem})
from_roots(R::FpPolyRing, rs::Vector{FpFieldElem})
```

Return the monic polynomial $\prod_i (x - r_i)$ in R, where the $r_i$ are the entries of rs, a repeated root counted as often as it occurs; one(R) if rs is empty. The entries must lie in the base ring of R.

**Examples**

```
julia> S, = residue_ring(ZZ, 12);

julia> R, x = polynomial_ring(S, :x);

julia> from_roots(R, S.([1, 5, 5]))
x^3 + x^2 + 11*x + 11
```

source

**Examples**

```
julia> RR = real_field()
Real field

julia> R, x = polynomial_ring(RR, "x")
(Univariate polynomial ring in x over RR, x)

julia> xs = [inv(RR(i)) for i=1:5]
5-element Vector{RealFieldElem}:
 1.0000000000000000000
 0.5000000000000000000
 [0.333333333333333333 +/- 4.24e-20]
 0.2500000000000000000
 [0.200000000000000000 +/- 2.44e-20]

julia> f = from_roots(R, xs)
x^5 + [-2.283333333333333333 +/- 4.54e-19]*x^4 + [1.875000000000000000 +/- 5.10e-19]*x^3 +
↪  [-0.708333333333333333 +/- 3.99e-19]*x^2 + [0.125000000000000000 +/- 3.69e-20]*x +
↪  [-0.008333333333333333 +/- 4.13e-21]

julia> all(x -> contains_zero(evaluate(f, x)), xs)
true
```

### Bounding absolute values of roots

Nemo.roots_upper_bound – Method.

```
roots_upper_bound(x::RealPolyRingElem) -> RealFieldElem
```

Returns an upper bound for the absolute value of all complex roots of $x$.

source

Nemo.roots_upper_bound – Method.

```
roots_upper_bound(x::ComplexPolyRingElem) -> RealFieldElem
```

Returns an upper bound for the absolute value of all complex roots of $x$.

source

### Lifting

When working over a residue ring it is useful to be able to lift to the base ring of the residue ring, e.g. from $\mathbb{Z}/n\mathbb{Z}$ to $\mathbb{Z}$.

AbstractAlgebra.lift – Method.

```
lift(R::ZZPolyRing, y::zzModPolyRingElem)
```

Lift from a polynomial over $\mathbb{Z}/n\mathbb{Z}$ to a polynomial over $\mathbb{Z}$ with minimal reduced non-negative coefficients. The ring R specifies the ring to lift into.

source

AbstractAlgebra.lift – Method.

```
lift(R::ZZPolyRing, y::fpPolyRingElem)
```

Lift from a polynomial over $\mathbb{Z}/n\mathbb{Z}$ to a polynomial over $\mathbb{Z}$ with minimal reduced non-negative coefficients. The ring R specifies the ring to lift into.

source

AbstractAlgebra.lift – Method.

```
lift(R::ZZPolyRing, y::ZZModPolyRingElem)
```

Lift from a polynomial over $\mathbb{Z}/n\mathbb{Z}$ to a polynomial over $\mathbb{Z}$ with minimal reduced non-negative coefficients. The ring R specifies the ring to lift into.

source

AbstractAlgebra.lift – Method.

```
lift(R::ZZPolyRing, y::FpPolyRingElem)
```

Lift from a polynomial over $\mathbb{Z}/n\mathbb{Z}$ to a polynomial over $\mathbb{Z}$ with minimal reduced non-negative coefficients. The ring R specifies the ring to lift into.

source

**Examples**

```
julia> R, = residue_ring(ZZ, 123456789012345678949)
(Integers modulo 123456789012345678949, Map: ZZ -> ZZ/(123456789012345678949))

julia> S, x = polynomial_ring(R, "x")
(Univariate polynomial ring in x over ZZ/(123456789012345678949), x)

julia> T, y = polynomial_ring(ZZ, "y")
(Univariate polynomial ring in y over ZZ, y)

julia> f = x^2 + 2x + 1
x^2 + 2*x + 1

julia> a = lift(T, f)
y^2 + 2*y + 1
```

### Overlapping and containment

Occasionally it is useful to be able to tell when inexact polynomials overlap or contain other exact or inexact polynomials. The following functions are provided for this purpose.

Nemo.overlaps – Method.

```
overlaps(x::RealPolyRingElem, y::RealPolyRingElem)
```

Return `true` if the coefficient balls of $x$ overlap the coefficient balls of $y$, otherwise return `false`.

source

Nemo.overlaps – Method.

```
overlaps(x::ComplexPolyRingElem, y::ComplexPolyRingElem)
```

Return `true` if the coefficient boxes of $x$ overlap the coefficient boxes of $y$, otherwise return `false`.

source

Base.contains – Method.

```
contains(x::RealPolyRingElem, y::RealPolyRingElem)
```

Return `true` if the coefficient balls of $x$ contain the corresponding coefficient balls of $y$, otherwise return `false`.

source

Base.contains – Method.

```
contains(x::ComplexPolyRingElem, y::ComplexPolyRingElem)
```

Return true if the coefficient boxes of $x$ contain the corresponding coefficient boxes of $y$, otherwise return false.

source

Base.contains – Method.

```
contains(x::RealPolyRingElem, y::ZZPolyRingElem)
```

Return true if the coefficient balls of $x$ contain the corresponding exact coefficients of $y$, otherwise return false.

source

Base.contains – Method.

```
contains(x::RealPolyRingElem, y::QQPolyRingElem)
```

Return true if the coefficient balls of $x$ contain the corresponding exact coefficients of $y$, otherwise return false.

source

Base.contains – Method.

```
contains(x::ComplexPolyRingElem, y::ZZPolyRingElem)
```

Return true if the coefficient boxes of $x$ contain the corresponding exact coefficients of $y$, otherwise return false.

source

Base.contains – Method.

```
contains(x::ComplexPolyRingElem, y::QQPolyRingElem)
```

Return true if the coefficient boxes of $x$ contain the corresponding exact coefficients of $y$, otherwise return false.

source

It is sometimes also useful to be able to determine if there is a unique integer contained in the coefficient of an inexact constant polynomial.

Nemo.unique_integer – Method.

```
unique_integer(x::RealPolyRingElem)
```

Return a tuple `(t, z)` where $t$ is `true` if there is a unique integer contained in each of the coefficients of $x$, otherwise sets $t$ to `false`. In the former case, $z$ is set to the integer polynomial.

source

Nemo.unique_integer – Method.

```
unique_integer(x::ComplexPolyRingElem)
```

Return a tuple `(t, z)` where $t$ is `true` if there is a unique integer contained in the (constant) polynomial $x$, along with that integer $z$ in case it is, otherwise sets $t$ to `false`.

source

**Examples**

```
julia> RR = real_field()
Real field

julia> R, x = polynomial_ring(RR, "x")
(Univariate polynomial ring in x over RR, x)

julia> f = x^2 + 2x + 1
x^2 + 2.0000000000000000000*x + 1

julia> h = f + RR("0 +/- 0.0001")
x^2 + 2.0000000000000000000*x + [1.000 +/- 1.01e-4]

julia> k = f + RR("0 +/- 0.0001") * x^4
[+/- 1.01e-4]*x^4 + x^2 + 2.0000000000000000000*x + 1

julia> contains(h, f)
true

julia> overlaps(f, k)
true

julia> t, z = unique_integer(k)
(true, x^2 + 2*x + 1)
```

```
julia> CC = complex_field()
Complex field

julia> C, y = polynomial_ring(CC, "y")
(Univariate polynomial ring in y over CC, y)

julia> m = y^2 + 2y + 1
y^2 + 2.0000000000000000000*y + 1
```

```
julia> n = m + CC("0 +/- 0.0001", "0 +/- 0.0001")
y^2 + 2.0000000000000000000*y + [1.000 +/- 1.01e-4] + [+/- 1.01e-4]*im

julia> contains(n, m)
true

julia> isreal(n)
false

julia> isreal(m)
true
```

### Factorisation

Certain polynomials can be factored (ZZPolyRingElem', zzModPolyRingElem, fpPolyRingElem, ZZModPolyRingElem, FpPolyRingEl and the interface follows the specification in AbstractAlgebra.jl. The following additional functions are available.

Nemo.factor_distinct_deg – Method.

```
factor_distinct_deg(x::zzModPolyRingElem)
```

Return the distinct degree factorisation of a squarefree polynomial $x$.

source

Nemo.factor_distinct_deg – Method.

```
factor_distinct_deg(x::fpPolyRingElem)
```

Return the distinct degree factorisation of a squarefree polynomial $x$.

source

Nemo.factor_distinct_deg – Method.

```
factor_distinct_deg(x::ZZModPolyRingElem)
```

Return the distinct degree factorisation of a squarefree polynomial $x$.

source

Nemo.factor_distinct_deg – Method.

```
factor_distinct_deg(x::ZZModPolyRingElem)
```

Return the distinct degree factorisation of a squarefree polynomial $x$.

source

Nemo.factor_distinct_deg – Method.

```
factor_distinct_deg(x::FqPolyRepPolyRingElem)
```

Return the distinct degree factorisation of a squarefree polynomial $x$.

source

Nemo.factor_distinct_deg – Method.

```
factor_distinct_deg(x::fqPolyRepPolyRingElem)
```

Return the distinct degree factorisation of a squarefree polynomial $x$.

source

**Examples**

```
julia> R, = residue_ring(ZZ, 23)
(Integers modulo 23, Map: ZZ -> ZZ/(23))

julia> S, x = polynomial_ring(R, "x")
(Univariate polynomial ring in x over ZZ/(23), x)

julia> f = x^2 + 2x + 1
x^2 + 2*x + 1

julia> g = x^3 + 3x + 1
x^3 + 3*x + 1

julia> R = factor(f*g)
(x + 1)^2 * (x^3 + 3*x + 1)

julia> S = factor_squarefree(f*g)
(x + 1)^2 * (x^3 + 3*x + 1)

julia> T = factor_distinct_deg((x + 1)*g*(x^5+x^3+x+1))
Dict{Int64, zzModPolyRingElem} with 3 entries:
  4 => x^4 + 7*x^3 + 4*x^2 + 5*x + 13
  3 => x^3 + 3*x + 1
  1 => x^2 + 17*x + 16
```

### Special functions

Nemo.cyclotomic – Method.

```
cyclotomic(n::Int, x::ZZPolyRingElem)
```

Return the $n$th cyclotomic polynomial, defined as $\Phi_n(x) = \prod_\omega (x - \omega)$, where $\omega$ runs over all the $n$th primitive roots of unity.

source

Nemo.swinnerton_dyer – Method.

```
swinnerton_dyer(n::Int, x::ZZPolyRingElem)
```

Return the Swinnerton-Dyer polynomial $S_n$, defined as the integer polynomial $S_n = \prod(x \pm \sqrt{2} \pm \sqrt{3} \pm \sqrt{5} \pm \ldots \pm \sqrt{p_n})$ where $p_n$ denotes the $n$-th prime number and all combinations of signs are taken. This polynomial has degree $2^n$ and is irreducible over the integers (it is the minimal polynomial of $\sqrt{2} + \ldots + \sqrt{p_n}$).

source

Nemo.cos_minpoly – Method.

```
cos_minpoly(n::Int, x::ZZPolyRingElem)
```

Return the minimal polynomial of $2\cos(2\pi/n)$. For suitable choice of $n$, this gives the minimal polynomial of $2\cos(a\pi)$ or $2\sin(a\pi)$ for any rational $a$.

source

Nemo.theta_qexp – Method.

```
theta_qexp(e::Int, n::Int, x::ZZPolyRingElem)
```

Return the $q$-expansion to length $n$ of the Jacobi theta function raised to the power $r$, i.e. $\vartheta(q)^r$ where $\vartheta(q) = 1 + \sum_{k=1}^{\infty} q^{k^2}$.

source

Nemo.eta_qexp – Method.

```
eta_qexp(e::Int, n::Int, x::ZZPolyRingElem)
```

Return the $q$-expansion to length $n$ of the Dedekind eta function (without the leading factor $q^{1/24}$) raised to the power $r$, i.e. $(q^{-1/24}\eta(q))^r = \prod_{k=1}^{\infty}(1-q^k)^r$. In particular, $r = -1$ gives the generating function of the partition function $p(k)$, and $r = 24$ gives, after multiplication by $q$, the modular discriminant $\Delta(q)$ which generates the Ramanujan tau function $\tau(k)$.

source

**Examples**

```
julia> R, x = polynomial_ring(ZZ, "x")
(Univariate polynomial ring in x over ZZ, x)

julia> h = cyclotomic(120, x)
x^32 + x^28 - x^20 - x^16 - x^12 + x^4 + 1

julia> j = swinnerton_dyer(5, x)
```

```
x^32 - 448*x^30 + 84864*x^28 - 9028096*x^26 + 602397952*x^24 - 26625650688*x^22 + 801918722048*x^20
↪  - 16665641517056*x^18 + 239210760462336*x^16 - 2349014746136576*x^14 + 15459151516270592*x^12 -
↪  65892492886671360*x^10 + 172580952324702208*x^8 - 255690851718529024*x^6 +
↪  183876928237731840*x^4 - 44660812492570624*x^2 + 2000989041197056

julia> k = cos_minpoly(30, x)
x^4 + x^3 - 4*x^2 - 4*x + 1

julia> l = theta_qexp(3, 30, x)
72*x^29 + 32*x^27 + 72*x^26 + 30*x^25 + 24*x^24 + 24*x^22 + 48*x^21 + 24*x^20 + 24*x^19 + 36*x^18 +
↪  48*x^17 + 6*x^16 + 48*x^14 + 24*x^13 + 8*x^12 + 24*x^11 + 24*x^10 + 30*x^9 + 12*x^8 + 24*x^6 +
↪  24*x^5 + 6*x^4 + 8*x^3 + 12*x^2 + 6*x + 1

julia> m = eta_qexp(24, 30, x)
-29211840*x^29 + 128406630*x^28 + 24647168*x^27 - 73279080*x^26 + 13865712*x^25 - 25499225*x^24 +
↪  21288960*x^23 + 18643272*x^22 - 12830688*x^21 - 4219488*x^20 - 7109760*x^19 + 10661420*x^18 +
↪  2727432*x^17 - 6905934*x^16 + 987136*x^15 + 1217160*x^14 + 401856*x^13 - 577738*x^12 -
↪  370944*x^11 + 534612*x^10 - 115920*x^9 - 113643*x^8 + 84480*x^7 - 16744*x^6 - 6048*x^5 +
↪  4830*x^4 - 1472*x^3 + 252*x^2 - 24*x + 1

julia> o = cyclotomic(10, 1 + x + x^2)
x^8 + 4*x^7 + 9*x^6 + 13*x^5 + 14*x^4 + 11*x^3 + 6*x^2 + 2*x + 1
```

# Chapter 13

# Multivariate polynomials

## 13.1 Introduction

Nemo allow the creation of sparse, distributed multivariate polynomials over any computable ring $R$. There are two different kinds of implementation: a generic one for the case where no specific implementation exists (provided by AbstractAlgebra.jl), and efficient implementations of polynomials over numerous specific rings, usually provided by C/C++ libraries.

The following table shows each of the polynomial types available in Nemo, the base ring $R$, and the Julia/Nemo types for that kind of polynomial (the type information is mainly of concern to developers).

| Base ring | Library | Element type | Parent type |
|---:|---:|---:|---:|
| Generic ring $R$ | AbstractAlgebra.jl | `Generic.MPoly{T}` | `Generic.MPolyRing{T}` |
| $\mathbb{Z}$ | FLINT | `ZZMPolyRingElem` | `ZZMPolyRing` |
| $\mathbb{Z}/n\mathbb{Z}$ (small $n$) | FLINT | `zzModMPolyRingElem` | `zzModMPolyRing` |
| $\mathbb{Q}$ | FLINT | `QQMPolyRingElem` | `QQMPolyRing` |
| $\mathbb{Z}/p\mathbb{Z}$ (small prime $p$) | FLINT | `fpMPolyRingElem` | `fpMPolyRing` |
| $\mathbb{F}_{p^n}$ (small $p$) | FLINT | `fqPolyRepMPolyRingElem` | `fqPolyRepMPolyRing` |

The string representation of the variables and the base ring $R$ of a generic polynomial is stored in its parent object.

All polynomial element types belong to the abstract type `MPolyRingElem` and all of the polynomial ring types belong to the abstract type `MPolyRing`. This enables one to write generic functions that can accept any Nemo multivariate polynomial type.

## 13.2 Polynomial functionality

All multivariate polynomial types in Nemo provide the multivariate polynomial functionality described by AbstractAlgebra:

https://nemocas.github.io/AbstractAlgebra.jl/stable/mpolynomial

Generic multivariate polynomials are also available.

We describe here only functions that are in addition to that guaranteed by AbstractAlgebra.jl, for specific coefficient rings.

# Chapter 14

# Power series and Laurent series

Nemo allows the creation of capped relative and absolute power series over any computable ring $R$. Capped relative power series are power series of the form $a_j x^j + a_{j+1}x^{j+1} + \cdots + a_{k-1}x^{k-1} + O(x^k)$ where $j \geq 0$, $a_j \in R$ and the relative precision $k - j$ is at most equal to some specified precision $n$. On the other hand capped absolute power series are power series of the form $a_j x^j + a_{j+1}x^{j+1} + \cdots + a_{n-1}x^{n-1} + O(x^n)$ where $j \geq 0$, $a_j \in R$ and the precision $n$ is fixed.

There are two different kinds of implementation: a generic one for the case where no specific implementation exists (provided by AbstractAlgebra.jl), and efficient implementations of power series over numerous specific rings, usually provided by C/C++ libraries.

The following table shows each of the relative power series types available in Nemo, the base ring $R$, and the Julia/Nemo types for that kind of series (the type information is mainly of concern to developers).

| Base ring | Library | Element type | Parent type |
|---|---|---|---|
| Generic ring $R$ | AbstractAlgebra.jl | ‘Generic.RelSeries{T} | `Generic.RelPowerSeriesRing{T}` |
| $\mathbb{Z}$ | FLINT | `ZZRelPowerSeriesRingElem` | `ZZRelPowerSeriesRing` |
| $\mathbb{Z}/n\mathbb{Z}$ (small $n$) | FLINT | `zzModRelPowerSeriesRingElem` | `zzModRelPowerSeriesRing` |
| $\mathbb{Z}/n\mathbb{Z}$ (large $n$) | FLINT | `ZZModRelPowerSeriesRingElem` | `ZZModRelPowerSeriesRing` |
| $\mathbb{Q}$ | FLINT | `QQRelPowerSeriesRingElem` | `QQRelPowerSeriesRing` |
| $\mathbb{F}_p$ (small $n$) | FLINT | `fpRelPowerSeriesRingElem` | `fpRelPowerSeriesRing` |
| $\mathbb{F}_p$ (large $n$) | FLINT | `FpRelPowerSeriesRingElem` | `FpRelPowerSeriesRing` |
| $\mathbb{F}_{p^n}$ (small $p$) | FLINT | `fqPolyRepRelPowerSeriesRingElem` | `fqPolyRepRelPowerSeriesRing` |
| $\mathbb{F}_{p^n}$ (large $p$) | FLINT | `FqPolyRepRelPowerSeriesRingElem` | `FqPolyRepRelPowerSeriesRing` |

All relative power series elements belong to the abstract type `RelPowerSeriesRingElem` and all of the relative power series ring types belong to the abstract type `RelPowerSeriesRing`.

The maximum relative precision, the string representation of the variable and the base ring $R$ of a generic power series are stored in its parent object.

Here is the corresponding table for the absolute power series types.

All absolute power series elements belong to the abstract type `AbsPowerSeriesRingElem` and all of the absolute power series ring types belong to the abstract type `AbsPowerSeriesRing`.

The absolute precision, the string representation of the variable and the base ring $R$ of a generic power series are stored in its parent object.

| Base ring | Library | Element type | Parent type |
|---|---|---|---|
| Generic ring $R$ | AbstractAlgebra.jl | `Generic.AbsSeries{T}` | `Generic.AbsPowerSeriesRing{T}` |
| $\mathbb{Z}$ | FLINT | `ZZAbsPowerSeriesRingElem` | `ZZAbsPowerSeriesRing` |
| $\mathbb{Z}/n\mathbb{Z}$ (small $n$) | FLINT | `zzModAbsPowerSeriesRingElem` | `zzModAbsPowerSeriesRing` |
| $\mathbb{Z}/n\mathbb{Z}$ (large $n$) | FLINT | `ZZModAbsPowerSeriesRingElem` | `ZZModAbsPowerSeriesRing` |
| $\mathbb{Q}$ | FLINT | `QQAbsPowerSeriesRingElem` | `QQAbsPowerSeriesRing` |
| $\mathbb{F}_p$ (small $n$) | FLINT | `fpAbsPowerSeriesRingElem` | `fpAbsPowerSeriesRing` |
| $\mathbb{F}_p$ (large $n$) | FLINT | `FpAbsPowerSeriesRingElem` | `FpAbsPowerSeriesRing` |
| $\mathbb{F}_{p^n}$ (small $n$) | FLINT | `fqPolyRepAbsPowerSeriesRingElem` | `fqPolyRepAbsPowerSeriesRing` |
| $\mathbb{F}_{p^n}$ (large $n$) | FLINT | `FqPolyRepAbsPowerSeriesRingElem` | `FqPolyRepAbsPowerSeriesRing` |

All power series element types belong to the abstract type `SeriesElem` and all of the power series ring types belong to the abstract type `SeriesRing`. This enables one to write generic functions that can accept any Nemo power series type.

AbstractAlgebra.jl also provides Nemo with a generic implementation of Laurent series over a given ring $R$. For completeness, we list it here.

| Base ring | Library | Element type | Parent type |
|---|---|---|---|
| Generic ring $R$ | AbstractAlgebra.jl | `Generic.LaurentSeriesRingElem{T}` | `Generic.LaurentSeriesRing{T}` |
| Generic field $K$ | AbstractAlgebra.jl | `Generic.LaurentSeriesFieldElem{T}` | `Generic.LaurentSeriesField{T}` |

## 14.1 Capped relative power series

Capped relative power series have their maximum relative precision capped at some value `prec_max`. This means that if the leading term of a nonzero power series element is $c_a x^a$ and the precision is $b$ then the power series is of the form $c_a x^a + c_{a+1}x^{a+1} + \ldots + O(x^{a+b})$.

The zero power series is simply taken to be $0 + O(x^b)$.

The capped relative model has the advantage that power series are stable multiplicatively. In other words, for nonzero power series $f$ and $g$ we have that `divexact(f*g), g) == f`.

However, capped relative power series are not additively stable, i.e. we do not always have $(f + g) - g = f$.

In the capped relative model we say that two power series are equal if they agree up to the minimum *absolute* precision of the two power series. Thus, for example, $x^5 + O(x^{10}) == 0 + O(x^5)$, since the minimum absolute precision is $5$.

During computations, it is possible for power series to lose relative precision due to cancellation. For example if $f = x^3 + x^5 + O(x^8)$ and $g = x^3 + x^6 + O(x^8)$ then $f - g = x^5 - x^6 + O(x^8)$ which now has relative precision $3$ instead of relative precision $5$.

Amongst other things, this means that equality is not transitive. For example $x^6 + O(x^{11}) == 0 + O(x^5)$ and $x^7 + O(x^{12}) == 0 + O(x^5)$ but $x^6 + O(x^{11}) \neq x^7 + O(x^{12})$.

Sometimes it is necessary to compare power series not just for arithmetic equality, as above, but to see if they have precisely the same precision and terms. For this purpose we introduce the `isequal` function.

For example, if $f = x^2 + O(x^7)$ and $g = x^2 + O(x^8)$ and $h = 0 + O(x^2)$ then $f == g$, $f == h$ and $g == h$, but `isequal(f, g)`, `isequal(f, h)` and `isequal(g, h)` would all return `false`. However, if $k = x^2 + O(x^7)$ then `isequal(f, k)` would return `true`.

There are further difficulties if we construct polynomial over power series. For example, consider the polynomial in $y$ over the power series ring in $x$ over the rationals. Normalisation of such polynomials is problematic. For instance, what is the leading coefficient of $(0 + O(x^{10}))y + (1 + O(x^{10}))$?

If one takes it to be $(0 + O(x^{10}))$ then some functions may not terminate due to the fact that algorithms may require the degree of polynomials to decrease with each iteration. Instead, the degree may remain constant and simply accumulate leading terms which are arithmetically zero but not identically zero.

On the other hand, when constructing power series over other power series, if we simply throw away terms which are arithmetically equal to zero, our computations may have different output depending on the order in which the power series are added!

One should be aware of these difficulties when working with power series. Power series, as represented on a computer, simply don't satisfy the axioms of a ring. They must be used with care in order to approximate operations in a mathematical power series ring.

Simply increasing the precision will not necessarily give a "more correct" answer and some computations may not even terminate due to the presence of arithmetic zeroes!

## 14.2 Capped absolute power series

An absolute power series ring over a ring $R$ with precision $p$ behaves very much like the quotient $R[x]/(x^p)$ of the polynomial ring over $R$.

## 14.3 Power series functionality

Power series rings in Nemo provide all the functionality described for power series in AbstractAlgebra:

https://nemocas.github.io/AbstractAlgebra.jl/stable/series

In addition, generic power series and Laurent series are provided by AbstractAlgebra.

We list below only the functionality that is Nemo specific for power series rings.

### Special functions

#### Examples

```
julia> T, z = power_series_ring(QQ, 30, "z")
(Univariate power series ring over QQ, z + O(z^31))

julia> a = 1 + z + 3z^2 + O(z^5)
1 + z + 3*z^2 + O(z^5)

julia> b = z + 2z^2 + 5z^3 + O(z^5)
z + 2*z^2 + 5*z^3 + O(z^5)

julia> d = divexact(z, exp(z + O(z^40)) - 1)
1 - 1//2*z + 1//12*z^2 - 1//720*z^4 + 1//30240*z^6 - 1//1209600*z^8 + 1//47900160*z^10 -
↪  691//1307674368000*z^12 + 1//74724249600*z^14 - 3617//10670622842880000*z^16 +
↪  43867//5109094217170944000*z^18 - 174611//802857662698291200000*z^20 +
↪  77683//141011000393918054400000*z^22 - 236364091//1693824136731743669452800000*z^24 +
↪  657931//18613452051997183180800000*z^26 - 3392780147//37893265687455865519472640000000*z^28 +
↪  O(z^29)
```

```
julia> f = exp(b)
1 + z + 5//2*z^2 + 43//6*z^3 + 193//24*z^4 + O(z^5)

julia> g = log(a)
z + 5//2*z^2 - 8//3*z^3 - 7//4*z^4 + O(z^5)

julia> h = sqrt(a)
1 + 1//2*z + 11//8*z^2 - 11//16*z^3 - 77//128*z^4 + O(z^5)

julia> k = sin(b)
z + 2*z^2 + 29//6*z^3 - z^4 + O(z^5)

julia> m = atanh(b)
z + 2*z^2 + 16//3*z^3 + 2*z^4 + O(z^5)
```

# Chapter 15

# Puiseux series

Nemo allows the creation of Puiseux series over any computable ring $R$. Puiseux series are series of the form $a_j x^{j/m} + a_{j+1}x^{(j+1)/m} + \cdots + a_{k-1}x^{(k-1)/m} + O(x^{k/m})$ where $m$ is a positive integer, $a_i \in R$ and the relative precision $k - j$ is at most equal to some specified precision $n$.

There are two different kinds of implementation: a generic one for the case where no specific implementation exists (provided by AbstractAlgebra.jl), and efficient implementations of Puiseux series over numerous specific rings, usually provided by C/C++ libraries.

The following table shows each of the Puiseux series types available in Nemo, the base ring $R$, and the Julia/Nemo types for that kind of series (the type information is mainly of concern to developers).

| Base ring | Library | Element type | Parent type |
|---|---|---|---|
| Generic ring $R$ | AbstractAlgebra.jl | `Generic.PuiseuxSeriesRingElem{T}` | `Generic.PuiseuxSeriesRing{T}` |
| Generic field $K$ | AbstractAlgebra.jl | `Generic.PuiseuxSeriesFieldElem{T}` | `Generic.PuiseuxSeriesField{T}` |
| $\mathbb{Z}$ | FLINT | `FlintPuiseuxSeriesRingElem{ZZLaurentSeriesRingElem}` | `FlintPuiseuxSeriesRing{ZZLaurentSeriesRingElem}` |

For convenience, `FlintPuiseuxSeriesRingElem` and `FlintPuiseuxSeriesFieldElem` both belong to a union type called `FlintPuiseuxSeriesElem`.

The maximum relative precision, the string representation of the variable and the base ring $R$ of a generic power series are stored in the parent object.

Note that unlike most other Nemo types, Puiseux series are parameterised by the type of the underlying Laurent series type (which must exist before Nemo can make use of it), instead of the type of the coefficients.

## 15.1 Puiseux power series

Puiseux series have their maximum relative precision capped at some value `prec_max`. This refers to the maximum precision of the underlying Laurent series. See the description of the generic Puiseux series in AbstractAlgebra.jl for details.

There are numerous important things to be aware of when working with Puiseux series, or series in general. Please refer to the documentation of generic Puiseux series and series in general in AbstractAlgebra.jl for details.

## 15.2 Puiseux series functionality

Puiseux series rings in Nemo implement all the same functionality that is available for AbstractAlgebra series rings, with the exception of the `pol_length` and `polcoeff` functions:

https://nemocas.github.io/AbstractAlgebra.jl/stable/series

In addition, generic Puiseux series are provided by AbstractAlgebra.jl

We list below only the functionality that differs from that described in AbstractAlgebra, for specific rings provided by Nemo.

### Special functions

`AbstractAlgebra.is_square` – Method.

```
is_square(a::NCRingElement)
```

Return `true` iff a is the square of a value in its own ring. See also `is_square(A::MatElem)` which tests whether a matrix has square shape.

source

`Base.exp` – Method.

```
exp(a::AbsPowerSeriesRingElem)
```

Return the exponential of the power series $a$.

source

```
exp(a::RelPowerSeriesRingElem)
```

Return the exponential of the power series $a$.

source

```
exp(a::Generic.LaurentSeriesElem)
```

Return the exponential of the power series $a$.

source

```
exp(a::Generic.PuiseuxSeriesElem{T}) where T <: RingElement
```

Return the exponential of the given Puiseux series $a$.

**Examples**

```
julia> T, z = puiseux_series_ring(QQ, 30, :z)
(Puiseux series field in z over rationals, z + O(z^31))

julia> exp(z + 2z^2 + 5z^3 + O(z^5))
1 + z + 5//2*z^2 + 43//6*z^3 + 193//24*z^4 + O(z^5)
```

source

Nemo.eta_qexp – Method.

```
eta_qexp(x::FlintPuiseuxSeriesElem{ZZLaurentSeriesRingElem})
```

Return the $q$-series for eta evaluated at $x$, which must currently be a rational power of the generator of the Puiseux series ring.

source

**Examples**

```
julia> S, z = puiseux_series_ring(ZZ, 30, "z")
(Puiseux series ring in z over ZZ, z + O(z^31))

julia> a = 1 + z + 3z^2 + O(z^5)
1 + z + 3*z^2 + O(z^5)

julia> h = sqrt(a^2)
1 + z + 3*z^2 + O(z^5)

julia> k = eta_qexp(z)
z^(1//24) - z^(25//24) + O(z^(31//24))
```

# Chapter 16

# Residue rings

Nemo allows the creation of residue rings of the form $R/(a)$ for an element $a$ of a ring $R$.

We don't require $(a)$ to be a prime or maximal ideal. Instead, we allow the creation of the residue ring $R/(a)$ for any nonzero $a$ and simply raise an exception if an impossible inverse is encountered during computations involving elements of $R/(a)$. Of course, a GCD function must be available for the base ring $R$.

There is a generic implementation of residue rings of this form in AbstractAlgebra.jl, which accepts any ring $R$ as base ring.

The associated types of parent object and elements for each kind of residue rings in Nemo are given in the following table.

| Base ring | Library | Element type | Parent type |
|---|---|---|---|
| Generic ring $R$ | AbstractAlgebra.jl | `EuclideanRingResidueRingElem{T}` | `EuclideanRingResidueRing{T}` |
| $\mathbb{Z}$ (Int modulus) | FLINT | `zzModRingElem` | `zzModRing` |
| $\mathbb{Z}$ (ZZ modulus) | FLINT | `ZZModRingElem` | `ZZModRing` |

The modulus $a$ of a residue ring is stored in its parent object.

All residue element types belong to the abstract type `ResElem` and all the residue ring parent object types belong to the abstract type `ResidueRing`. This enables one to write generic functions that accept any Nemo residue type.

## 16.1 Residue functionality

All the residue rings in Nemo provide the functionality described in AbstractAlgebra for residue rings:

https://nemocas.github.io/AbstractAlgebra.jl/stable/residue

In addition, generic residue rings are available.

We describe Nemo specific residue ring functionality below.

### GCD

`Base.gcdx` – Method.

```
gcdx(a::zzModRingElem, b::zzModRingElem)
```

Compute the extended gcd with the Euclidean structure inherited from $\mathbb{Z}$.

source

Base.gcdx – Method.

```
gcdx(a::ZZModRingElem, b::ZZModRingElem)
```

Compute the extended gcd with the Euclidean structure inherited from $\mathbb{Z}$.

source

**Examples**

```
julia> R, = residue_ring(ZZ, 123456789012345678949);

julia> g, s, t = gcdx(R(123), R(456))
(1, 12345678901234567928, 41152263004115226322)
```

# Part VI

# Fields

# Chapter 17

# Fraction fields

Nemo allows the creation of fraction fields over any ring $R$. We don't require $R$ to be an integral domain, however no attempt is made to deal with the general case. Two fractions $a/b$ and $c/d$ are equal in Nemo iff $ad = bc$. Thus, in practice, a greatest common divisor function is currently required for the ring $R$.

In order to make the representation $a/b$ unique for printing, we have a notion of canonical unit for elements of a ring $R$. When canonicalising $a/b$, each of the elements $a$ and $b$ is first divided by the canonical unit of $b$.

The `canonical_unit` function is defined for elements of every Nemo ring. It must have the properties

```
canonical_unit(u) == u
canonical_unit(a*b) == canonical_unit(a)*canonical_unit(b)
```

for any unit $u$ of the ring in question, and $a$ and $b$ arbitrary elements of the ring.

For example, the canonical unit of an integer is its sign. Thus a fraction of integers always has positive denominator after canonicalisation.

The canonical unit of a polynomial is the canonical unit of its leading coefficient, etc.

There are two different kinds of implementation of fraction fields in Nemo: a generic one for the case where no specific implementation exists (provided by AbstractAlgebra.jl), and efficient implementations of fractions over specific rings, usually provided by C/C++ libraries.

The following table shows each of the fraction types available in Nemo, the base ring $R$, and the Julia/Nemo types for that kind of fraction (the type information is mainly of concern to developers).

| Base ring | Library | Element type | Parent type |
|---|---|---|---|
| Generic ring $R$ | AbstractAlgebra.jl | `Generic.FracFieldElem{T}` | `Generic.FracField{T}` |
| $\mathbb{Z}$ | FLINT | `QQFieldElem` | `QQField` |

All fraction element types belong to the abstract type `FracElem` and all of the fraction field types belong to the abstract type `FracField`. This enables one to write generic functions that can accept any Nemo fraction type.

## 17.1 Fraction functionality

All fraction types in Nemo provide functionality for fields described in AbstractAlgebra.jl:

https://nemocas.github.io/AbstractAlgebra.jl/stable/field

In addition all the fraction field functionality of AbstractAlgebra.jl is provided, along with generic fractions fields as described here:

https://nemocas.github.io/AbstractAlgebra.jl/stable/fraction

### Basic manipulation

`Base.sign` – Method.

```
sign(a::QQFieldElem)
```

Return the sign of $a$ ($-1$, $0$ or $1$) as a fraction.

source

`Nemo.height` – Method.

```
height(a::QQFieldElem)
```

Return the height of the fraction $a$, namely the largest of the absolute values of the numerator and denominator.

source

`Nemo.height_bits` – Method.

```
height_bits(a::QQFieldElem)
```

Return the number of bits of the height of the fraction $a$.

source

`Base.:<<` – Method.

```
<<(a::QQFieldElem, b::Int)
```

Return $a \times 2^b$.

source

`Base.:>>` – Method.

```
>>(a::QQFieldElem, b::Int)
```

Return $a/2^b$.

source

`Base.floor` – Method.

```
floor(a::QQFieldElem)
```

Return the greatest integer that is less than or equal to $a$. The result is returned as a rational with denominator $1$.

source

Base.ceil – Method.

```
ceil(a::QQFieldElem)
```

Return the least integer that is greater than or equal to $a$. The result is returned as a rational with denominator $1$.

source

**Examples**

```
julia> d = abs(ZZ(11)//3)
11//3

julia> 4 <= ZZ(7)//ZZ(3)
false
```

### Modular arithmetic

The following functions are available for rationals.

Base.mod – Method.

```
mod(a::QQFieldElem, b::ZZRingElem)
mod(a::QQFieldElem, b::Integer)
```

Return $a \pmod b$ where $b$ is an integer coprime to the denominator of $a$.

**Examples**

```
julia> mod(-ZZ(2)//3, 7)
4

julia> mod(ZZ(1)//2, ZZ(5))
3
```

source

### Rational Reconstruction

Rational reconstruction is available for rational numbers.

`Nemo.reconstruct` – Method.

```
reconstruct(a::ZZRingElem, m::ZZRingElem)
reconstruct(a::ZZRingElem, m::Integer)
reconstruct(a::Integer, m::ZZRingElem)
reconstruct(a::Integer, m::Integer)
```

Attempt to return a rational number $n/d$ such that $0 \leq |n| \leq \lfloor\sqrt{m/2}\rfloor$ and $0 < d \leq \lfloor\sqrt{m/2}\rfloor$ such that $\gcd(n, d) = 1$ and $a \equiv nd^{-1} \pmod{m}$. If no solution exists, an exception is thrown.

**Examples**

```
julia> a = reconstruct(7, 13)
1//2

julia> b = reconstruct(ZZ(15), 31)
-1//2

julia> c = reconstruct(ZZ(123), ZZ(237))
9//2
```

source

`Nemo.reconstruct` – Method.

```
reconstruct(a::ZZRingElem, m::ZZRingElem, N::ZZRingElem, D::ZZRingElem)
```

Attempt to return a rational number $n/d$ such that $0 \leq |n| \leq N$ and $0 < d \leq D$ such that $2ND < m$, $\gcd(n, d) = 1$, and $a \equiv nd^{-1} \pmod{m}$.

Returns a tuple (`success`, `n`/`d`), where `success` signals the success of reconstruction.

The behaviour is undefined if $a$ is negative or larger than $m$.

source

## 17.2 Rational enumeration

Various methods exist to enumerate rationals.

`Nemo.next_minimal` – Method.

```
next_minimal(a::QQFieldElem)
```

Given $a$, return the next rational number in the sequence obtained by enumerating all positive denominators $q$, and for each $q$ enumerating the numerators $1 \leq p < q$ in order and generating both $p/q$ and $q/p$, but skipping all $\gcd(p, q) \neq 1$. Starting with zero, this generates every non-negative rational number

once and only once, with the first few entries being $0, 1, 1/2, 2, 1/3, 3, 2/3, 3/2, 1/4, 4, 3/4, 4/3, \ldots$. This enumeration produces the rational numbers in order of minimal height. It has the disadvantage of being somewhat slower to compute than the Calkin-Wilf enumeration. If $a < 0$ we throw a `DomainError()`.

**Examples**

```
julia> next_minimal(ZZ(2)//3)
3//2
```

source

Nemo.next_signed_minimal – Method.

```
next_signed_minimal(a::QQFieldElem)
```

Given a signed rational number $a$ assumed to be in canonical form, return the next element in the minimal-height sequence generated by `next_minimal` but with negative numbers interleaved. The sequence begins $0, 1, -1, 1/2, -1/2, 2, -2, 1/3, -1/3, \ldots$. Starting with zero, this generates every rational number once and only once, in order of minimal height.

**Examples**

```
julia> next_signed_minimal(-ZZ(21)//31)
31//21
```

source

Nemo.next_calkin_wilf – Method.

```
next_calkin_wilf(a::QQFieldElem)
```

Return the next number after $a$ in the breadth-first traversal of the Calkin-Wilf tree. Starting with zero, this generates every non-negative rational number once and only once, with the first few entries being $0, 1, 1/2, 2, 1/3, 3/2, 2/3, 3, 1/4, 4/3, 3/5, 5/2, 2/5, \ldots$. Despite the appearance of the initial entries, the Calkin-Wilf enumeration does not produce the rational numbers in order of height: some small fractions will appear late in the sequence. This order has the advantage of being faster to produce than the minimal-height order.

**Examples**

```
julia> next_calkin_wilf(ZZ(321)//113)
113//244
```

source

Nemo.next_signed_calkin_wilf – Method.

```
next_signed_calkin_wilf(a::QQFieldElem)
```

Given a signed rational number $a$ returns the next element in the Calkin-Wilf sequence with negative numbers interleaved. The sequence begins $0, 1, -1, 1/2, -1/2, 2, -2, 1/3, -1/3, \ldots$. Starting with zero, this generates every rational number once and only once, but not in order of minimal height.

**Examples**

```
julia> next_signed_calkin_wilf(-ZZ(51)//(17))
1//4
```

source

### Random generation

Nemo.rand_bits – Method.

```
rand_bits(::QQField, b::Int)
```

Return a random signed rational whose numerator and denominator both have $b$ bits before canonicalisation. Note that the resulting numerator and denominator can be smaller than $b$ bits.

source

### Special functions

The following special functions are available for specific rings in Nemo.

Nemo.harmonic – Method.

```
harmonic(n::Int)
```

Return the harmonic number $H_n = 1 + 1/2 + 1/3 + \cdots + 1/n$. Table lookup is used for $H_n$ whose numerator and denominator fit in a single limb. For larger $n$, a divide and conquer strategy is used.

**Examples**

```
julia> a = harmonic(12)
86021//27720
```

source

Nemo.bernoulli – Method.

```
bernoulli(n::Int)
```

Return the Bernoulli number $B_n$ for non-negative $n$.

See also bernoulli_cache.

**Examples**

```
julia> d = bernoulli(12)
-691//2730
```

source

Nemo.bernoulli_cache – Method.

```
bernoulli_cache(n::Int)
```

Precomputes and caches all the Bernoulli numbers up to $B_n$. This is much faster than repeatedly calling bernoulli(k). Once cached, subsequent calls to bernoulli(k) for any $k \leq n$ will read from the cache, making them virtually free.

See also bernoulli.

**Examples**

```
julia> bernoulli_cache(100)

julia> e = bernoulli(100)
-945980378191221252952274330694937218727028415330669361333856962043113954151972477 11//33330
```

source

Nemo.dedekind_sum – Method.

```
dedekind_sum(h::ZZRingElem, k::ZZRingElem)
```

Return the Dedekind sum $s(h, k)$ for arbitrary $h$ and $k$.

**Examples**

```
julia> b = dedekind_sum(12, 13)
-11//13

julia> c = dedekind_sum(-120, ZZ(1305))
-575//522
```

source

Nemo.simplest_between – Method.

```
simplest_between(l::QQFieldElem, r::QQFieldElem)
```

Return the simplest fraction in the closed interval $[l, r]$. A canonical fraction $a_1/b_1$ is defined to be simpler than $a_2/b_2$ if and only if $b_1 < b_2$ or $b_1 = b_2$ and $a_1 < a_2$.

**Examples**

```
julia> simplest_between(QQ(1//10), QQ(3//10))
1//4
```

source

# Chapter 18

# Rationals

Nemo provides much functionality for the rational numbers. See the section on Fraction Fields where all the basic functionality is documented, along with the extra functionality only available for the rational numbers themselves.

# Chapter 19

# Algebraic closure of the rational numbers

An implementation of the field of all algebraic numbers, that is, an algebraic closure of the field of rational numbers, is provided through the type `QQBarField` and the corresponding element type `QQBarFieldElem`.

Note that the field of algebraic closure is implemented as an ordered field; see Comparing algebraic numbers.

## 19.1 Constructing the field of algebraic numbers

```
julia> algebraic_closure(QQ)
Algebraic closure of rational field
```

### Printing

Before looking at examples, we comment on the printing of algebraic numbers. As an illustration, consider the algebraic number $\sqrt[3]{2}$ with minimal polynomial $X^3 - 2$. This is printed as follows:

```
julia> Qbar = algebraic_closure(QQ);

julia> z = root(Qbar(2), 3)
{a3: 1.25992}

julia> Qx, x = QQ[:x]
(Univariate polynomial ring in x over QQ, x)

julia> minpoly(Qx, z) # to see the minimal polynomial
x^3 - 2
```

The first part a3 indicates that this is an algebraic number of degree $3$, which is followed by `1.25992`, an approximation of the root.

## 19.2 Algebraic number functionality

Methods to construct algebraic numbers include:

- Conversion from other numbers and through arithmetic operations

- Computing the roots of a given polynomial
- Computing the eigenvalues of a given matrix
- Random generation
- Exact trigonometric functions (see later section)
- Guessing (see later section)

**Examples**

Arithmetic:

```
julia> Qb = algebraic_closure(QQ);

julia> ZZRingElem(Qb(3))
3

julia> QQFieldElem(Qb(3) // 2)
3//2

julia> Qb(-1) ^ (Qb(1) // 3)
{a2: 0.500000 + 0.866025*im}
```

Solving the quintic equation:

```
julia> Qb = algebraic_closure(QQ);

julia> R, x = polynomial_ring(QQ, "x")
(Univariate polynomial ring in x over QQ, x)

julia> v = roots(Qb, x^5-x-1)
5-element Vector{QQBarFieldElem}:
 {a5: 1.16730}
 {a5: 0.181232 + 1.08395*im}
 {a5: 0.181232 - 1.08395*im}
 {a5: -0.764884 + 0.352472*im}
 {a5: -0.764884 - 0.352472*im}

julia> v[1]^5 - v[1] - 1 == 0
true
```

Computing exact eigenvalues of a matrix:

```
julia> Qb = algebraic_closure(QQ);

julia> eigenvalues(Qb, ZZ[1 1 0; 0 1 1; 1 0 1])
3-element Vector{QQBarFieldElem}:
 {a1: 2.00000}
 {a2: 0.500000 + 0.866025*im}
 {a2: 0.500000 - 0.866025*im}
```

### Interface

`AbstractAlgebra.Generic.roots` – Method.

```
roots(R::QQBarField, f::ZZPolyRingElem)
```

Return all the roots of the polynomial `f` in the field of algebraic numbers `R`. The output array is sorted in the default sort order for algebraic numbers. Roots of multiplicity higher than one are repeated according to their multiplicity.

source

`AbstractAlgebra.Generic.roots` – Method.

```
roots(R::QQBarField, f::QQPolyRingElem)
```

Return all the roots of the polynomial `f` in the field of algebraic numbers `R`. The output array is sorted in the default sort order for algebraic numbers. Roots of multiplicity higher than one are repeated according to their multiplicity.

source

`Nemo.eigenvalues` – Method.

```
eigenvalues(R::QQBarField, A::ZZMatrix)
eigenvalues(R::QQBarField, A::QQMatrix)
```

Return the eigenvalues `A` in the field of algebraic numbers `R`. The output array is sorted in the default sort order for algebraic numbers.

source

```
eigenvalues(L::Field, M::MatElem{T}) where T <: RingElem
```

Return the eigenvalues of `M` over the field `L`.

source

`Nemo.eigenvalues_with_multiplicities` – Method.

```
eigenvalues_with_multiplicities(R::QQBarField, A::ZZMatrix)
eigenvalues_with_multiplicities(R::QQBarField, A::QQMatrix)
```

Return the eigenvalues `A` in the field of algebraic numbers `R` together with their algebraic multiplicities as a vector of tuples. The output array is sorted in the default sort order for algebraic numbers.

source

```
eigenvalues_with_multiplicities(L::Field, M::MatElem{T}) where T <: RingElem
```

Return the eigenvalues of `M` over the field `L` together with their algebraic multiplicities as a vector of tuples.

source

`Nemo.eigenvalues` – Method.

```
eigenvalues(R::QQBarField, A::ZZMatrix)
eigenvalues(R::QQBarField, A::QQMatrix)
```

Return the eigenvalues `A` in the field of algebraic numbers `R`. The output array is sorted in the default sort order for algebraic numbers.

source

```
eigenvalues(L::Field, M::MatElem{T}) where T <: RingElem
```

Return the eigenvalues of `M` over the field `L`.

source

`Nemo.eigenvalues_with_multiplicities` – Method.

```
eigenvalues_with_multiplicities(R::QQBarField, A::ZZMatrix)
eigenvalues_with_multiplicities(R::QQBarField, A::QQMatrix)
```

Return the eigenvalues `A` in the field of algebraic numbers `R` together with their algebraic multiplicities as a vector of tuples. The output array is sorted in the default sort order for algebraic numbers.

source

```
eigenvalues_with_multiplicities(L::Field, M::MatElem{T}) where T <: RingElem
```

Return the eigenvalues of `M` over the field `L` together with their algebraic multiplicities as a vector of tuples.

source

`Base.rand` – Method.

```
rand(R::QQBarField; degree::Int, bits::Int, randtype::Symbol=:null)
```

Return a random algebraic number with degree up to `degree` and coefficients up to `bits` in size. By default, both real and complex numbers are generated. Set the optional `randtype` to `:real` or `:nonreal` to generate a specific type of number. Note that nonreal numbers require `degree` at least 2.

source

### Numerical evaluation

#### Examples

Algebraic numbers can be evaluated numerically to arbitrary precision by converting to real or complex Arb fields:

```
julia> Qb = algebraic_closure(QQ);

julia> RR = ArbField(64); RR(sqrt(Qb(2)))
[1.414213562373095049 +/- 3.45e-19]

julia> CC = AcbField(32); CC(Qb(-1) ^ (Qb(1) // 4))
[0.707106781 +/- 2.74e-10] + [0.707106781 +/- 2.74e-10]*im
```

### Minimal polynomials, conjugates, and properties

#### Examples

Retrieving the minimal polynomial and algebraic conjugates of a given algebraic number:

```
julia> Qb = algebraic_closure(QQ);

julia> minpoly(polynomial_ring(ZZ, "x")[1], Qb(1+2im))
x^2 - 2*x + 5

julia> conjugates(Qb(1+2im))
2-element Vector{QQBarFieldElem}:
 {a2: 1.00000 + 2.00000*im}
 {a2: 1.00000 - 2.00000*im}
```

#### Interface

Various properties are implemented, including `iszero`, `isone`, and `isinteger`:

```
julia> Qb = algebraic_closure(QQ);

julia> iszero(Qb(0)), isone(Qb(1)), isinteger(Qb(3//1))
(true, true, true)
```

The full interface includes:

`Nemo.is_rational` – Method.

```
is_rational(x::QQBarFieldElem)
```

Return whether x is a rational number.

source

`Base.isreal` – Method.

```
isreal(x::QQBarFieldElem)
```

Return whether x is a real number.

source

`AbstractAlgebra.degree` – Method.

```
degree(x::QQBarFieldElem)
```

Return the degree of the minimal polynomial of x.

source

`Nemo.is_algebraic_integer` – Method.

```
is_algebraic_integer(x::QQBarFieldElem)
```

Return whether x is an algebraic integer.

source

`AbstractAlgebra.minpoly` – Method.

```
minpoly(R::ZZPolyRing, x::QQBarFieldElem)
```

Return the minimal polynomial of x as an element of the polynomial ring R.

source

`AbstractAlgebra.minpoly` – Method.

```
minpoly(R::QQPolyRing, x::QQBarFieldElem)
```

Return the minimal polynomial of x as an element of the polynomial ring R.

source

`Nemo.conjugates` – Method.

```
conjugates(a::QQBarFieldElem)
```

Return all the roots of the polynomial f in the field of algebraic numbers R. The output array is sorted in the default sort order for algebraic numbers.

source

`Base.denominator` – Method.

```
denominator(x::QQBarFieldElem)
```

Return the denominator of x, defined as the leading coefficient of the minimal polynomial of x. The result is returned as an ZZRingElem.

source

Base.numerator – Method.

```
numerator(x::QQBarFieldElem)
```

Return the numerator of x, defined as x multiplied by its denominator. The result is an algebraic integer.

source

Nemo.height – Method.

```
height(x::QQBarFieldElem)
```

Return the height of the algebraic number x. The result is an ZZRingElem integer.

source

Nemo.height_bits – Method.

```
height_bits(x::QQBarFieldElem)
```

Return the height of the algebraic number x measured in bits. The result is a Julia integer.

source

### Complex parts

#### Examples

```
julia> Qb = algebraic_closure(QQ);

julia> real(sqrt(Qb(1im)))
{a2: 0.707107}

julia> abs(sqrt(Qb(1im)))
{a1: 1.00000}

julia> floor(sqrt(Qb(1000)))
{a1: 31.0000}

julia> sign(Qb(-10-20im))
{a4: -0.447214 - 0.894427*im}
```

**Interface**

`Base.real` – Method.

```
real(a::QQBarFieldElem)
```

Return the real part of a.

source

`Base.imag` – Method.

```
imag(a::QQBarFieldElem)
```

Return the imaginary part of a.

source

`Base.abs` – Method.

```
abs(a::QQBarFieldElem)
```

Return the absolute value of a.

source

`Base.abs2` – Method.

```
abs2(a::QQBarFieldElem)
```

Return the squared absolute value of a.

source

`Base.conj` – Method.

```
conj(a::QQBarFieldElem)
```

Return the complex conjugate of a.

source

`Base.sign` – Method.

```
sign(a::QQBarFieldElem)
```

Return the complex sign of a, defined as zero if a is zero and as $a/|a|$ otherwise.

source

`Nemo.csgn` – Method.

```
csgn(a::QQBarFieldElem)
```

Return the extension of the real sign function taking the value 1 strictly in the right half plane, -1 strictly in the left half plane, and the sign of the imaginary part when on the imaginary axis. Equivalently, $\mathrm{csgn}(x) = x/\sqrt{x^2}$ except that the value is 0 at zero.

source

`Nemo.sign_real` – Method.

```
sign_real(a::QQBarFieldElem)
```

Return the sign of the real part of a.

source

`Nemo.sign_imag` – Method.

```
sign_imag(a::QQBarFieldElem)
```

Return the sign of the imaginary part of a.

source

### Comparing algebraic numbers

The operators == and != check exactly for equality.

We provide various comparison functions for ordering algebraic numbers:

- Standard comparison for real numbers (<, `isless`)
- Real parts
- Imaginary parts
- Absolute values
- Absolute values of real or imaginary parts
- Root sort order

The standard comparison will throw if either argument is nonreal.

The various comparisons for complex parts are provided as separate operations since these functions are far more efficient than explicitly computing the complex parts and then doing real comparisons.

The root sort order is a total order for complex algebraic numbers used to order the output of `roots` and `conjugates` canonically. We define this order as follows: real roots come first, in descending order. Nonreal roots are subsequently ordered first by real part in descending order, then in ascending order by the absolute

value of the imaginary part, and then in descending order of the sign of the imaginary part. This implies that complex conjugate roots are adjacent, with the root in the upper half plane first.

**Examples**

```
julia> Qb = algebraic_closure(QQ);

julia> 1 < sqrt(Qb(2)) < Qb(3)//2
true

julia> x = Qb(3+4im)
{a2: 3.00000 + 4.00000*im}

julia> is_equal_abs(x, -x)
true

julia> is_equal_abs_imag(x, 2-x)
true

julia> is_less_real(x, x // 2)
false
```

**Interface**

Nemo.is_equal_real – Method.

```
is_equal_real(a::QQBarFieldElem, b::QQBarFieldElem)
```

Compares the real parts of a and b.

source

Nemo.is_equal_imag – Method.

```
is_equal_imag(a::QQBarFieldElem, b::QQBarFieldElem)
```

Compares the imaginary parts of a and b.

source

Nemo.is_equal_abs – Method.

```
is_equal_abs(a::QQBarFieldElem, b::QQBarFieldElem)
```

Compares the absolute values of a and b.

source

Nemo.is_equal_abs_real – Method.

```
is_equal_abs_real(a::QQBarFieldElem, b::QQBarFieldElem)
```

Compares the absolute values of the real parts of a and b.

source

Nemo.is_equal_abs_imag – Method.

```
is_equal_abs_imag(a::QQBarFieldElem, b::QQBarFieldElem)
```

Compares the absolute values of the imaginary parts of a and b.

source

Nemo.is_less_real – Method.

```
is_less_real(a::QQBarFieldElem, b::QQBarFieldElem)
```

Compares the real parts of a and b.

source

Nemo.is_less_imag – Method.

```
is_less_imag(a::QQBarFieldElem, b::QQBarFieldElem)
```

Compares the imaginary parts of a and b.

source

Nemo.is_less_abs – Method.

```
is_less_abs(a::QQBarFieldElem, b::QQBarFieldElem)
```

Compares the absolute values of a and b.

source

Nemo.is_less_abs_real – Method.

```
is_less_abs_real(a::QQBarFieldElem, b::QQBarFieldElem)
```

Compares the absolute values of the real parts of a and b.

source

Nemo.is_less_abs_imag – Method.

```
is_less_abs_imag(a::QQBarFieldElem, b::QQBarFieldElem)
```

Compares the absolute values of the imaginary parts of a and b.

source

Nemo.is_less_root_order – Method.

```
is_less_root_order(a::QQBarFieldElem, b::QQBarFieldElem)
```

Compares the a and b in root sort order.

source

### Roots and trigonometric functions

#### Examples

```
julia> Qb =  algebraic_closure(QQ);

julia> root(Qb(2), 5)
{a5: 1.14870}

julia> sinpi(Qb(7) // 13)
{a12: 0.992709}

julia> tanpi(atanpi(sqrt(Qb(2)) + 1))
{a2: 2.41421}

julia> root_of_unity(Qb, 5)
{a4: 0.309017 + 0.951057*im}

julia> root_of_unity(Qb, 5, 4)
{a4: 0.309017 - 0.951057*im}

julia> w = (1 - sqrt(Qb(-3)))//2
{a2: 0.500000 - 0.866025*im}

julia> is_root_of_unity(w)
true

julia> is_root_of_unity(w + 1)
false

julia> root_of_unity_as_args(w)
(6, 5)
```

#### Interface

Base.sqrt – Method.

```
sqrt(a::QQBarFieldElem; check::Bool=true)
```

Return the principal square root of a.

source

`AbstractAlgebra.root` – Method.

```
root(a::QQBarFieldElem, n::Int)
```

Return the principal n-th root of a. Requires positive n.

source

`Nemo.root_of_unity` – Method.

```
root_of_unity(C::QQBarField, n::Int, k::Int = 1)
```

Return the root of unity $e^{2\pi ik/n}$ as an element of the field of algebraic numbers C.

source

`Nemo.root_of_unity` – Method.

```
root_of_unity(C::QQBarField, n::Int, k::Int = 1)
```

Return the root of unity $e^{2\pi ik/n}$ as an element of the field of algebraic numbers C.

source

`Nemo.is_root_of_unity` – Method.

```
is_root_of_unity(a::QQBarFieldElem)
```

Return whether the given algebraic number is a root of unity.

source

`Nemo.root_of_unity_as_args` – Method.

```
root_of_unity_as_args(a::QQBarFieldElem)
```

Return a pair of integers `(q, p)` such that the given a equals $e^{2\pi ip/q}$. The denominator q will be minimal, with $0 \leq p < q$. Throws if a is not a root of unity.

source

`Nemo.exp_pi_i` – Method.

```
exp_pi_i(a::QQBarFieldElem)
```

Return $e^{\pi i a}$ as an algebraic number. Throws if this value is transcendental.

source

Nemo.log_pi_i – Method.

```
log_pi_i(a::QQBarFieldElem)
```

Return $\log(a)/(\pi i)$ as an algebraic number. Throws if this value is transcendental or undefined.

source

Base.Math.sinpi – Method.

```
sinpi(a::QQBarFieldElem)
```

Return $\sin(\pi a)$ as an algebraic number. Throws if this value is transcendental.

**Examples**

```
julia> QQBar = algebraic_closure(QQ);

julia> x = sinpi(QQBar(1//3))
{a2: 0.866025}
```

source

Base.Math.cospi – Method.

```
cospi(a::QQBarFieldElem)
```

Return $\cos(\pi a)$ as an algebraic number. Throws if this value is transcendental.

**Examples**

```
julia> QQBar = algebraic_closure(QQ);

julia> x = cospi(QQBar(1//6))
{a2: 0.866025}
```

source

Base.Math.sincospi – Method.

```
sincospi(a::QQBarFieldElem)
```

Return $\sin(\pi a)$ and $\cos(\pi a)$ as a pair of algebraic numbers. Throws if either value is transcendental.

**Examples**

```
julia> QQBar = algebraic_closure(QQ);

julia> s, c = sincospi(QQBar(1//3))
({a2: 0.866025}, {a1: 0.500000})
```

source

`Base.Math.tanpi` – Method.

```
tanpi(a::QQBarFieldElem)
```

Return $\tan(\pi a)$ as an algebraic number. Throws if this value is transcendental or undefined.

source

`Nemo.asinpi` – Method.

```
asinpi(a::QQBarFieldElem)
```

Return $\operatorname{asin}(a)/\pi$ as an algebraic number. Throws if this value is transcendental.

source

`Nemo.acospi` – Method.

```
acospi(a::QQBarFieldElem)
```

Return $\operatorname{acos}(a)/\pi$ as an algebraic number. Throws if this value is transcendental.

source

`Nemo.atanpi` – Method.

```
atanpi(a::QQBarFieldElem)
```

Return $\operatorname{atan}(a)/\pi$ as an algebraic number. Throws if this value is transcendental or undefined.

source

### Guessing

#### Examples

An algebraic number can be recovered from a numerical value:

```
julia> Qb = algebraic_closure(QQ);

julia> RR = real_field(); guess(Qb, RR("1.41421356 +/- 1e-6"), 2)
{a2: 1.41421}
```

Warning: the input should be an enclosure. If you have a floating-point approximation, you should add an error estimate; otherwise, at best the only algebraic number that can be guessed is the binary floating-point number itself, at worst no guess is possible.

```
julia> Qb = algebraic_closure(QQ);

julia> RR = real_field();

julia> x = RR(0.1)       # note: 53-bit binary approximation of 1//10 without radius
[0.10000000000000000555 +/- 1.12e-21]

julia> guess(Qb, x, 1)
ERROR: No suitable algebraic number found
[...]

julia> guess(Qb, x + RR("+/- 1e-10"), 1)
{a1: 0.100000}
```

#### Interface

Nemo.guess – Function.

```
guess(R::QQBarField, x::AcbFieldElem, maxdeg::Int, maxbits::Int=0)
guess(R::QQBarField, x::ArbFieldElem, maxdeg::Int, maxbits::Int=0)
guess(R::QQBarField, x::ComplexFieldElem, maxdeg::Int, maxbits::Int=0)
guess(R::QQBarField, x::RealFieldElem, maxdeg::Int, maxbits::Int=0)
```

Try to reconstruct an algebraic number from a given numerical enclosure x. The algorithm looks for candidates up to degree maxdeg and with coefficients up to size maxbits (which defaults to the precision of x if not given). Throws if no suitable algebraic number can be found.

Guessing typically requires high precision to succeed, and it does not make much sense to call this function with input precision smaller than $O(maxdeg \cdot maxbits)$. If this function succeeds, then the output is guaranteed to be contained in the enclosure x, but failure does not prove that such an algebraic number with the specified parameters does not exist.

This function does a single iteration with the target parameters. For best performance, one should invoke this function repeatedly with successively larger parameters when the size of the intended solution is unknown or may be much smaller than a worst-case bound.

source

## Chapter 20

# Important note on performance

The default algebraic number type represents algebraic numbers in canonical form using minimal polynomials. This works well for representing individual algebraic numbers, but it does not provide the best performance for field arithmetic. For fast calculation in $\overline{\mathbb{Q}}$, `CalciumField` should typically be used instead (see the section on Exact real and complex numbers). Alternatively, to compute in a fixed subfield of $\overline{\mathbb{Q}}$, you may fix a generator $a$ and construct a number field to represent $\mathbb{Q}(a)$.

# Chapter 21

# Exact real and complex numbers

Exact real and complex numbers are provided by FLINT. The corresponding Nemo types retain the historical name Calcium, from the separate library that is now part of FLINT. Internally, a number $z$ is represented as an element of an extension field of the rational numbers. That is,

$$z \in \mathbb{Q}(a_1, \ldots, a_n)$$

where $a_1, \ldots, a_n$ are symbolically defined algebraic or transcendental real or complex numbers such as $\pi$, $\sqrt{2}$ or $e^{\sqrt{2}\pi i}$. The user does not normally need to worry about the details of the internal representation; FLINT constructs extension numbers and fields automatically as needed to perform operations.

The user must create a `CalciumField` instance which represents the mathematical domain $\mathbb{C}$. This parent object holds a cache of extension numbers and fields used to represent individual elements. It also stores various options for evaluation (documented further below).

| Library | Element type | Parent type |
|---|---|---|
| FLINT | `CalciumFieldElem` | `CalciumField` |

Please note the following:

- It is in the nature of exact complex arithmetic that some operations must be implemented using incomplete heuristics. For example, testing whether an element is zero will not always succeed. When FLINT is unable to perform a task, Nemo will throw an exception. This ensures that `CalciumFieldElems` behave exactly and never silently return wrong results.
- `CalciumFieldElem` values can optionally hold special non-numerical values:
  - Unsigned infinity $\hat{\infty}$
  - Signed infinities ($\pm\infty$, $\pm i\infty$, and more generally $e^{i\theta} \cdot \infty$)
  - Undefined
  - Unknown

  By default, such special values are disallowed so that a `CalciumField` represents the mathematical field $\mathbb{C}$, and any operation that would result in a special value (for example, $1/0 = \hat{\infty}$) will throw an exception. To allow special values, pass `extended=true` to the `CalciumField` constructor (see Constructor options).

- `CalciumField` instances only support single-threaded use. You must create a separate parent object for each thread to do parallel computation.
- When performing an operation involving two `CalciumFieldElem` operands with different parent objects, Nemo will arbitrarily coerce the operands (and hence the result) to one of the parents.

## 21.1 Constructor options

The `CalciumField` parent stores various options that affect simplification power, performance, or appearance. The user can override any of the default values using `C = CalciumField(options=dict)` where `dict` is a dictionary with `Symbol => Int` pairs. To retrieve the option values as a dictionary (including any default values not set by the user), call `options(C)`.

The following options are supported:

| Option | Explanation |
|---:|---:|
| `:verbose` | Enable debug output |
| `:print_flags` | Flags controlling print style |
| `:mpoly_ord` | Monomial order for polynomials |
| `:prec_limit` | Precision limit for numerical evaluation |
| `:qqbar_deg_limit` | Degree limit for algebraic numbers |
| `:low_prec` | Initial precision for numerical evaluation |
| `:smooth_limit` | Factor size limit for smooth integer factorization |
| `:lll_prec` | Precision for integer relation detection |
| `:pow_limit` | Maximum exponent for in-field powering |
| `:use_gb` | Enable Gröbner basis computation |
| `:gb_length_limit` | Maximum ideal basis length during Gröbner basis computation |
| `:gb_poly_length_limit` | Maximum polynomial length during Gröbner basis computation |
| `:gb_poly_bits_limit` | Maximum bit size during Gröbner basis computation |
| `:gb_vieta_limit` | Maximum degree to use Vieta's formulas |
| `:trig_form` | Default form of trigonometric functions |

An important function of these options is to control how hard FLINT will try to find an answer before it gives up. For example:

- Setting `:prec_limit => 65536` will allow FLINT to use up to 65536 bits of precision (instead of the default 4096) to prove inequalities.
- Setting `:qqbar_deg_limit => typemax(Int)` (instead of the default 120) will force most calculations involving algebraic numbers to run to completion, no matter how long this will take.
- Setting `:use_gb => 0` (instead of the default 1) disables use of Gröbner bases. In general, this will negatively impact FLINT's ability to simplify field elements and prove equalities, but it can speed up calculations where Gröbner bases are unnecessary.

For a detailed explanation, refer to the following section in the FLINT documentation: https://flintlib.org/doc/ca.html#context-options

## 21.2 Basic examples

```
julia> C = CalciumField()
Exact complex field

julia> exp(C(pi) * C(1im)) + 1
0

julia> log(C(-1))
3.14159*I {a*b where a = 3.14159 [Pi], b = I [b^2+1=0]}

julia> log(C(-1)) ^ 2
-9.86960 {-a^2 where a = 3.14159 [Pi], b = I [b^2+1=0]}

julia> log(C(10)^23) // log(C(100))
11.5000 {23/2}

julia> 4*atan(C(1)//5) - atan(C(1)//239) == C(pi)//4
true

julia> Cx, x = polynomial_ring(C, "x")
(Univariate polynomial ring in x over exact complex field, x)

julia> (a, b) = (sqrt(C(2)), sqrt(C(3)))
(1.41421 {a where a = 1.41421 [a^2-2=0]}, 1.73205 {a where a = 1.73205 [a^2-3=0]})

julia> (x-a-b)*(x-a+b)*(x+a-b)*(x+a+b)
x^4 + -10*x^2 + 1
```

## 21.3 Conversions and numerical evaluation

Exact complex numbers can be created from integers (`ZZ`), rationals (`QQ`) and algebraic numbers (`algebraic_closure(QQ)`), and through the application of arithmetic operations and transcendental functions.

`CalciumFieldElems` can be converted to integer, rational and algebraic fields provided that the values are integer, rational or algebraic. An exception is thrown if the value does not belong to the target domain, if FLINT is unable to prove that the value belongs to the target domain, or if FLINT is unable to compute the explicit value because of evaluation limits.

```
julia> QQ(C(1))
1

julia> algebraic_closure(QQ)(sqrt(C(2)) // 2)
{a2: 0.707107}

julia> QQ(C(pi))
ERROR: unable to convert to a rational number
[...]

julia> QQ(C(10) ^ C(10^9))
ERROR: unable to convert to a rational number
[...]
```

To compute arbitrary-precision numerical enclosures, convert to `ArbField` or `AcbField`:

```
julia> CC = AcbField(64)
Complex Field with 64 bits of precision and error bounds

julia> CC(exp(C(1im)))
[0.54030230586813971740 +/- 9.37e-22] + [0.84147098480789650665 +/- 2.51e-21]*im
```

The constructor

```
(R::AcbField)(a::CalciumFieldElem; parts::Bool=false)
```

returns an enclosure of the complex number *a*. It attempts to obtain a relative accuracy of *prec* bits where *prec* is the precision of the target field, but it is not guaranteed that this goal is achieved.

If *parts* is set to *true*, it attempts to achieve the target accuracy for both real and imaginary parts. This can be significantly more expensive if one part is smaller than the other, or if the number is nontrivially purely real or purely imaginary (in which case an exact proof attempt is made).

```
julia> x = sin(C(1), form=:exponential)
0.841471 + 0e-24*I {(-a^2*b+b)/(2*a) where a = 0.540302 + 0.841471*I [Exp(1.00000*I {b})], b = I
↪  [b^2+1=0]}

julia> AcbField(64)(x)
[0.84147098480789650665 +/- 2.51e-21] + [+/- 4.77e-29]*im

julia> AcbField(64)(x, parts=true)
[0.84147098480789650665 +/- 2.51e-21]
```

The constructor

```
(R::ArbField)(a::CalciumFieldElem; check::Bool=true)
```

returns a real enclosure. If *check* is set to *true* (default), the number *a* is verified to be real, and an exception is thrown if this cannot be determined. With *check* set to *false*, this function returns an enclosure of the real part of *a* without checking that the imaginary part is zero. This can be significantly faster.

## 21.4 Comparisons and properties

Except where otherwise noted, predicate functions such as `iszero`, `==`, `<` and `isreal` act on the mathematical values of `CalciumFieldElem`s. For example, although evaluating $x = \sqrt{2}\sqrt{3}$ and $y = \sqrt{6}$ results in different internal representations ($x \in \mathbb{Q}(\sqrt{3}, \sqrt{2})$ and $y \in \mathbb{Q}(\sqrt{6})$), the numbers compare as equal:

```
julia> x = sqrt(C(2)) * sqrt(C(3))
2.44949 {a*b where a = 1.73205 [a^2-3=0], b = 1.41421 [b^2-2=0]}

julia> y = sqrt(C(6))
2.44949 {a where a = 2.44949 [a^2-6=0]}

julia> x == y
true
```

```
julia> iszero(x - y)
true

julia> isinteger(x - y)
true
```

Predicate functions return *true* if the property is provably true and *false* if the property if provably false. If FLINT is unable to prove the truth value, an exception is thrown. For example, with default settings, FLINT is currently able to prove that $e^{e^{-1000}} \neq 1$, but it fails to prove $e^{e^{-3000}} \neq 1$:

```
julia> x = exp(exp(C(-1000)))
1.00000 {a where a = 1.00000 [Exp(5.07596e-435 {b})], b = 5.07596e-435 [Exp(-1000)]}

julia> x == 1
false

julia> x = exp(exp(C(-3000)))
1.00000 {a where a = 1.00000 [Exp(1.30784e-1303 {b})], b = 1.30784e-1303 [Exp(-3000)]}

julia> x == 1
ERROR: Unable to perform operation (failed deciding truth of a predicate): isequal
[...]
```

In this case, we can get an answer by allowing a higher working precision:

```
julia> C2 = CalciumField(options=Dict(:prec_limit => 10^5));

julia> exp(exp(C2(-3000))) == 1
false
```

Real numbers can be ordered and sorted the usual way. We illustrate finding square roots that are well-approximated by integers:

```
julia> sort([sqrt(C(n)) for n=0:10], by=x -> abs(x - floor(x + C(1)//2)))
11-element Vector{CalciumFieldElem}:
 0
 1
 2
 3
 3.16228 {a where a = 3.16228 [a^2-10=0]}
 2.82843 {2*a where a = 1.41421 [a^2-2=0]}
 2.23607 {a where a = 2.23607 [a^2-5=0]}
 1.73205 {a where a = 1.73205 [a^2-3=0]}
 2.64575 {a where a = 2.64575 [a^2-7=0]}
 1.41421 {a where a = 1.41421 [a^2-2=0]}
 2.44949 {a where a = 2.44949 [a^2-6=0]}
```

As currently implemented, order comparisons involving nonreal numbers yield *false* (in both directions) rather than throwing an exception:

```
julia> C(lim) < C(lim)
false

julia> C(lim) > C(lim)
false
```

This behavior may be changed or may become configurable in the future.

**Interface**

Various properties are implemented, including `iszero`, `isone`, and `isinteger`:

```
julia> iszero(C(0)), isone(C(1)), isinteger(C(2))
(true, true, true)
```

The full interface includes:

`Nemo.is_algebraic` – Method.

```
is_algebraic(a::CalciumFieldElem)
```

Return whether a is an algebraic number.

source

`Nemo.is_rational` – Method.

```
is_rational(a::CalciumFieldElem)
```

Return whether a is a rational number.

source

`Base.isreal` – Method.

```
isreal(a::CalciumFieldElem)
```

Return whether a is a real number. This returns `false` if a is a pure real infinity.

source

`Nemo.is_imaginary` – Method.

```
is_imaginary(a::CalciumFieldElem)
```

Return whether a is an imaginary number. This returns `false` if a is a pure imaginary infinity.

source

## 21.5 Infinities and special values

By default, `CalciumField` does not permit creating values that are not numbers, and any non-number value (unsigned infinity, signed infinity, Undefined) will result in an exception. This also applies to the special value Unknown, used in situations where FLINT is unable to prove that a value is a number. To enable special values, use `extended=true`.

```
julia> C = CalciumField()
Exact complex field

julia> 1 // C(0)
ERROR: DomainError with UnsignedInfinity:
Non-number result
[...]

julia> Cext = CalciumField(extended=true)
Exact complex field (extended)

julia> 1 // Cext(0)
UnsignedInfinity
```

Note that special values do not satisfy the properties of a mathematical ring or field. You will likely get meaningless results if you put infinities in matrices or polynomials.

`Nemo.unsigned_infinity` – Method.

```
unsigned_infinity(C::CalciumField)
```

Return unsigned infinity ($\tilde{\infty}$) as an element of `C`. This throws an exception if `C` does not allow special values.

source

`Nemo.infinity` – Method.

```
infinity(C::CalciumField)
```

Return positive infinity ($+\infty$) as an element of `C`. This throws an exception if `C` does not allow special values.

source

`Nemo.infinity` – Method.

```
infinity(a::CalciumFieldElem)
```

Return the signed infinity ($a \cdot \infty$). This throws an exception if the parent of `a` does not allow special values.

source

`Nemo.undefined` – Method.

```
undefined(C::CalciumField)
```

Return the special value Undefined as an element of C. This throws an exception if C does not allow special values.

source

Nemo.unknown – Method.

```
unknown(C::CalciumField)
```

Return the special meta-value Unknown as an element of C. This throws an exception if C does not allow special values.

source

Nemo.is_number – Method.

```
is_number(a::CalciumFieldElem)
```

Return whether a is a number, i.e. not an infinity or undefined.

source

Nemo.is_undefined – Method.

```
is_undefined(a::CalciumFieldElem)
```

Return whether a is the special value *Undefined*.

source

Base.isinf – Method.

```
isinf(a::CalciumFieldElem)
```

Return whether a is any infinity (signed or unsigned).

source

Nemo.is_uinf – Method.

```
is_uinf(a::CalciumFieldElem)
```

Return whether a is unsigned infinity.

source

Nemo.is_signed_inf – Method.

```
is_signed_inf(a::CalciumFieldElem)
```

Return whether a is any signed infinity.

source

Nemo.is_unknown – Method.

```
is_unknown(a::CalciumFieldElem)
```

Return whether a is the special value *Unknown*. This is a representation property and not a mathematical predicate.

source

## 21.6 Complex parts

Functions for computing components of real and complex numbers will perform automatic symbolic simplifications in special cases. In general, such operations will introduce new extension numbers.

```
julia> real(C(2+3im))
2

julia> sign(C(2im))
1.00000*I {a where a = I [a^2+1=0]}

julia> sign(C(2+3im))
0.554700 + 0.832050*I {a where a = 0.554700 + 0.832050*I [13*a^4+10*a^2+13=0]}

julia> angle(C(2+2im))
0.785398 {(a)/4 where a = 3.14159 [Pi]}

julia> angle(C(2+3im))
0.982794 {a where a = 0.982794 [Arg(2.00000 + 3.00000*I {3*b+2})], b = I [b^2+1=0]}

julia> angle(C(2+3im)) == atan(C(3)//2)
true

julia> floor(C(pi) ^ 100)
5.18785e+49 {5187848314319613192086261524630301356268676068040 5}

julia> ZZ(floor(C(pi) ^ 100))
5187848314319613192086261524630301356268676068040 5
```

### Interface

Base.real – Method.

```
real(a::CalciumFieldElem)
```

Return the real part of a.

source

Base.imag – Method.

```
imag(a::CalciumFieldElem)
```

Return the imaginary part of a.

source

Base.angle – Method.

```
angle(a::CalciumFieldElem)
```

Return the complex argument of a.

source

Nemo.csgn – Method.

```
csgn(a::CalciumFieldElem)
```

Return the extension of the real sign function taking the value 1 strictly in the right half plane, -1 strictly in the left half plane, and the sign of the imaginary part when on the imaginary axis. Equivalently, $\operatorname{csgn}(x) = x/\sqrt{x^2}$ except that the value is 0 at zero.

source

Base.sign – Method.

```
sign(a::CalciumFieldElem)
```

Return the complex sign of a, defined as zero if a is zero and as $a/|a|$ for any other complex number. This function also extracts the sign when a is a signed infinity.

source

Base.abs – Method.

```
abs(a::CalciumFieldElem)
```

Return the absolute value of a.

source

Base.conj – Method.

```
conj(a::CalciumFieldElem; form::Symbol=:default)
```

Return the complex conjugate of a. The optional `form` argument allows specifying the representation. In `:shallow` form, $\overline{a}$ is introduced as a new extension number if it no straightforward simplifications are possible. In `:deep` form, complex conjugation is performed recursively.

source

`Base.floor` – Method.

```
floor(a::CalciumFieldElem)
```

Return the floor function of a.

source

`Base.ceil` – Method.

```
ceil(a::CalciumFieldElem)
```

Return the ceiling function of a.

source

## 21.7 Elementary and special functions

Elementary and special functions generally create new extension numbers. In special cases, simplifications occur automatically.

```
julia> exp(C(1))
2.71828 {a where a = 2.71828 [Exp(1)]}

julia> exp(C(0))
1

julia> atan(C(1))
0.785398 {(a)/4 where a = 3.14159 [Pi]}

julia> cos(C(1))^2 + sin(C(1))^2
1

julia> log(1 // exp(sqrt(C(2))+1)) == -sqrt(C(2)) - 1
true

julia> gamma(C(2+3im))
-0.0823953 + 0.0917743*I {a where a = -0.0823953 + 0.0917743*I [Gamma(2.00000 + 3.00000*I
↪  {3*b+2})], b = I [b^2+1=0]}

julia> gamma(C(5) // 2)
1.32934 {(3*a)/4 where a = 1.77245 [Sqrt(3.14159 {b})], b = 3.14159 [Pi]}
```

```
julia> erf(C(1))
0.842701 {a where a = 0.842701 [Erf(1)]}

julia> erf(C(1)) + erfc(C(1))
1
```

Some functions allow representing the result in different forms:

```
julia> s1 = sin(C(1))
0.841471 + 0e-24*I {(-a^2*b+b)/(2*a) where a = 0.540302 + 0.841471*I [Exp(1.00000*I {b})], b = I
↪  [b^2+1=0]}

julia> s2 = sin(C(1), form=:direct)
0.841471 {a where a = 0.841471 [Sin(1)]}

julia> s3 = sin(C(1), form=:exponential)
0.841471 + 0e-24*I {(-a^2*b+b)/(2*a) where a = 0.540302 + 0.841471*I [Exp(1.00000*I {b})], b = I
↪  [b^2+1=0]}

julia> s4 = sin(C(1), form=:tangent)
0.841471 {(2*a)/(a^2+1) where a = 0.546302 [Tan(0.500000 {1/2})]}

julia> s1 == s2 == s3 == s4
true

julia> isreal(s1) && isreal(s2) && isreal(s3) && isreal(s4)
true
```

The exponential form is currently used by default since it tends to be the most useful for symbolic simplification. The `:direct` and `:tangent` forms are likely to be better for numerical evaluation. The default behavior of trigonometric functions can be changed using the `:trig_form` option of `CalciumField`.

Proving equalities involving transcendental function values is a difficult problem in general. FLINT will sometimes fail even in elementary cases. Here is an example of two constant trigonometric identities where the first succeeds and the second fails:

```
julia> a = sqrt(C(2)) + 1;

julia> cos(a) + cos(2*a) + cos(3*a) == sin(7*a//2)//(2*sin(a//2)) - C(1)//2
true

julia> sin(3*a) == 4 * sin(a) * sin(C(pi)//3 - a) * sin(C(pi)//3 + a)
ERROR: Unable to perform operation (failed deciding truth of a predicate): isequal
[...]
```

A possible workaround is to fall back on a numerical comparison:

```
julia> abs(cos(a) + cos(2*a) + cos(3*a) - (sin(7*a//2)//(2*sin(a//2)) - C(1)//2)) <= C(10)^-100
true
```

Of course, this is not a rigorous proof that the numbers are equal, and `CalciumField` is overkill here; it would be far more efficient to use `ArbField` directly to check that the numbers are approximately equal.

**Interface**

`Nemo.const_pi` – Method.

```
const_pi(C::CalciumField)
```

Return the constant $\pi$ as an element of `C`.

source

`Nemo.const_euler` – Method.

```
const_euler(C::CalciumField)
```

Return Euler's constant $\gamma$ as an element of `C`.

source

`Nemo.onei` – Method.

```
onei(C::CalciumField)
```

Return the imaginary unit $i$ as an element of `C`.

source

`Base.sqrt` – Method.

```
Base.sqrt(a::CalciumFieldElem; check::Bool=true)
```

Return the principal square root of `a`.

source

`Base.exp` – Method.

```
exp(a::CalciumFieldElem)
```

Return the exponential function of `a`.

source

`Base.log` – Method.

```
log(a::CalciumFieldElem)
```

Return the natural logarithm of a.

source

Nemo.pow – Method.

```
pow(a::CalciumFieldElem, b::Int; form::Symbol=:default)
```

Return *a* raised to the integer power b. The optional `form` argument allows specifying the representation. In `:default` form, this is equivalent to a ^ b, which may create a new extension number $a^b$ if the exponent b is too large (as determined by the parent option `:pow_limit` or `:prec_limit` depending on the case). In `:arithmetic` form, the exponentiation is performed arithmetically in the field of a, regardless of the size of the exponent b.

source

Base.sin – Method.

```
sin(a::CalciumFieldElem; form::Symbol=:default)
```

Return the sine of a. The optional `form` argument allows specifying the representation. In `:default` form, the result is determined by the `:trig_form` option of the parent object. In `:exponential` form, the value is represented using complex exponentials. In `:tangent` form, the value is represented using tangents. In `:direct` form, the value is represented directly using a sine or cosine.

source

Base.cos – Method.

```
cos(a::CalciumFieldElem; form::Symbol=:default)
```

Return the cosine of a. The optional `form` argument allows specifying the representation. In `:default` form, the result is determined by the `:trig_form` option of the parent object. In `:exponential` form, the value is represented using complex exponentials. In `:tangent` form, the value is represented using tangents. In `:direct` form, the value is represented directly using a sine or cosine.

source

Base.tan – Method.

```
tan(a::CalciumFieldElem; form::Symbol=:default)
```

Return the tangent of a. The optional `form` argument allows specifying the representation. In `:default` form, the result is determined by the `:trig_form` option of the parent object. In `:exponential` form, the value is represented using complex exponentials. In `:direct` or `:tangent` form, the value is represented directly using tangents. In `:sine_cosine` form, the value is represented using sines or cosines.

source

Base.atan – Method.

```
atan(a::CalciumFieldElem; form::Symbol=:default)
```

Return the inverse tangent of a. The optional `form` argument allows specifying the representation. In `:default` form, the result is determined by the `:trig_form` option of the parent object. In `:logarithm` form, the value is represented using complex logarithms. In `:direct` or `:arctangent` form, the value is represented directly using arctangents.

source

Base.asin – Method.

```
asin(a::CalciumFieldElem; form::Symbol=:default)
```

Return the inverse sine of a. The optional `form` argument allows specifying the representation. In `:default` form, the result is determined by the `:trig_form` option of the parent object. In `:logarithm` form, the value is represented using complex logarithms. In `:direct` form, the value is represented directly using an inverse sine or cosine.

source

Base.acos – Method.

```
acos(a::CalciumFieldElem; form::Symbol=:default)
```

Return the inverse cosine of a. The optional `form` argument allows specifying the representation. In `:default` form, the result is determined by the `:trig_form` option of the parent object. In `:logarithm` form, the value is represented using complex logarithms. In `:direct` form, the value is represented directly using an inverse sine or cosine.

source

Nemo.gamma – Method.

```
gamma(a::CalciumFieldElem)
```

Return the gamma function of a.

source

Nemo.erf – Method.

```
erf(a::CalciumFieldElem)
```

Return the error function of a.

source

`Nemo.erfi` – Method.

```
erfi(a::CalciumFieldElem)
```

Return the imaginary error function of a.

source

`Nemo.erfc` – Method.

```
erfc(a::CalciumFieldElem)
```

Return the complementary error function of a.

source

## 21.8 Rewriting and simplification

`Nemo.complex_normal_form` – Method.

```
complex_normal_form(a::CalciumFieldElem, deep::Bool=true)
```

Returns the input rewritten using standardizing transformations over the complex numbers:

- Elementary functions are rewritten in terms of exponentials, roots and logarithms.
- Complex parts are rewritten using logarithms, square roots, and (deep) complex conjugates.
- Algebraic numbers are rewritten in terms of cyclotomic fields where applicable.

If deep is set, the rewriting is applied recursively to the tower of extension numbers; otherwise, the rewriting is only applied to the top-level extension numbers.

The result is not a normal form in the strong sense (the same number can have many possible representations even after applying this transformation), but this transformation can nevertheless be a useful heuristic for simplification.

source

# Chapter 22

# Arbitrary precision complex balls

Arbitrary precision complex ball arithmetic is supplied by FLINT which provides a ball representation which tracks error bounds rigorously. Complex numbers are represented in rectangular form $a + bi$ where $a, b$ are `RealFieldElem` balls.

The corresponding field is constructed using the `ComplexField` constructor. This constructs the parent object for the complex field.

The types of complex boxes in Nemo are given in the following table, along with the libraries that provide them and the associated types of the parent objects.

| Library | Field | Element type | Parent type |
|---|---|---|---|
| FLINT | $\mathbb{C}$ (boxes) | `ComplexFieldElem` | `ComplexField` |

All the complex field types belong to the `Field` abstract type and the types of elements in this field, i.e. complex boxes in this case, belong to the `FieldElem` abstract type.

## 22.1 Complex ball functionality

The complex balls in Nemo provide all the field functionality defined by AbstractAlgebra:.

https://nemocas.github.io/AbstractAlgebra.jl/stable/field

Below, we document the additional functionality provided for complex balls.

### Precision management

See Precision management.

### Complex field constructors

In order to construct complex boxes in Nemo, one must first construct the complex field itself. This is accomplished with the following constructor.

`Nemo.complex_field` – Method.

```
complex_field()
```

Return the field of complex numbers modelled via complex balls.

See `precision` and `set_precision!` on how to control the precision.

source

Here is an example of creating the complex field and using the resulting parent object to coerce values into the resulting field.

**Examples**

```
julia> CC = complex_field()
Complex field

julia> a = CC("0.25")
0.25000000000000000000

julia> b = CC("0.1")
[0.10000000000000000000 +/- 2.72e-21]

julia> c = CC(0.5)
0.50000000000000000000

julia> d = CC(12)
12.000000000000000000
```

Note that whilst one can coerce double precision floating point values into a complex field, unless those values can be represented exactly in double precision the resulting ball can't be any more precise than the double precision supplied.

If instead, values can be represented precisely using decimal arithmetic then one can supply them to the complex field using a string. In this case, the complex field will store them to the global ball precision.

If the values can be stored precisely as a binary floating point number, the complex field will store the values exactly. See the function `is_exact` below for more information.

### Constructors

`Nemo.onei` – Method.

```
onei(r::ComplexField)
```

Return exact one times $i$ in the given complex field.

source

**Examples**

```
julia> c = onei(CC)
1.0000000000000000000*im
```

## 22.2 Basic functionality

The following basic functionality is provided by the complex field implementation in Nemo, to support construction of generic rings over the complex field.

```
parent_type(::Type{ComplexFieldElem})
```

Gives the type of the parent object of a complex field element.

```
elem_type(R::ComplexField)
```

Given the parent object for a complex field, return the type of elements of the field.

```
mul!(c::ComplexFieldElem, a::ComplexFieldElem, b::ComplexFieldElem)
```

Multiply $a$ by $b$ and set the existing complex field element $c$ to the result. This function is provided for performance reasons as it saves allocating a new object for the result and eliminates associated garbage collection.

```
deepcopy(a::ComplexFieldElem)
```

Return a copy of the complex field element $a$, recursively copying the internal data. Complex field elements are mutable in Nemo so a shallow copy is not sufficient.

Given the parent object `C` for a complex field, the following coercion functions are provided to coerce various elements into the complex field.

```
C()
```

Coerce zero into the complex field.

```
C(n::Integer)
C(f::ZZRingElem)
C(q::QQFieldElem)
```

Coerce an integer or rational value into the complex field.

```
C(f::Float64)
C(f::BigFloat)
```

Coerce the given floating point number into the complex field.

```
C(f::AbstractString)
C(f::AbstractString, g::AbstractString)
```

Coerce the decimal number, given as a string, into the complex field. In each case $f$ is the real part and $g$ is the imaginary part.

```
C(f::RealFieldElem)
```

Coerce the given real ball into the complex field.

```
C(f::ComplexFieldElem)
```

Take a complex field element that is already in a complex field and simply return it. A copy of the original is not made.

Here are some examples of coercing elements into the complex field.

```
julia> RR = real_field()
Real field

julia> CC = complex_field()
Complex field

julia> a = CC(3)
3.0000000000000000000

julia> b = CC(QQ(2,3))
[0.66666666666666666 +/- 8.48e-20]

julia> c = CC("3 +/- 0.0001")
[3.000 +/- 1.01e-4]

julia> d = CC("-1.24e+12345")
[-1.2400000000000000000e+12345 +/- 4.16e+12325]

julia> f = CC("nan +/- inf")
nan

julia> g = CC(RR(3))
3.0000000000000000000
```

In addition to the above, developers of custom complex field types must ensure that they provide the equivalent of the function `base_ring(R::ComplexField)` which should return `Union{}`. In addition to this they should ensure that each complex field element contains a field `parent` specifying the parent object of the complex field element, or at least supply the equivalent of the function `parent(a::ComplexFieldElem)` to return the parent object of a complex field element.

### Basic manipulation

`Base.isfinite` – Method.

```
isfinite(x::ComplexFieldElem)
```

Return `true` if $x$ is finite, i.e. its real and imaginary parts have finite midpoint and radius, otherwise return `false`.

source

`Nemo.is_exact` – Method.

```
is_exact(x::ComplexFieldElem)
```

Return `true` if $x$ is exact, i.e. has its real and imaginary parts have zero radius, otherwise return `false`.

source

`Base.isinteger` – Method.

```
isinteger(x::ComplexFieldElem)
```

Return `true` if $x$ is an exact integer, otherwise return `false`.

source

`Nemo.accuracy_bits` – Method.

```
accuracy_bits(x::ComplexFieldElem)
```

Return the relative accuracy of $x$ measured in bits, capped between `typemax(Int)` and `-typemax(Int)`.

source

**Examples**

```
julia> a = CC("1.2 +/- 0.001")
[1.20 +/- 1.01e-3]

julia> b = CC(3)
3.0000000000000000000

julia> isreal(a)
true

julia> isfinite(b)
true

julia> isinteger(b)
true

julia> c = real(a)
[1.20 +/- 1.01e-3]

julia> d = imag(b)
0

julia> f = accuracy_bits(a)
9
```

### Containment

It is often necessary to determine whether a given exact value or box is contained in a given complex box or whether two boxes overlap. The following functions are provided for this purpose.

`Nemo.overlaps` – Method.

```
overlaps(x::ComplexFieldElem, y::ComplexFieldElem)
```

Returns `true` if any part of the box $x$ overlaps any part of the box $y$, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::ComplexFieldElem, y::ComplexFieldElem)
```

Returns `true` if the box $x$ contains the box $y$, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::ComplexFieldElem, y::Integer)
```

Returns `true` if the box $x$ contains the given integer value, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::ComplexFieldElem, y::ZZRingElem)
```

Returns `true` if the box $x$ contains the given integer value, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::ComplexFieldElem, y::QQFieldElem)
```

Returns `true` if the box $x$ contains the given rational value, otherwise return `false`.

source

The following functions are also provided for determining if a box intersects a certain part of the complex number plane.

`Nemo.contains_zero` – Method.

```
contains_zero(x::ComplexFieldElem)
```

Returns `true` if the box $x$ contains zero, otherwise return `false`.

source

**Examples**

```
julia> x = CC("1 +/- 0.001")
[1.00 +/- 1.01e-3]

julia> y = CC("3")
3.000000000000000000

julia> overlaps(x, y)
false

julia> contains(x, y)
false

julia> contains(y, 3)
true

julia> contains(x, ZZ(1)//2)
false

julia> contains_zero(x)
false
```

### Comparison

Nemo provides a full range of comparison operations for complex boxes.

In addition to the standard comparisons, we introduce an exact equality. This is distinct from arithmetic equality implemented by ==, which merely compares up to the minimum of the precisions of its operands.

`Base.isequal` – Method.

```
isequal(x::ComplexFieldElem, y::ComplexFieldElem)
```

Return `true` if the boxes $x$ and $y$ are precisely equal, i.e. their real and imaginary parts have the same midpoints and radii.

source

A full range of ad hoc comparison operators is provided. These are implemented directly in Julia, but we document them as though only == were provided.

**Examples**

| Function |
|---|
| ==(x::ComplexFieldElem, y::Integer) |
| ==(x::Integer, y::ComplexFieldElem) |
| ==(x::ComplexFieldElem, y::ZZRingElem) |
| ==(x::ZZRingElem, y::ComplexFieldElem) |
| ==(x::RealFieldElem, y::ZZRingElem) |
| ==(x::ZZRingElem, y::RealFieldElem) |
| ==(x::ComplexFieldElem, y::Float64) |
| ==(x::Float64, y::ComplexFieldElem) |

```
julia> x = CC("1 +/- 0.001")
[1.00 +/- 1.01e-3]

julia> y = CC("3")
3.0000000000000000000

julia> z = CC("4")
4.0000000000000000000

julia> isequal(x, deepcopy(x))
true

julia> x == 3
false

julia> ZZ(3) == y
true

julia> z != 1.23
true
```

### Absolute value

#### Examples

```
julia> x = CC("-1 +/- 0.001")
[-1.00 +/- 1.01e-3]

julia> a = abs(x)
[1.00 +/- 1.01e-3]
```

### Shifting

#### Examples

```
julia> x = CC("-3 +/- 0.001")
[-3.00 +/- 1.01e-3]

julia> a = ldexp(x, 23)
```

```
[-2.52e+7 +/- 4.26e+4]

julia> b = ldexp(x, -ZZ(15))
[-9.16e-5 +/- 7.78e-8]
```

### Miscellaneous operations

`Nemo.trim` – Method.

```
trim(x::ComplexFieldElem)
```

Return an `ComplexFieldElem` box containing $x$ but which may be more economical, by rounding off insignificant bits from midpoints.

source

`Nemo.unique_integer` – Method.

```
unique_integer(x::ComplexFieldElem)
```

Return a pair where the first value is a boolean and the second is an `ZZRingElem` integer. The boolean indicates whether the box $x$ contains a unique integer. If this is the case, the second return value is set to this unique integer.

source

#### Examples

```
julia> x = CC("-3 +/- 0.001", "0.1")
[-3.00 +/- 1.01e-3] + [0.10000000000000000000 +/- 2.72e-21]*im

julia> a = trim(x)
[-3.00 +/- 1.01e-3] + [0.10000000000000000000 +/- 2.72e-21]*im

julia> b, c = unique_integer(x)
(false, 0)

julia> d = conj(x)
[-3.00 +/- 1.01e-3] + [-0.10000000000000000000 +/- 2.72e-21]*im

julia> f = angle(x)
[3.1083 +/- 3.95e-5]
```

### Constants

`Nemo.const_pi` – Method.

```
const_pi(r::ComplexField)
```

Return $\pi = 3.14159\ldots$ as an element of $r$.

source

**Examples**

```
julia> CC = complex_field()
Complex field

julia> a = const_pi(CC)
[3.141592653589793239 +/- 5.96e-19]

julia> set_precision!(ComplexField, 200) do
         println(const_pi(CC))
       end
[3.14159265358979323846264338327950288419716939937510582097494 +/- 5.73e-60]
```

### Mathematical and special functions

Nemo.rsqrt – Method.

```
rsqrt(x::ComplexFieldElem)
```

Return the reciprocal of the square root of $x$, i.e. $1/\sqrt{x}$.

source

Base.cispi – Method.

```
cispi(x::ComplexFieldElem)
```

Return the exponential of $\pi i x$.

source

Nemo.root_of_unity – Method.

```
root_of_unity(C::ComplexField, k::Int)
```

Return $\exp(2\pi i/k)$.

source

Nemo.log_sinpi – Method.

```
log_sinpi(x::ComplexFieldElem)
```

Return $\log \sin(\pi x)$, constructed without branch cuts off the real line.

source

Nemo.gamma – Method.

```
gamma(x::ComplexFieldElem)
```

Return the Gamma function evaluated at $x$.

source

Nemo.lgamma – Method.

```
lgamma(x::ComplexFieldElem)
```

Return the logarithm of the Gamma function evaluated at $x$.

source

Nemo.rgamma – Method.

```
rgamma(x::ComplexFieldElem)
```

Return the reciprocal of the Gamma function evaluated at $x$.

source

Nemo.digamma – Method.

```
digamma(x::ComplexFieldElem)
```

Return the logarithmic derivative of the gamma function evaluated at $x$, i.e. $\psi(x)$.

source

Nemo.zeta – Method.

```
zeta(x::ComplexFieldElem)
```

Return the Riemann zeta function evaluated at $x$.

source

Nemo.barnes_g – Method.

```
barnes_g(x::ComplexFieldElem)
```

Return the Barnes $G$-function, evaluated at $x$.

source

Nemo.log_barnes_g – Method.

```
log_barnes_g(x::ComplexFieldElem)
```

Return the logarithm of the Barnes $G$-function, evaluated at $x$.

source

Nemo.erf – Method.

```
erf(x::ComplexFieldElem)
```

Return the error function evaluated at $x$.

source

Nemo.erfi – Method.

```
erfi(x::ComplexFieldElem)
```

Return the imaginary error function evaluated at $x$.

source

Nemo.exp_integral_ei – Method.

```
exp_integral_ei(x::ComplexFieldElem)
```

Return the exponential integral evaluated at $x$.

source

Nemo.sin_integral – Method.

```
sin_integral(x::ComplexFieldElem)
```

Return the sine integral evaluated at $x$.

source

Nemo.cos_integral – Method.

```
cos_integral(x::ComplexFieldElem)
```

Return the exponential cosine integral evaluated at $x$.

source

Nemo.sinh_integral – Method.

```
sinh_integral(x::ComplexFieldElem)
```

Return the hyperbolic sine integral evaluated at $x$.

source

Nemo.cosh_integral – Method.

```
cosh_integral(x::ComplexFieldElem)
```

Return the hyperbolic cosine integral evaluated at $x$.

source

Nemo.dedekind_eta – Method.

```
dedekind_eta(x::ComplexFieldElem)
```

Return the Dedekind eta function $\eta(\tau)$ at $\tau = x$.

source

Nemo.modular_weber_f – Method.

```
modular_weber_f(x::ComplexFieldElem)
```

Return the modular Weber function $\mathfrak{f}(\tau) = \frac{\eta^2(\tau)}{\eta(\tau/2)\eta(2\tau)}$, at $x$ in the complex upper half plane.

source

Nemo.modular_weber_f1 – Method.

```
modular_weber_f1(x::ComplexFieldElem)
```

Return the modular Weber function $\mathfrak{f}_1(\tau) = \frac{\eta(\tau/2)}{\eta(\tau)}$, at $x$ in the complex upper half plane.

source

Nemo.modular_weber_f2 – Method.

```
modular_weber_f2(x::ComplexFieldElem)
```

Return the modular Weber function $\mathfrak{f}_2(\tau) = \frac{\sqrt{2}\eta(2\tau)}{\eta(\tau)}$ at $x$ in the complex upper half plane.

source

Nemo.j_invariant – Method.

```
j_invariant(x::ComplexFieldElem)
```

Return the $j$-invariant $j(\tau)$ at $\tau = x$.

source

Nemo.modular_lambda – Method.

```
modular_lambda(x::ComplexFieldElem)
```

Return the modular lambda function $\lambda(\tau)$ at $\tau = x$.

source

Nemo.modular_delta – Method.

```
modular_delta(x::ComplexFieldElem)
```

Return the modular delta function $\Delta(\tau)$ at $\tau = x$.

source

Nemo.eisenstein_g – Method.

```
eisenstein_g(k::Int, x::ComplexFieldElem)
```

Return the non-normalized Eisenstein series $G_k(\tau)$ of $\mathrm{SL}_2(\mathbb{Z})$. Also defined for $\tau = i\infty$.

source

Nemo.hilbert_class_polynomial – Method.

```
hilbert_class_polynomial(D::Int, R::ZZPolyRing)
```

Return in the ring $R$ the Hilbert class polynomial of discriminant $D$, which is only defined for $D < 0$ and $D \equiv 0, 1 \pmod 4$.

source

Nemo.elliptic_k – Method.

```
elliptic_k(x::ComplexFieldElem)
```

Return the complete elliptic integral $K(x)$.

source

Nemo.elliptic_e – Method.

```
elliptic_e(x::ComplexFieldElem)
```

Return the complete elliptic integral $E(x)$.

source

Nemo.agm – Method.

```
agm(x::ComplexFieldElem)
```

Return the arithmetic-geometric mean of $1$ and $x$.

source

Nemo.agm – Method.

```
agm(x::ComplexFieldElem, y::ComplexFieldElem)
```

Return the arithmetic-geometric mean of $x$ and $y$.

source

Nemo.polygamma – Method.

```
polygamma(s::ComplexFieldElem, a::ComplexFieldElem)
```

Return the generalised polygamma function $\psi(s, z)$.

source

Nemo.zeta – Method.

```
zeta(s::ComplexFieldElem, a::ComplexFieldElem)
```

Return the Hurwitz zeta function $\zeta(s, a)$.

source

AbstractAlgebra.Generic.rising_factorial – Method.

```
rising_factorial(x::ComplexFieldElem, n::Int)
```

Return the rising factorial $x(x+1)\ldots(x+n-1)$.

source

AbstractAlgebra.Generic.rising_factorial2 – Method.

```
rising_factorial2(x::ComplexFieldElem, n::Int)
```

Return a tuple containing the rising factorial $x(x+1)\ldots(x+n-1)$ and its derivative.

source

Nemo.polylog – Method.

```
polylog(s::Union{ComplexFieldElem,Int}, a::ComplexFieldElem)
```

Return the polylogarithm $\mathrm{Li}_s(a)$.

source

Nemo.log_integral – Method.

```
log_integral(x::ComplexFieldElem)
```

Return the logarithmic integral, evaluated at $x$.

source

Nemo.log_integral_offset – Method.

```
log_integral_offset(x::ComplexFieldElem)
```

Return the offset logarithmic integral, evaluated at $x$.

source

Nemo.exp_integral_e – Method.

```
exp_integral_e(s::ComplexFieldElem, x::ComplexFieldElem)
```

Return the generalised exponential integral $E_s(x)$.

source

Nemo.gamma – Method.

```
gamma(s::ComplexFieldElem, x::ComplexFieldElem)
```

Return the upper incomplete gamma function $\Gamma(s, x)$.

source

Nemo.gamma_regularized – Method.

```
gamma_regularized(s::ComplexFieldElem, x::ComplexFieldElem)
```

Return the regularized upper incomplete gamma function $\Gamma(s, x)/\Gamma(s)$.

source

Nemo.gamma_lower – Method.

```
gamma_lower(s::ComplexFieldElem, x::ComplexFieldElem)
```

Return the lower incomplete gamma function $\gamma(s, x)/\Gamma(s)$.

source

Nemo.gamma_lower_regularized – Method.

```
gamma_lower_regularized(s::ComplexFieldElem, x::ComplexFieldElem)
```

Return the regularized lower incomplete gamma function $\gamma(s, x)/\Gamma(s)$.

source

Nemo.airy_ai – Method.

```
airy_ai(x::ComplexFieldElem)
```

Return the Airy function $\mathrm{Ai}(x)$.

source

Nemo.airy_ai_prime – Method.

```
airy_ai_prime(x::ComplexFieldElem)
```

Return the derivative of the Airy function $\mathrm{Ai}'(x)$.

source

Nemo.airy_bi – Method.

```
airy_bi(x::ComplexFieldElem)
```

Return the Airy function $\mathrm{Bi}(x)$.

source

Nemo.airy_bi_prime – Method.

```
airy_bi_prime(x::ComplexFieldElem)
```

Return the derivative of the Airy function $\mathrm{Bi}'(x)$.

source

Nemo.bessel_j – Method.

```
bessel_j(nu::ComplexFieldElem, x::ComplexFieldElem)
```

Return the Bessel function $J_\nu(x)$.

source

Nemo.bessel_y – Method.

```
bessel_y(nu::ComplexFieldElem, x::ComplexFieldElem)
```

Return the Bessel function $Y_\nu(x)$.

source

Nemo.bessel_i – Method.

```
bessel_i(nu::ComplexFieldElem, x::ComplexFieldElem)
```

Return the Bessel function $I_\nu(x)$.

source

Nemo.bessel_k – Method.

```
bessel_k(nu::ComplexFieldElem, x::ComplexFieldElem)
```

Return the Bessel function $K_\nu(x)$.

source

Nemo.hypergeometric_1f1 – Method.

```
hypergeometric_1f1(a::ComplexFieldElem, b::ComplexFieldElem, x::ComplexFieldElem)
```

Return the confluent hypergeometric function ${}_1F_1(a, b, x)$.

source

Nemo.hypergeometric_1f1_regularized – Method.

```
hypergeometric_1f1_regularized(a::ComplexFieldElem, b::ComplexFieldElem, x::ComplexFieldElem)
```

Return the regularized confluent hypergeometric function ${}_1F_1(a, b, x)/\Gamma(b)$.

source

Nemo.hypergeometric_u – Method.

```
hypergeometric_u(a::ComplexFieldElem, b::ComplexFieldElem, x::ComplexFieldElem)
```

Return the confluent hypergeometric function $U(a, b, x)$.

source

Nemo.hypergeometric_2f1 – Method.

```
hypergeometric_2f1(a::ComplexFieldElem, b::ComplexFieldElem, c::ComplexFieldElem,
↪  x::ComplexFieldElem; flags=0)
```

Return the Gauss hypergeometric function ${}_2F_1(a, b, c, x)$.

source

Nemo.jacobi_theta – Method.

```
jacobi_theta(z::ComplexFieldElem, tau::ComplexFieldElem)
```

Return a tuple of four elements containing the Jacobi theta function values $\theta_1, \theta_2, \theta_3, \theta_4$ evaluated at $z, \tau$.

source

Nemo.weierstrass_p – Method.

```
weierstrass_p(z::ComplexFieldElem, tau::ComplexFieldElem)
```

Return the Weierstrass elliptic function $\wp(z, \tau)$.

source

**Examples**

```
julia> s = CC(1, 2)
1.0000000000000000000 + 2.0000000000000000000*im

julia> z = CC("1.23", "3.45")
[1.2300000000000000000 +/- 3.49e-20] + [3.4500000000000000000 +/- 8.70e-20]*im

julia> a = sin(z)^2 + cos(z)^2
[1.00000000000000 +/- 2.76e-16] + [+/- 2.11e-16]*im

julia> b = zeta(z)
[0.685803329024164062 +/- 2.05e-19] + [-0.038574782404586856 +/- 2.71e-19]*im

julia> c = bessel_j(s, z)
[0.63189634741402481 +/- 4.04e-18] + [0.00970090757446076 +/- 3.79e-18]*im

julia> d = hypergeometric_1f1(s, s+1, z)
[-1.3355297330012291 +/- 4.42e-17] + [-0.1715020340928697 +/- 3.61e-17]*im
```

### Linear dependence

Nemo.lindep – Method.

```
lindep(A::Vector{ComplexFieldElem}, bits::Int)
```

Find a small linear combination of the entries of the array $A$ that is small (using LLL). The entries are first scaled by the given number of bits before truncating the real and imaginary parts to integers for use in LLL. This function can be used to find linear dependence between a list of complex numbers. The algorithm is heuristic only and returns an array of Nemo integers representing the linear combination.

source

Nemo.lindep – Method.

```
lindep(A::Matrix{ComplexFieldElem}, bits::Int)
```

Find a (common) small linear combination of the entries in each row of the array $A$, that is small (using LLL). It is assumed that the complex numbers in each row of the array share the same linear combination. The entries are first scaled by the given number of bits before truncating the real and imaginary parts to integers for use in LLL. This function can be used to find a common linear dependence shared across a number of lists of complex numbers. The algorithm is heuristic only and returns an array of Nemo integers representing the common linear combination.

source

#### Examples

```
julia> # These are two of the roots of x^5 + 3x + 1

julia> a = CC(1.005066947858862242879105188836477525 3, -0.937259156692891826979035858687615135 85)
[1.0050669478588623029 +/- 2.25e-20] - [0.93725915669289183718 +/- 1.50e-21]*im
```

```
julia> b = CC(-0.331989029584509316202500694922316523 19)
-[0.33198902958450932088 +/- 4.15e-22]

julia> V1 = [CC(1), a, a^2, a^3, a^4, a^5]; # We recover the polynomial from one root....

julia> W = lindep(V1, 20)
6-element Vector{ZZRingElem}:
 1
 3
 0
 0
 0
 1

julia> V2 = [CC(1), b, b^2, b^3, b^4, b^5]; # ...or from two

julia> Vs = [transpose(V1); transpose(V2)];

julia> X = lindep(Vs, 20)
6-element Vector{ZZRingElem}:
 1
 3
 0
 0
 0
 1
```

# Chapter 23

# Arbitrary precision real balls

Arbitrary precision real ball arithmetic is supplied by FLINT which provides a ball representation which tracks error bounds rigorously. Real numbers are represented in mid-rad interval form $[m \pm r] = [m - r, m + r]$.

The types of real balls in Nemo are given in the following table, along with the libraries that provide them and the associated types of the parent objects.

| Library | Field | Element type | Parent type |
|---|---|---|---|
| FLINT | $\mathbb{R}$ (balls) | `RealFieldElem` | `RealField` |

The real field types belong to the `Field` abstract type and the types of elements in this field, i.e. balls in this case, belong to the `FieldElem` abstract type.

## 23.1 Real ball functionality

Real balls in Nemo provide all the field functionality described in AbstractAlgebra:

https://nemocas.github.io/AbstractAlgebra.jl/stable/field

Below, we document the additional functionality provided for real balls.

### Precision management

Precision for ball arithmetic and creation of elements can be controlled using the functions:

`Base.precision` – Method.

```
precision(::Type{Balls})
```

Return the precision for ball arithmetic.

**Examples**

```
julia> set_precision!(Balls, 200); precision(Balls)
200
```

source

AbstractAlgebra.set_precision! – Method.

```
set_precision!(::Type{Balls}, n::Int)
```

Set the precision for all ball arithmetic to be n.

**Examples**

```
julia> const_pi(real_field())
[3.141592653589793239 +/- 5.96e-19]

julia> set_precision!(Balls, 200); const_pi(real_field())
[3.14159265358979323846264338327950288419716939937510582097494 +/- 5.73e-60]
```

source

AbstractAlgebra.set_precision! – Method.

```
set_precision!(f, ::Type{Balls}, n::Int)
```

Change ball arithmetic precision to n for the duration of f..

**Examples**

```
julia> set_precision!(Balls, 4) do
         const_pi(real_field())
       end
[3e+0 +/- 0.376]

julia> set_precision!(Balls, 200) do
         const_pi(real_field())
       end
[3.1415926535897932385 +/- 3.74e-20]
```

source

**Info**

These functions are not thread-safe.

### Constructors

In order to construct real balls in Nemo, one must first construct the real field itself. This is accomplished with the following constructor.

Nemo.real_field – Method.

```
real_field()
```

Return the field of real numbers modelled via real balls.

See `precision` and `set_precision!` on how to control the precision.

source

Here is an example of creating the real field and using the resulting parent object to coerce values into the resulting field.

**Examples**

```
julia> RR = real_field()
Real field

julia> a = RR("0.25")
0.25000000000000000000

julia> b = RR("0.1 +/- 0.001")
[0.10 +/- 1.01e-3]

julia> c = RR(0.5)
0.50000000000000000000

julia> d = RR(12)
12.000000000000000000
```

Note that whilst one can coerce double precision floating point values into a real field, unless those values can be represented exactly in double precision the resulting ball can't be any more precise than the double precision supplied.

If instead, values can be represented precisely using decimal arithmetic then one can supply them to the real field using a string. In this case, the real field will store them to the global ball precision.

If the values can be stored precisely as a binary floating point number, the real field will store the values exactly. See the function `is_exact` below for more information.

### Real ball constructors

Using coercion into the real field, new elements can be created.

**Examples**

```
julia> c = RR(1)
1.0000000000000000000

julia> d = RR(1//2)
0.50000000000000000000
```

Note that for the construction, also the precision can be supplied:

```
julia> c = RR(1//3, precision=100)
[0.33333333333333333333 +/- 3.34e-21]

julia> d = RR(1//3, precision=4)
[0.3 +/- 0.0438]
```

### Conversions

```
julia> convert(Float64, RR(1//3))
0.3333333333333333
```

### Basic manipulation

Nemo.is_nonzero – Method.

```
is_nonzero(x::RealFieldElem)
```

Return `true` if $x$ is certainly not equal to zero, otherwise return `false`.

source

Base.isfinite – Method.

```
isfinite(x::RealFieldElem)
```

Return `true` if $x$ is finite, i.e. having finite midpoint and radius, otherwise return `false`.

source

Nemo.is_exact – Method.

```
is_exact(x::RealFieldElem)
```

Return `true` if $x$ is exact, i.e. has zero radius, otherwise return `false`.

source

Base.isinteger – Method.

```
isinteger(x::RealFieldElem)
```

Return `true` if $x$ is an exact integer, otherwise return `false`.

source

AbstractAlgebra.is_positive – Method.

```
is_positive(x::RealFieldElem)
```

Return `true` if $x$ is certainly positive, otherwise return `false`.

source

Nemo.is_nonnegative – Method.

```
is_nonnegative(x::RealFieldElem)
```

Return `true` if $x$ is certainly non-negative, otherwise return `false`.

source

`AbstractAlgebra.is_negative` – Method.

```
is_negative(x::RealFieldElem)
```

Return `true` if $x$ is certainly negative, otherwise return `false`.

source

`Nemo.is_nonpositive` – Method.

```
is_nonpositive(x::RealFieldElem)
```

Return `true` if $x$ is certainly nonpositive, otherwise return `false`.

source

`Nemo.midpoint` – Method.

```
midpoint(x::RealFieldElem)
```

Return the midpoint of the ball $x$ as an Arb ball.

source

`Nemo.radius` – Method.

```
radius(x::RealFieldElem)
```

Return the radius of the ball $x$ as an Arb ball.

source

`Nemo.accuracy_bits` – Method.

```
accuracy_bits(x::RealFieldElem)
```

Return the relative accuracy of $x$ measured in bits, capped between `typemax(Int)` and `-typemax(Int)`.

source

**Examples**

```
julia> RR = real_field()
Real field

julia> a = RR("1.2 +/- 0.001")
[1.20 +/- 1.01e-3]

julia> b = RR(3)
3.0000000000000000000

julia> is_positive(a)
true

julia> isfinite(b)
true

julia> isinteger(b)
true

julia> is_negative(a)
false

julia> c = radius(a)
[0.0010000000038417056203 +/- 1.12e-23]

julia> d = midpoint(b)
3.0000000000000000000

julia> f = accuracy_bits(a)
9
```

### Printing

Printing real balls can at first sight be confusing. Lets look at the following example:

```
julia> a = RR(1)
1.0000000000000000000

julia> b = RR(2)
2.0000000000000000000

julia> c = RR(12)
12.000000000000000000

julia> x = ball(a, b)
[+/- 3.01]

julia> y = ball(c, b)
[1e+1 +/- 4.01]

julia> mid = midpoint(x)
1.0000000000000000000

julia> rad = radius(x)
[2.0000000037252902985 +/- 3.81e-20]
```

```
julia> print(x, "\n", y, "\n", mid, "\n", rad)
[+/- 3.01]
[1e+1 +/- 4.01]
1.0000000000000000000
[2.0000000037252902985 +/- 3.81e-20]
```

The first reason that x is not printed as `[1 +/- 2]` is that the midpoint does not have a greater exponent than the radius in its scientific notation. For similar reasons y is not printed as `[12 +/- 2]`.

The second reason is that we get an additional error term after our addition. As we see, `radius(x)` is not equal to $2$, which when printed rounds it up to a reasonable decimal place. This is because real balls keep track of rounding errors of basic arithmetic.

### Containment

It is often necessary to determine whether a given exact value or ball is contained in a given real ball or whether two balls overlap. The following functions are provided for this purpose.

`Nemo.overlaps` – Method.

```
overlaps(x::RealFieldElem, y::RealFieldElem)
```

Returns `true` if any part of the ball $x$ overlaps any part of the ball $y$, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::RealFieldElem, y::RealFieldElem)
```

Returns `true` if the ball $x$ contains the ball $y$, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::RealFieldElem, y::Integer)
```

Returns `true` if the ball $x$ contains the given integer value, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::RealFieldElem, y::ZZRingElem)
```

Returns `true` if the ball $x$ contains the given integer value, otherwise return `false`.

source

Base.contains – Method.

```
contains(x::RealFieldElem, y::QQFieldElem)
```

Returns true if the ball $x$ contains the given rational value, otherwise return false.

source

Base.contains – Method.

```
contains(x::RealFieldElem, y::Rational{T}) where {T <: Integer}
```

Returns true if the ball $x$ contains the given rational value, otherwise return false.

source

Base.contains – Method.

```
contains(x::RealFieldElem, y::BigFloat)
```

Returns true if the ball $x$ contains the given floating point value, otherwise return false.

source

The following functions are also provided for determining if a ball intersects a certain part of the real number line.

Nemo.contains_zero – Method.

```
contains_zero(x::RealFieldElem)
```

Returns true if the ball $x$ contains zero, otherwise return false.

source

Nemo.contains_negative – Method.

```
contains_negative(x::RealFieldElem)
```

Returns true if the ball $x$ contains any negative value, otherwise return false.

source

Nemo.contains_positive – Method.

```
contains_positive(x::RealFieldElem)
```

Returns true if the ball $x$ contains any positive value, otherwise return false.

source

Nemo.contains_nonnegative – Method.

```
contains_nonnegative(x::RealFieldElem)
```

Returns true if the ball $x$ contains any non-negative value, otherwise return false.

source

Nemo.contains_nonpositive – Method.

```
contains_nonpositive(x::RealFieldElem)
```

Returns true if the ball $x$ contains any nonpositive value, otherwise return false.

source

**Examples**

```
julia> x = RR("1 +/- 0.001")
[1.00 +/- 1.01e-3]

julia> y = RR("3")
3.0000000000000000000

julia> overlaps(x, y)
false

julia> contains(x, y)
false

julia> contains(y, 3)
true

julia> contains(x, ZZ(1)//2)
false

julia> contains_zero(x)
false

julia> contains_positive(y)
true
```

### Comparison

Nemo provides a full range of comparison operations for real balls. Note that a ball is considered less than another ball if every value in the first ball is less than every value in the second ball, etc.

In addition to the standard comparison operators, we introduce an exact equality. This is distinct from arithmetic equality implemented by ==, which merely compares up to the minimum of the precisions of its operands.

Base.isequal – Method.

```
isequal(x::RealFieldElem, y::RealFieldElem)
```

Return `true` if the balls $x$ and $y$ are precisely equal, i.e. have the same midpoints and radii.

source

We also provide a full range of ad hoc comparison operators. These are implemented directly in Julia, but we document them as though `isless` and == were provided.

| Function |
|---|
| ==(x::RealFieldElem, y::Integer) |
| ==(x::Integer, y::RealFieldElem) |
| ==(x::RealFieldElem, y::ZZRingElem) |
| ==(x::ZZRingElem, y::RealFieldElem) |
| ==(x::RealFieldElem, y::Float64) |
| ==(x::Float64, y::RealFieldElem) |
| isless(x::RealFieldElem, y::Integer) |
| isless(x::Integer, y::RealFieldElem) |
| isless(x::RealFieldElem, y::ZZRingElem) |
| isless(x::ZZRingElem, y::RealFieldElem) |
| isless(x::RealFieldElem, y::Float64) |
| isless(x::Float64, y::RealFieldElem) |
| isless(x::RealFieldElem, y::BigFloat) |
| isless(x::BigFloat, y::RealFieldElem) |
| isless(x::RealFieldElem, y::QQFieldElem) |
| isless(x::QQFieldElem, y::RealFieldElem) |

**Examples**

```
julia> x = RR("1 +/- 0.001")
[1.00 +/- 1.01e-3]

julia> y = RR("3")
3.0000000000000000000

julia> z = RR("4")
4.0000000000000000000

julia> isequal(x, deepcopy(x))
true

julia> x == 3
false

julia> ZZ(3) < z
true

julia> x != 1.23
true
```

### Absolute value

#### Examples

```
julia> x = RR("-1 +/- 0.001")
[-1.00 +/- 1.01e-3]

julia> a = abs(x)
[1.00 +/- 1.01e-3]
```

### Shifting

#### Examples

```
julia> x = RR("-3 +/- 0.001")
[-3.00 +/- 1.01e-3]

julia> a = ldexp(x, 23)
[-2.52e+7 +/- 4.26e+4]

julia> b = ldexp(x, -ZZ(15))
[-9.16e-5 +/- 7.78e-8]
```

### Miscellaneous operations

Nemo.add_error! – Method.

```
add_error!(x::RealFieldElem, y::RealFieldElem)
```

Adds the absolute values of the midpoint and radius of $y$ to the radius of $x$.

source

Nemo.trim – Method.

```
trim(x::RealFieldElem)
```

Return an RealFieldElem interval containing $x$ but which may be more economical, by rounding off insignificant bits from the midpoint.

source

Nemo.unique_integer – Method.

```
unique_integer(x::RealFieldElem)
```

Return a pair where the first value is a boolean and the second is an ZZRingElem integer. The boolean indicates whether the interval $x$ contains a unique integer. If this is the case, the second return value is set to this unique integer.

source

Nemo.setunion – Method.

```
setunion(x::RealFieldElem, y::RealFieldElem)
```

Return an RealFieldElem containing the union of the intervals represented by $x$ and $y$.

source

**Examples**

```
julia> x = RR("-3 +/- 0.001")
[-3.00 +/- 1.01e-3]

julia> y = RR("2 +/- 0.5")
[2e+0 +/- 0.501]

julia> a = trim(x)
[-3.00 +/- 1.01e-3]

julia> b, c = unique_integer(x)
(true, -3)

julia> d = setunion(x, y)
[+/- 3.01]
```

### Constants

Nemo.const_pi – Method.

```
const_pi(r::RealField)
```

Return $\pi = 3.14159\ldots$ as an element of $r$.

source

Nemo.const_e – Method.

```
const_e(r::RealField)
```

Return $e = 2.71828\ldots$ as an element of $r$.

source

Nemo.const_log2 – Method.

```
const_log2(r::RealField)
```

Return $\log(2) = 0.69314\ldots$ as an element of $r$.

source

Nemo.const_log10 – Method.

```
const_log10(r::RealField)
```

Return $\log(10) = 2.302585\ldots$ as an element of $r$.

source

Nemo.const_euler – Method.

```
const_euler(r::RealField)
```

Return Euler's constant $\gamma = 0.577215\ldots$ as an element of $r$.

source

Nemo.const_catalan – Method.

```
const_catalan(r::RealField)
```

Return Catalan's constant $C = 0.915965\ldots$ as an element of $r$.

source

Nemo.const_khinchin – Method.

```
const_khinchin(r::RealField)
```

Return Khinchin's constant $K = 2.685452\ldots$ as an element of $r$.

source

Nemo.const_glaisher – Method.

```
const_glaisher(r::RealField)
```

Return Glaisher's constant $A = 1.282427\ldots$ as an element of $r$.

source

**Examples**

```
julia> a = const_pi(RR)
[3.141592653589793239 +/- 5.96e-19]

julia> b = const_e(RR)
[2.718281828459045235 +/- 4.29e-19]
```

```
julia> c = const_euler(RR)
[0.5772156649015328606 +/- 4.35e-20]

julia> d = const_glaisher(RR)
[1.282427129100622637 +/- 3.01e-19]
```

### Mathematical and special functions

Base.log – Method.

```
log(x::RealFieldElem; precision::Int = precision(Balls))
```

Return the natural logarithm of $x$ using the given precision.

source

Base.log – Method.

```
log(b::Int, x::RealFieldElem; precision::Int = precision(Balls))
```

Return the base $b$ logarithm of $x$ using the given precision, computed exactly when possible. The base $b$ must be greater than $1$.

source

Nemo.rsqrt – Method.

```
rsqrt(x::RealFieldElem)
```

Return the reciprocal of the square root of $x$, i.e. $1/\sqrt{x}$.

source

Nemo.sqrt1pm1 – Method.

```
sqrt1pm1(x::RealFieldElem)
```

Return $\sqrt{1+x} - 1$, evaluated accurately for small $x$.

source

Nemo.sqrtpos – Method.

```
sqrtpos(x::RealFieldElem)
```

Return the sqrt root of $x$, assuming that $x$ represents a non-negative number. Thus any negative number in the input interval is discarded.

source

Nemo.gamma – Method.

```
gamma(x::RealFieldElem)
```

Return the Gamma function evaluated at $x$.

source

Nemo.lgamma – Method.

```
lgamma(x::RealFieldElem)
```

Return the logarithm of the Gamma function evaluated at $x$.

source

Nemo.rgamma – Method.

```
rgamma(x::RealFieldElem)
```

Return the reciprocal of the Gamma function evaluated at $x$.

source

Nemo.digamma – Method.

```
digamma(x::RealFieldElem)
```

Return the logarithmic derivative of the gamma function evaluated at $x$, i.e. $\psi(x)$.

source

Nemo.gamma – Method.

```
gamma(s::RealFieldElem, x::RealFieldElem)
```

Return the upper incomplete gamma function $\Gamma(s, x)$.

source

Nemo.gamma_regularized – Method.

```
gamma_regularized(s::RealFieldElem, x::RealFieldElem)
```

Return the regularized upper incomplete gamma function $\Gamma(s, x)/\Gamma(s)$.

source

Nemo.gamma_lower – Method.

```
gamma_lower(s::RealFieldElem, x::RealFieldElem)
```

Return the lower incomplete gamma function $\gamma(s, x)/\Gamma(s)$.

source

Nemo.gamma_lower_regularized – Method.

```
gamma_lower_regularized(s::RealFieldElem, x::RealFieldElem)
```

Return the regularized lower incomplete gamma function $\gamma(s, x)/\Gamma(s)$.

source

Nemo.zeta – Method.

```
zeta(x::RealFieldElem)
```

Return the Riemann zeta function evaluated at $x$.

source

Nemo.atan2 – Method.

```
atan2(y::RealFieldElem, x::RealFieldElem)
```

Return $\operatorname{atan2}(y, x) = \arg(x + yi)$. Same as `atan(y, x)`.

source

Nemo.agm – Method.

```
agm(x::RealFieldElem, y::RealFieldElem)
```

Return the arithmetic-geometric mean of $x$ and $y$

source

Nemo.zeta – Method.

```
zeta(s::RealFieldElem, a::RealFieldElem)
```

Return the Hurwitz zeta function $\zeta(s, a)$.

source

AbstractAlgebra.root – Method.

```
root(x::RealFieldElem, n::Int)
```

Return the $n$-th root of $x$. We require $x \geq 0$.

source

Base.factorial – Method.

```
factorial(x::RealFieldElem)
```

Return the factorial of $x$.

source

Base.factorial – Method.

```
factorial(n::Int, r::RealField)
```

Return the factorial of $n$ in the given field.

source

Base.binomial – Method.

```
binomial(x::RealFieldElem, n::UInt)
```

Return the binomial coefficient $\binom{x}{n}$.

source

Base.binomial – Method.

```
binomial(n::UInt, k::UInt, r::RealField)
```

Return the binomial coefficient $\binom{n}{k}$ in the given field.

source

Nemo.fibonacci – Method.

```
fibonacci(n::ZZRingElem, r::RealField)
```

Return the $n$-th Fibonacci number in the given field.

source

Nemo.fibonacci – Method.

```
fibonacci(n::Int, r::RealField)
```

Return the $n$-th Fibonacci number in the given field.

source

Nemo.gamma – Method.

```
gamma(x::ZZRingElem, r::RealField)
```

Return the Gamma function evaluated at $x$ in the given field.

source

Nemo.gamma – Method.

```
gamma(x::QQFieldElem, r::RealField)
```

Return the Gamma function evaluated at $x$ in the given field.

source

Nemo.zeta – Method.

```
zeta(n::Int, r::RealField)
```

Return the Riemann zeta function $\zeta(n)$ as an element of the given field.

source

Nemo.bernoulli – Method.

```
bernoulli(n::Int, r::RealField)
```

Return the $n$-th Bernoulli number as an element of the given field.

source

AbstractAlgebra.Generic.rising_factorial – Method.

```
rising_factorial(x::RealFieldElem, n::Int)
```

Return the rising factorial $x(x+1)\ldots(x+n-1)$.

source

```
rising_factorial(x::RingElement, n::Integer)
```

Return the rising factorial of $x$, i.e. $x(x+1)(x+2)\cdots(x+n-1)$. If $n < 0$ we throw a `DomainError()`.

**Examples**

```
julia> R, x = ZZ[:x];

julia> rising_factorial(x, 1)
x

julia> rising_factorial(x, 2)
x^2 + x

julia> rising_factorial(4, 2)
20
```

source

`AbstractAlgebra.Generic.rising_factorial` – Method.

```
rising_factorial(x::QQFieldElem, n::Int, r::RealField)
```

Return the rising factorial $x(x+1)\ldots(x+n-1)$ as an element of the given field.

source

`AbstractAlgebra.Generic.rising_factorial2` – Method.

```
rising_factorial2(x::RealFieldElem, n::Int)
```

Return a tuple containing the rising factorial $x(x+1)\ldots(x+n-1)$ and its derivative.

source

```
rising_factorial2(x::RingElement, n::Integer)
```

Return a tuple containing the rising factorial $x(x+1)\cdots(x+n-1)$ and its derivative. If $n < 0$ we throw a `DomainError()`.

**Examples**

```
julia> R, x = ZZ[:x];

julia> rising_factorial2(x, 1)
(x, 1)

julia> rising_factorial2(x, 2)
(x^2 + x, 2*x + 1)

julia> rising_factorial2(4, 2)
(20, 9)
```

source

Nemo.polylog – Method.

```
polylog(s::Union{RealFieldElem,Int}, a::RealFieldElem)
```

Return the polylogarithm $\mathrm{Li}_s(a)$.

source

AbstractAlgebra.chebyshev_t – Method.

```
chebyshev_t(n::Int, x::RealFieldElem)
```

Return the value of the Chebyshev polynomial $T_n(x)$.

source

AbstractAlgebra.chebyshev_u – Method.

```
chebyshev_u(n::Int, x::RealFieldElem)
```

Return the value of the Chebyshev polynomial $U_n(x)$.

source

Nemo.chebyshev_t2 – Method.

```
chebyshev_t2(n::Int, x::RealFieldElem)
```

Return the tuple $(T_n(x), T_{n-1}(x))$.

source

Nemo.chebyshev_u2 – Method.

```
chebyshev_u2(n::Int, x::RealFieldElem)
```

Return the tuple $(U_n(x), U_{n-1}(x))$

source

Nemo.bell – Method.

```
bell(n::ZZRingElem, r::RealField)
```

Return the Bell number $B_n$ as an element of $r$.

source

Nemo.bell – Method.

```
bell(n::Int, r::RealField)
```

Return the Bell number $B_n$ as an element of $r$.

source

Nemo.numpart – Method.

```
numpart(n::ZZRingElem, r::RealField)
```

Return the number of partitions $p(n)$ as an element of $r$.

source

Nemo.numpart – Method.

```
numpart(n::Int, r::RealField)
```

Return the number of partitions $p(n)$ as an element of $r$.

source

Nemo.airy_ai – Method.

```
airy_ai(x::RealFieldElem)
```

Return the Airy function $\mathrm{Ai}(x)$.

source

Nemo.airy_ai_prime – Method.

```
airy_ai_prime(x::RealFieldElem)
```

Return the derivative of the Airy function $\mathrm{Ai}'(x)$.

source

Nemo.airy_bi – Method.

```
airy_bi(x::RealFieldElem)
```

Return the Airy function $\mathrm{Bi}(x)$.

source

Nemo.airy_bi_prime – Method.

```
airy_bi_prime(x::RealFieldElem)
```

Return the derivative of the Airy function $\mathrm{Bi}'(x)$.

source

**Examples**

```
julia> a = floor(exp(RR(1)))
2.0000000000000000000

julia> b = sinpi(QQ(5,6), RR)
0.50000000000000000000

julia> c = gamma(QQ(1,3), RR)
[2.678938534707747634 +/- 7.13e-19]

julia> d = bernoulli(1000, RR)
[-5.3187044694155220 36e+1769 +/- 6.61e+1750]

julia> f = polylog(3, RR(-10))
[-5.92106480375697 +/- 6.68e-15]
```

### Linear dependence

Nemo.lindep – Method.

```
lindep(A::Vector{RealFieldElem}, bits::Int)
```

Find a small linear combination of the entries of the array $A$ that is small (using LLL). The entries are first scaled by the given number of bits before truncating to integers for use in LLL. This function can be used to find linear dependence between a list of real numbers. The algorithm is heuristic only and returns an array of Nemo integers representing the linear combination.

**Examples**

```
julia> RR = real_field()
Real field

julia> a = RR(-0.331989029584509316202500694922316523 19)
[-0.33198902958450932088 +/- 4.15e-22]

julia> V = [RR(1), a, a^2, a^3, a^4, a^5]
6-element Vector{RealFieldElem}:
 1.0000000000000000000
 [-0.33198902958450932088 +/- 4.15e-22]
 [0.11021671576446420510 +/- 7.87e-21]
 [-0.03659074051063616184 +/- 4.17e-21]
 [0.012147724433904692427 +/- 4.99e-22]
 [-0.004032911246472051677 +/- 6.25e-22]
```

```
julia> W = lindep(V, 20)
6-element Vector{ZZRingElem}:
 1
 3
 0
 0
 0
 1
```

source

Nemo.simplest_rational_inside – Method.

```
simplest_rational_inside(x::RealFieldElem)
```

Return the simplest fraction inside the ball $x$. A canonical fraction $a_1/b_1$ is defined to be simpler than $a_2/b_2$ iff $b_1 < b_2$ or $b_1 = b_2$ and $a_1 < a_2$.

**Examples**

```
julia> RR = real_field()
Real field

julia> simplest_rational_inside(const_pi(RR))
8717442233//2774848045
```

source

### Random generation

Base.rand – Method.

```
rand(r::RealField; randtype::Symbol=:urandom)
```

Return a random element in given field.

The randtype default is :urandom which return an RealFieldElem contained in $[0, 1]$.

The rest of the methods return non-uniformly distributed values in order to exercise corner cases. The option :randtest will return a finite number, and :randtest_exact the same but with a zero radius. The option :randtest_precise return an RealFieldElem with a radius around $2^{-\mathrm{prec}}$ the magnitude of the midpoint, while :randtest_wide return a radius that might be big relative to its midpoint. The :randtest_special-option might return a midpoint and radius whose values are NaN or inf.

source

```
rand([rng=Random.default_rng(),] G::SymmetricGroup)
```

Return a random permutation from G.

source

**Examples**

```
a = rand(RR)
b = rand(RR; randtype = :null_exact)
c = rand(RR; randtype = :exact)
d = rand(RR; randtype = :special)
```

# Chapter 24

# Fixed precision real balls

Fixed precision real ball arithmetic is supplied by FLINT which provides a ball representation which tracks error bounds rigorously. Real numbers are represented in mid-rad interval form $[m \pm r] = [m - r, m + r]$.

The corresponding Nemo types retain the historical name Arb, from the separate library that is now part of FLINT. Arb stands for "*a*rbitrary-precision *r*eal *b*alls" (see Arbitrary precision real balls for arbitrary precision real balls in Nemo).

The Arb field is constructed using the `ArbField` constructor. This constructs the parent object for the Arb field.

The types of real balls in Nemo are given in the following table, along with the libraries that provide them and the associated types of the parent objects.

| Library | Field | Element type | Parent type |
|---|---|---|---|
| FLINT | $\mathbb{R}$ (balls) | `ArbFieldElem` | `ArbField` |

All the real field types belong to the `Field` abstract type and the types of elements in this field, i.e. balls in this case, belong to the `FieldElem` abstract type.

## 24.1 Real ball functionality

Real balls in Nemo provide all the field functionality described in AbstractAlgebra:

https://nemocas.github.io/AbstractAlgebra.jl/stable/field

Below, we document the additional functionality provided for real balls.

### Constructors

In order to construct real balls in Nemo, one must first construct the Arb field itself. This is accomplished with the following constructor.

```
ArbField(prec::Int)
```

Return the Arb field with precision in bits `prec` used for operations on interval midpoints. The precision used for interval radii is a fixed implementation-defined constant (30 bits).

Here is an example of creating an Arb field and using the resulting parent object to coerce values into the resulting field.

### Examples

```
julia> RR = ArbField(64)
Real Field with 64 bits of precision and error bounds

julia> a = RR("0.25")
0.25000000000000000000

julia> b = RR("0.1 +/- 0.001")
[0.10 +/- 1.01e-3]

julia> c = RR(0.5)
0.50000000000000000000

julia> d = RR(12)
12.000000000000000000
```

Note that whilst one can coerce double precision floating point values into an Arb field, unless those values can be represented exactly in double precision the resulting ball can't be any more precise than the double precision supplied.

If instead, values can be represented precisely using decimal arithmetic then one can supply them to the real field using a string. In this case, the real field will store them to the precision specified when creating the Arb field.

If the values can be stored precisely as a binary floating point number, the Arb field will store the values exactly. See the function `is_exact` below for more information.

## Real ball constructors

`Nemo.ball` – Method.

```
ball(x::ArbFieldElem, y::ArbFieldElem)
```

Constructs an Arb ball enclosing $x_m \pm (|x_r| + |y_m| + |y_r|)$, given the pair $(x, y) = (x_m \pm x_r, y_m \pm y_r)$.

source

### Examples

```
julia> RR = ArbField(64)
Real Field with 64 bits of precision and error bounds

julia> c = ball(RR(3), RR("0.0001"))
[3.000 +/- 1.01e-4]
```

## Conversions

```
julia> RR = ArbField(64)
Real Field with 64 bits of precision and error bounds
```

```
julia> convert(Float64, RR(1//3))
0.3333333333333333
```

### Basic manipulation

Nemo.is_nonzero – Method.

```
is_nonzero(x::ArbFieldElem)
```

Return `true` if $x$ is certainly not equal to zero, otherwise return `false`.

source

Base.isfinite – Method.

```
isfinite(x::ArbFieldElem)
```

Return `true` if $x$ is finite, i.e. having finite midpoint and radius, otherwise return `false`.

source

Nemo.is_exact – Method.

```
is_exact(x::ArbFieldElem)
```

Return `true` if $x$ is exact, i.e. has zero radius, otherwise return `false`.

source

Base.isinteger – Method.

```
isinteger(x::ArbFieldElem)
```

Return `true` if $x$ is an exact integer, otherwise return `false`.

source

AbstractAlgebra.is_positive – Method.

```
is_positive(x::ArbFieldElem)
```

Return `true` if $x$ is certainly positive, otherwise return `false`.

source

Nemo.is_nonnegative – Method.

```
is_nonnegative(x::ArbFieldElem)
```

Return `true` if $x$ is certainly non-negative, otherwise return `false`.

source

`AbstractAlgebra.is_negative` – Method.

```
is_negative(x::ArbFieldElem)
```

Return `true` if $x$ is certainly negative, otherwise return `false`.

source

`Nemo.is_nonpositive` – Method.

```
is_nonpositive(x::ArbFieldElem)
```

Return `true` if $x$ is certainly nonpositive, otherwise return `false`.

source

`Nemo.midpoint` – Method.

```
midpoint(x::ArbFieldElem)
```

Return the midpoint of the ball $x$ as an Arb ball.

source

`Nemo.radius` – Method.

```
radius(x::ArbFieldElem)
```

Return the radius of the ball $x$ as an Arb ball.

source

`Nemo.accuracy_bits` – Method.

```
accuracy_bits(x::ArbFieldElem)
```

Return the relative accuracy of $x$ measured in bits, capped between `typemax(Int)` and `-typemax(Int)`.

source

**Examples**

```
julia> RR = ArbField(64)
Real Field with 64 bits of precision and error bounds

julia> a = RR("1.2 +/- 0.001")
[1.20 +/- 1.01e-3]

julia> b = RR(3)
3.0000000000000000000

julia> is_positive(a)
true

julia> isfinite(b)
true

julia> isinteger(b)
true

julia> is_negative(a)
false

julia> c = radius(a)
[0.0010000000038417056203 +/- 1.12e-23]

julia> d = midpoint(b)
3.0000000000000000000

julia> f = accuracy_bits(a)
9
```

### Printing

Printing real balls can at first sight be confusing. Lets look at the following example:

```
julia> RR = ArbField(64)
Real Field with 64 bits of precision and error bounds

julia> a = RR(1)
1.0000000000000000000

julia> b = RR(2)
2.0000000000000000000

julia> c = RR(12)
12.000000000000000000

julia> x = ball(a, b)
[+/- 3.01]

julia> y = ball(c, b)
[1e+1 +/- 4.01]

julia> mid = midpoint(x)
1.0000000000000000000
```

```
julia> rad = radius(x)
[2.0000000037252902985 +/- 3.81e-20]

julia> print(x, "\n", y, "\n", mid, "\n", rad)
[+/- 3.01]
[1e+1 +/- 4.01]
1.0000000000000000000
[2.0000000037252902985 +/- 3.81e-20]
```

The first reason that x is not printed as `[1 +/- 2]` is that the midpoint does not have a greater exponent than the radius in its scientific notation. For similar reasons y is not printed as `[12 +/- 2]`.

The second reason is that we get an additional error term after our addition. As we see, `radius(x)` is not equal to $2$, which when printed rounds it up to a reasonable decimal place. This is because real balls keep track of rounding errors of basic arithmetic.

### Containment

It is often necessary to determine whether a given exact value or ball is contained in a given real ball or whether two balls overlap. The following functions are provided for this purpose.

`Nemo.overlaps` – Method.

```
overlaps(x::ArbFieldElem, y::ArbFieldElem)
```

Returns `true` if any part of the ball $x$ overlaps any part of the ball $y$, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::ArbFieldElem, y::ArbFieldElem)
```

Returns `true` if the ball $x$ contains the ball $y$, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::ArbFieldElem, y::Integer)
```

Returns `true` if the ball $x$ contains the given integer value, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::ArbFieldElem, y::ZZRingElem)
```

Returns `true` if the ball $x$ contains the given integer value, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::ArbFieldElem, y::QQFieldElem)
```

Returns `true` if the ball $x$ contains the given rational value, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::ArbFieldElem, y::Rational{T}) where {T <: Integer}
```

Returns `true` if the ball $x$ contains the given rational value, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::ArbFieldElem, y::BigFloat)
```

Returns `true` if the ball $x$ contains the given floating point value, otherwise return `false`.

source

The following functions are also provided for determining if a ball intersects a certain part of the real number line.

`Nemo.contains_zero` – Method.

```
contains_zero(x::ArbFieldElem)
```

Returns `true` if the ball $x$ contains zero, otherwise return `false`.

source

`Nemo.contains_negative` – Method.

```
contains_negative(x::ArbFieldElem)
```

Returns `true` if the ball $x$ contains any negative value, otherwise return `false`.

source

Nemo.contains_positive – Method.

```
contains_positive(x::ArbFieldElem)
```

Returns true if the ball $x$ contains any positive value, otherwise return false.

source

Nemo.contains_nonnegative – Method.

```
contains_nonnegative(x::ArbFieldElem)
```

Returns true if the ball $x$ contains any non-negative value, otherwise return false.

source

Nemo.contains_nonpositive – Method.

```
contains_nonpositive(x::ArbFieldElem)
```

Returns true if the ball $x$ contains any nonpositive value, otherwise return false.

source

**Examples**

```
julia> RR = ArbField(64)
Real Field with 64 bits of precision and error bounds

julia> x = RR("1 +/- 0.001")
[1.00 +/- 1.01e-3]

julia> y = RR("3")
3.0000000000000000000

julia> overlaps(x, y)
false

julia> contains(x, y)
false

julia> contains(y, 3)
true

julia> contains(x, ZZ(1)//2)
false

julia> contains_zero(x)
false

julia> contains_positive(y)
true
```

### Comparison

Nemo provides a full range of comparison operations for Arb balls. Note that a ball is considered less than another ball if every value in the first ball is less than every value in the second ball, etc.

In addition to the standard comparison operators, we introduce an exact equality. This is distinct from arithmetic equality implemented by ==, which merely compares up to the minimum of the precisions of its operands.

`Base.isequal` – Method.

```
isequal(x::ArbFieldElem, y::ArbFieldElem)
```

Return `true` if the balls $x$ and $y$ are precisely equal, i.e. have the same midpoints and radii.

source

We also provide a full range of ad hoc comparison operators. These are implemented directly in Julia, but we document them as though `isless` and == were provided.

| Function |
|---|
| ==(x::ArbFieldElem, y::Integer) |
| ==(x::Integer, y::ArbFieldElem) |
| ==(x::ArbFieldElem, y::ZZRingElem) |
| ==(x::ZZRingElem, y::ArbFieldElem) |
| ==(x::ArbFieldElem, y::Float64) |
| ==(x::Float64, y::ArbFieldElem) |
| isless(x::ArbFieldElem, y::Integer) |
| isless(x::Integer, y::ArbFieldElem) |
| isless(x::ArbFieldElem, y::ZZRingElem) |
| isless(x::ZZRingElem, y::ArbFieldElem) |
| isless(x::ArbFieldElem, y::Float64) |
| isless(x::Float64, y::ArbFieldElem) |
| isless(x::ArbFieldElem, y::BigFloat) |
| isless(x::BigFloat, y::ArbFieldElem) |
| isless(x::ArbFieldElem, y::QQFieldElem) |
| isless(x::QQFieldElem, y::ArbFieldElem) |

#### Examples

```
julia> RR = ArbField(64)
Real Field with 64 bits of precision and error bounds

julia> x = RR("1 +/- 0.001")
[1.00 +/- 1.01e-3]

julia> y = RR("3")
3.0000000000000000000

julia> z = RR("4")
4.0000000000000000000

julia> isequal(x, deepcopy(x))
true
```

```
julia> x == 3
false

julia> ZZ(3) < z
true

julia> x != 1.23
true

julia> 3 == y
true
```

### Absolute value

#### Examples

```
julia> RR = ArbField(64)
Real Field with 64 bits of precision and error bounds

julia> x = RR("-1 +/- 0.001")
[-1.00 +/- 1.01e-3]

julia> a = abs(x)
[1.00 +/- 1.01e-3]
```

### Shifting

#### Examples

```
julia> RR = ArbField(64)
Real Field with 64 bits of precision and error bounds

julia> x = RR("-3 +/- 0.001")
[-3.00 +/- 1.01e-3]

julia> a = ldexp(x, 23)
[-2.52e+7 +/- 4.26e+4]

julia> b = ldexp(x, -ZZ(15))
[-9.16e-5 +/- 7.78e-8]
```

### Miscellaneous operations

`Nemo.add_error!` – Method.

```
add_error!(x::ArbFieldElem, y::ArbFieldElem)
```

Adds the absolute values of the midpoint and radius of $y$ to the radius of $x$.

source

Nemo.trim – Method.

```
trim(x::ArbFieldElem)
```

Return an ArbFieldElem interval containing $x$ but which may be more economical, by rounding off insignificant bits from the midpoint.

source

Nemo.unique_integer – Method.

```
unique_integer(x::ArbFieldElem)
```

Return a pair where the first value is a boolean and the second is an ZZRingElem integer. The boolean indicates whether the interval $x$ contains a unique integer. If this is the case, the second return value is set to this unique integer.

source

Nemo.setunion – Method.

```
setunion(x::ArbFieldElem, y::ArbFieldElem)
```

Return an ArbFieldElem containing the union of the intervals represented by $x$ and $y$.

source

**Examples**

```
julia> RR = ArbField(64)
Real Field with 64 bits of precision and error bounds

julia> x = RR("-3 +/- 0.001")
[-3.00 +/- 1.01e-3]

julia> y = RR("2 +/- 0.5")
[2e+0 +/- 0.501]

julia> a = trim(x)
[-3.00 +/- 1.01e-3]

julia> b, c = unique_integer(x)
(true, -3)

julia> d = setunion(x, y)
[+/- 3.01]
```

### Constants

Nemo.const_pi – Method.

```
const_pi(r::ArbField)
```

Return $\pi = 3.14159\ldots$ as an element of $r$.

source

Nemo.const_e – Method.

```
const_e(r::ArbField)
```

Return $e = 2.71828\ldots$ as an element of $r$.

source

Nemo.const_log2 – Method.

```
const_log2(r::ArbField)
```

Return $\log(2) = 0.69314\ldots$ as an element of $r$.

source

Nemo.const_log10 – Method.

```
const_log10(r::ArbField)
```

Return $\log(10) = 2.302585\ldots$ as an element of $r$.

source

Nemo.const_euler – Method.

```
const_euler(r::ArbField)
```

Return Euler's constant $\gamma = 0.577215\ldots$ as an element of $r$.

source

Nemo.const_catalan – Method.

```
const_catalan(r::ArbField)
```

Return Catalan's constant $C = 0.915965\ldots$ as an element of $r$.

source

Nemo.const_khinchin – Method.

```
const_khinchin(r::ArbField)
```

Return Khinchin's constant $K = 2.685452\ldots$ as an element of $r$.

source

Nemo.const_glaisher – Method.

```
const_glaisher(r::ArbField)
```

Return Glaisher's constant $A = 1.282427\ldots$ as an element of $r$.

source

**Examples**

```
julia> RR = ArbField(200)
Real Field with 200 bits of precision and error bounds

julia> a = const_pi(RR)
[3.14159265358979323846264338327950288419716939937510582097494 +/- 5.73e-60]

julia> b = const_e(RR)
[2.71828182845904523536028747135266249775724709369995957496697 +/- 7.06e-60]

julia> c = const_euler(RR)
[0.57721566490153286060651209008240243104215933593992359880576 7 +/- 5.37e-61]

julia> d = const_glaisher(RR)
[1.28242712910062263687534256886979172776768892732500119206374 +/- 2.18e-60]
```

### Mathematical and special functions

Base.log – Method.

```
log(b::Int, x::ArbFieldElem)
```

Return the base $b$ logarithm of $x$, computed exactly when possible. The base $b$ must be greater than $1$.

source

Nemo.rsqrt – Method.

```
rsqrt(x::ArbFieldElem)
```

Return the reciprocal of the square root of $x$, i.e. $1/\sqrt{x}$.

source

Nemo.sqrt1pm1 – Method.

```
sqrt1pm1(x::ArbFieldElem)
```

Return $\sqrt{1+x}-1$, evaluated accurately for small $x$.

source

Nemo.sqrtpos – Method.

```
sqrtpos(x::ArbFieldElem)
```

Return the sqrt root of $x$, assuming that $x$ represents a non-negative number. Thus any negative number in the input interval is discarded.

source

Nemo.gamma – Method.

```
gamma(x::ArbFieldElem)
```

Return the Gamma function evaluated at $x$.

source

Nemo.lgamma – Method.

```
lgamma(x::ArbFieldElem)
```

Return the logarithm of the Gamma function evaluated at $x$.

source

Nemo.rgamma – Method.

```
rgamma(x::ArbFieldElem)
```

Return the reciprocal of the Gamma function evaluated at $x$.

source

Nemo.digamma – Method.

```
digamma(x::ArbFieldElem)
```

Return the logarithmic derivative of the gamma function evaluated at $x$, i.e. $\psi(x)$.

source

Nemo.gamma – Method.

```
gamma(s::ArbFieldElem, x::ArbFieldElem)
```

Return the upper incomplete gamma function $\Gamma(s, x)$.

source

Nemo.gamma_regularized – Method.

```
gamma_regularized(s::ArbFieldElem, x::ArbFieldElem)
```

Return the regularized upper incomplete gamma function $\Gamma(s, x)/\Gamma(s)$.

source

Nemo.gamma_lower – Method.

```
gamma_lower(s::ArbFieldElem, x::ArbFieldElem)
```

Return the lower incomplete gamma function $\gamma(s, x)/\Gamma(s)$.

source

Nemo.gamma_lower_regularized – Method.

```
gamma_lower_regularized(s::ArbFieldElem, x::ArbFieldElem)
```

Return the regularized lower incomplete gamma function $\gamma(s, x)/\Gamma(s)$.

source

Nemo.zeta – Method.

```
zeta(x::ArbFieldElem)
```

Return the Riemann zeta function evaluated at $x$.

source

Nemo.atan2 – Method.

```
atan2(y::ArbFieldElem, x::ArbFieldElem)
```

Return $\operatorname{atan2}(y, x) = \arg(x + yi)$. Same as `atan(y, x)`.

source

Nemo.agm – Method.

```
agm(x::ArbFieldElem, y::ArbFieldElem)
```

Return the arithmetic-geometric mean of $x$ and $y$

source

`Nemo.zeta` – Method.

```
zeta(s::ArbFieldElem, a::ArbFieldElem)
```

Return the Hurwitz zeta function $\zeta(s, a)$.

source

`AbstractAlgebra.root` – Method.

```
root(x::ArbFieldElem, n::Int)
```

Return the $n$-th root of $x$. We require $x \geq 0$.

source

`Base.factorial` – Method.

```
factorial(x::ArbFieldElem)
```

Return the factorial of $x$.

source

`Base.factorial` – Method.

```
factorial(n::Int, r::ArbField)
```

Return the factorial of $n$ in the given Arb field.

source

`Base.binomial` – Method.

```
binomial(x::ArbFieldElem, n::UInt)
```

Return the binomial coefficient $\binom{x}{n}$.

source

`Base.binomial` – Method.

```
binomial(n::UInt, k::UInt, r::ArbField)
```

Return the binomial coefficient $\binom{n}{k}$ in the given Arb field.

source

Nemo.fibonacci – Method.

```
fibonacci(n::ZZRingElem, r::ArbField)
```

Return the $n$-th Fibonacci number in the given Arb field.

source

Nemo.fibonacci – Method.

```
fibonacci(n::Int, r::ArbField)
```

Return the $n$-th Fibonacci number in the given Arb field.

source

Nemo.gamma – Method.

```
gamma(x::ZZRingElem, r::ArbField)
```

Return the Gamma function evaluated at $x$ in the given Arb field.

source

Nemo.gamma – Method.

```
gamma(x::QQFieldElem, r::ArbField)
```

Return the Gamma function evaluated at $x$ in the given Arb field.

source

Nemo.zeta – Method.

```
zeta(n::Int, r::ArbField)
```

Return the Riemann zeta function $\zeta(n)$ as an element of the given Arb field.

source

Nemo.bernoulli – Method.

```
bernoulli(n::Int, r::ArbField)
```

Return the $n$-th Bernoulli number as an element of the given Arb field.

source

AbstractAlgebra.Generic.rising_factorial – Method.

```
rising_factorial(x::ArbFieldElem, n::Int)
```

Return the rising factorial $x(x+1)\ldots(x+n-1)$ as an Arb.

source

AbstractAlgebra.Generic.rising_factorial – Method.

```
rising_factorial(x::QQFieldElem, n::Int, r::ArbField)
```

Return the rising factorial $x(x+1)\ldots(x+n-1)$ as an element of the given Arb field.

source

AbstractAlgebra.Generic.rising_factorial2 – Method.

```
rising_factorial2(x::ArbFieldElem, n::Int)
```

Return a tuple containing the rising factorial $x(x+1)\ldots(x+n-1)$ and its derivative.

source

Nemo.polylog – Method.

```
polylog(s::Union{ArbFieldElem,Int}, a::ArbFieldElem)
```

Return the polylogarithm $\mathrm{Li}_s(a)$.

source

AbstractAlgebra.chebyshev_t – Method.

```
chebyshev_t(n::Int, x::ArbFieldElem)
```

Return the value of the Chebyshev polynomial $T_n(x)$.

source

AbstractAlgebra.chebyshev_u – Method.

```
chebyshev_u(n::Int, x::ArbFieldElem)
```

Return the value of the Chebyshev polynomial $U_n(x)$.

source

Nemo.chebyshev_t2 – Method.

```
chebyshev_t2(n::Int, x::ArbFieldElem)
```

Return the tuple $(T_n(x), T_{n-1}(x))$.

source

Nemo.chebyshev_u2 – Method.

```
chebyshev_u2(n::Int, x::ArbFieldElem)
```

Return the tuple $(U_n(x), U_{n-1}(x))$

source

Nemo.bell – Method.

```
bell(n::ZZRingElem, r::ArbField)
```

Return the Bell number $B_n$ as an element of $r$.

source

Nemo.bell – Method.

```
bell(n::Int, r::ArbField)
```

Return the Bell number $B_n$ as an element of $r$.

source

Nemo.numpart – Method.

```
numpart(n::ZZRingElem, r::ArbField)
```

Return the number of partitions $p(n)$ as an element of $r$.

source

Nemo.numpart – Method.

```
numpart(n::Int, r::ArbField)
```

Return the number of partitions $p(n)$ as an element of $r$.

source

Nemo.airy_ai – Method.

```
airy_ai(x::ArbFieldElem)
```

Return the Airy function $\mathrm{Ai}(x)$.

source

Nemo.airy_ai_prime – Method.

```
airy_ai_prime(x::ArbFieldElem)
```

Return the derivative of the Airy function $\mathrm{Ai}'(x)$.

source

Nemo.airy_bi – Method.

```
airy_bi(x::ArbFieldElem)
```

Return the Airy function $\mathrm{Bi}(x)$.

source

Nemo.airy_bi_prime – Method.

```
airy_bi_prime(x::ArbFieldElem)
```

Return the derivative of the Airy function $\mathrm{Bi}'(x)$.

source

**Examples**

```
julia> RR = ArbField(64)
Real Field with 64 bits of precision and error bounds

julia> a = floor(exp(RR(1)))
2.0000000000000000000

julia> b = sinpi(QQ(5,6), RR)
0.50000000000000000000
```

```
julia> c = gamma(QQ(1,3), ArbField(256))
[2.67893853470774763365569294097467764412868937795730110095042832759041761016 77 +/- 6.71e-77]

julia> d = bernoulli(1000, ArbField(53))
[-5.318704469415522e+1769 +/- 8.20e+1753]

julia> f = polylog(3, RR(-10))
[-5.92106480375697 +/- 6.68e-15]
```

### Linear dependence

Nemo.lindep – Method.

```
lindep(A::Vector{ArbFieldElem}, bits::Int)
```

Find a small linear combination of the entries of the array $A$ that is small (using LLL). The entries are first scaled by the given number of bits before truncating to integers for use in LLL. This function can be used to find linear dependence between a list of real numbers. The algorithm is heuristic only and returns an array of Nemo integers representing the linear combination.

**Examples**

```
julia> RR = ArbField(64)
Real Field with 64 bits of precision and error bounds

julia> a = RR(-0.331989029584509316202500694922316523 19)
[-0.33198902958450932088 +/- 4.15e-22]

julia> V = [RR(1), a, a^2, a^3, a^4, a^5]
6-element Vector{ArbFieldElem}:
 1.0000000000000000000
 [-0.33198902958450932088 +/- 4.15e-22]
 [0.11021671576446420510 +/- 7.87e-21]
 [-0.03659074051063616184 +/- 4.17e-21]
 [0.012147724433904692427 +/- 4.99e-22]
 [-0.004032911246472051677 +/- 6.25e-22]

julia> W = lindep(V, 20)
6-element Vector{ZZRingElem}:
 1
 3
 0
 0
 0
 1
```

source

**Examples**

```
julia> RR = ArbField(128)
Real Field with 128 bits of precision and error bounds

julia> a = RR(-0.331989029584509316202500694922316523l9) # real root of x^5 + 3x + 1
[-0.331989029584509320880414406929048709571 +/- 3.62e-40]

julia> V = [RR(1), a, a^2, a^3, a^4, a^5]
6-element Vector{ArbFieldElem}:
 1.000000000000000000000000000000000000000
 [-0.331989029584509320880414406929048709571 +/- 3.62e-40]
 [0.110216715764464205102727554344054759368 +/- 3.32e-40]
 [-0.036590740510636161838468003150601571018 4 +/- 8.30e-41]
 [0.012147724433904692427423258042916492052 4 +/- 2.83e-41]
 [-0.004032911246472051676627948728260318189 05 +/- 7.87e-42]

julia> W = lindep(V, 20)
6-element Vector{ZZRingElem}:
 1
 3
 0
 0
 0
 1
```

Nemo.simplest_rational_inside – Method.

```
simplest_rational_inside(x::ArbFieldElem)
```

Return the simplest fraction inside the ball $x$. A canonical fraction $a_1/b_1$ is defined to be simpler than $a_2/b_2$ iff $b_1 < b_2$ or $b_1 = b_2$ and $a_1 < a_2$.

**Examples**

```
julia> RR = ArbField(64)
Real Field with 64 bits of precision and error bounds

julia> simplest_rational_inside(const_pi(RR))
8717442233//2774848045
```

source

### Random generation

Base.rand – Method.

```
rand(r::ArbField; randtype::Symbol=:urandom)
```

Return a random element in given Arb field.

The randtype default is :urandom which return an ArbFieldElem contained in $[0, 1]$.

The rest of the methods return non-uniformly distributed values in order to exercise corner cases. The option `:randtest` will return a finite number, and `:randtest_exact` the same but with a zero radius. The option `:randtest_precise` return an `ArbFieldElem` with a radius around $2^{-\mathrm{prec}}$ the magnitude of the midpoint, while `:randtest_wide` return a radius that might be big relative to its midpoint. The `:randtest_special`-option might return a midpoint and radius whose values are `NaN` or `inf`.

source

**Examples**

```
RR = ArbField(100)

a = rand(RR)
b = rand(RR; randtype = :null_exact)
c = rand(RR; randtype = :exact)
d = rand(RR; randtype = :special)
```

# Chapter 25

# Fixed precision complex balls

Fixed precision complex ball arithmetic is supplied by FLINT which provides a ball representation which tracks error bounds rigorously. Complex numbers are represented in rectangular form $a+bi$ where $a, b$ are `ArbFieldElem` balls.

The name AcbField is inherited from FLINT, where it stands for "*a*rbitrary-precision *c*omplex *b*alls".

The Acb field is constructed using the `AcbField` constructor. This constructs the parent object for the Acb field.

The types of complex boxes in Nemo are given in the following table, along with the libraries that provide them and the associated types of the parent objects.

| Library | Field | Element type | Parent type |
|---|---|---|---|
| FLINT | $\mathbb{C}$ (boxes) | `AcbFieldElem` | `AcbField` |

All the complex field types belong to the `Field` abstract type and the types of elements in this field, i.e. complex boxes in this case, belong to the `FieldElem` abstract type.

## 25.1 Complex ball functionality

The complex balls in Nemo provide all the field functionality defined by AbstractAlgebra:.

https://nemocas.github.io/AbstractAlgebra.jl/stable/field

Below, we document the additional functionality provided for complex balls.

### Complex field constructors

In order to construct complex boxes in Nemo, one must first construct the Acb field itself. This is accomplished with the following constructor.

```
AcbField(prec::Int)
```

Return the Acb field with precision in bits `prec` used for operations on interval midpoints. The precision used for interval radii is a fixed implementation-defined constant (30 bits).

Here is an example of creating an Acb field and using the resulting parent object to coerce values into the resulting field.

**Examples**

```
julia> CC = AcbField(64)
Complex Field with 64 bits of precision and error bounds

julia> a = CC("0.25")
0.25000000000000000000

julia> b = CC("0.1")
[0.10000000000000000000 +/- 2.72e-21]

julia> c = CC(0.5)
0.50000000000000000000

julia> d = CC(12)
12.000000000000000000
```

Note that whilst one can coerce double precision floating point values into an Acb field, unless those values can be represented exactly in double precision the resulting ball can't be any more precise than the double precision supplied.

If instead, values can be represented precisely using decimal arithmetic then one can supply them to the Acb field using a string. In this case, the Acb field will store them to the precision specified when creating the Acb field.

If the values can be stored precisely as a binary floating point number, the Acb field will store the values exactly. See the function `is_exact` below for more information.

### Constructors

`Nemo.onei` – Method.

```
onei(r::AcbField)
```

Return exact one times $i$ in the given Arb complex field.

source

### Examples

```
julia> CC = AcbField(64)
Complex Field with 64 bits of precision and error bounds

julia> c = onei(CC)
1.0000000000000000000*im
```

## 25.2 Basic functionality

The following basic functionality is provided by the Acb field implementation in Nemo, to support construction of generic rings over Acb fields.

```
parent_type(::Type{AcbFieldElem})
```

Gives the type of the parent object of an Acb field element.

```
elem_type(R::AcbField)
```

Given the parent object for an Acb field, return the type of elements of the field.

```
mul!(c::AcbFieldElem, a::AcbFieldElem, b::AcbFieldElem)
```

Multiply $a$ by $b$ and set the existing Acb field element $c$ to the result. This function is provided for performance reasons as it saves allocating a new object for the result and eliminates associated garbage collection.

```
deepcopy(a::AcbFieldElem)
```

Return a copy of the Acb field element $a$, recursively copying the internal data. Acb field elements are mutable in Nemo so a shallow copy is not sufficient.

Given the parent object R for an Acb field, the following coercion functions are provided to coerce various elements into the Acb field.

```
R()
```

Coerce zero into the Acb field.

```
R(n::Integer)
R(f::ZZRingElem)
R(q::QQFieldElem)
```

Coerce an integer or rational value into the Acb field.

```
R(f::Float64)
R(f::BigFloat)
```

Coerce the given floating point number into the Acb field.

```
R(f::AbstractString)
R(f::AbstractString, g::AbstractString)
```

Coerce the decimal number, given as a string, into the Acb field. In each case $f$ is the real part and $g$ is the imaginary part.

```
R(f::ArbFieldElem)
```

Coerce the given Arb real ball into the Acb field.

```
R(f::AcbFieldElem)
```

Take an Acb field element that is already in an Acb field and simply return it. A copy of the original is not made.

Here are some examples of coercing elements into the Acb field.

```
julia> RR = ArbField(64)
Real Field with 64 bits of precision and error bounds

julia> CC = AcbField(64)
Complex Field with 64 bits of precision and error bounds

julia> a = CC(3)
3.0000000000000000000

julia> b = CC(QQ(2,3))
[0.66666666666666666 +/- 8.48e-20]

julia> c = CC("3 +/- 0.0001")
[3.000 +/- 1.01e-4]

julia> d = CC("-1.24e+12345")
[-1.2400000000000000000e+12345 +/- 4.16e+12325]

julia> f = CC("nan +/- inf")
nan

julia> g = CC(RR(3))
3.0000000000000000000
```

### Basic manipulation

`Base.isfinite` – Method.

```
isfinite(x::AcbFieldElem)
```

Return `true` if $x$ is finite, i.e. its real and imaginary parts have finite midpoint and radius, otherwise return `false`.

source

`Nemo.is_exact` – Method.

```
is_exact(x::AcbFieldElem)
```

Return `true` if $x$ is exact, i.e. has its real and imaginary parts have zero radius, otherwise return `false`.

source

`Base.isinteger` – Method.

```
isinteger(x::AcbFieldElem)
```

Return `true` if $x$ is an exact integer, otherwise return `false`.

source

Nemo.accuracy_bits – Method.

```
accuracy_bits(x::AcbFieldElem)
```

Return the relative accuracy of $x$ measured in bits, capped between `typemax(Int)` and `-typemax(Int)`.

source

**Examples**

```
julia> CC = AcbField(64)
Complex Field with 64 bits of precision and error bounds

julia> a = CC("1.2 +/- 0.001")
[1.20 +/- 1.01e-3]

julia> b = CC(3)
3.0000000000000000000

julia> isreal(a)
true

julia> isfinite(b)
true

julia> isinteger(b)
true

julia> c = real(a)
[1.20 +/- 1.01e-3]

julia> d = imag(b)
0

julia> f = accuracy_bits(a)
9
```

### Containment

It is often necessary to determine whether a given exact value or box is contained in a given complex box or whether two boxes overlap. The following functions are provided for this purpose.

Nemo.overlaps – Method.

```
overlaps(x::AcbFieldElem, y::AcbFieldElem)
```

Returns `true` if any part of the box $x$ overlaps any part of the box $y$, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::AcbFieldElem, y::AcbFieldElem)
```

Returns `true` if the box $x$ contains the box $y$, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::AcbFieldElem, y::Integer)
```

Returns `true` if the box $x$ contains the given integer value, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::AcbFieldElem, y::ZZRingElem)
```

Returns `true` if the box $x$ contains the given integer value, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::AcbFieldElem, y::QQFieldElem)
```

Returns `true` if the box $x$ contains the given rational value, otherwise return `false`.

source

The following functions are also provided for determining if a box intersects a certain part of the complex number plane.

`Nemo.contains_zero` – Method.

```
contains_zero(x::AcbFieldElem)
```

Returns `true` if the box $x$ contains zero, otherwise return `false`.

source

**Examples**

```
julia> CC = AcbField(64)
Complex Field with 64 bits of precision and error bounds

julia> x = CC("1 +/- 0.001")
[1.00 +/- 1.01e-3]

julia> y = CC("3")
3.0000000000000000000

julia> overlaps(x, y)
false

julia> contains(x, y)
false

julia> contains(y, 3)
true

julia> contains(x, ZZ(1)//2)
false

julia> contains_zero(x)
false
```

### Comparison

Nemo provides a full range of comparison operations for complex boxes.

In addition to the standard comparisons, we introduce an exact equality. This is distinct from arithmetic equality implemented by ==, which merely compares up to the minimum of the precisions of its operands.

`Base.isequal` – Method.

```
isequal(x::AcbFieldElem, y::AcbFieldElem)
```

Return `true` if the boxes $x$ and $y$ are precisely equal, i.e. their real and imaginary parts have the same midpoints and radii.

source

A full range of ad hoc comparison operators is provided. These are implemented directly in Julia, but we document them as though only == were provided.

**Examples**

```
julia> CC = AcbField(64)
Complex Field with 64 bits of precision and error bounds

julia> x = CC("1 +/- 0.001")
[1.00 +/- 1.01e-3]
```

| Function |
|---|
| ==(x::AcbFieldElem, y::Integer) |
| ==(x::Integer, y::AcbFieldElem) |
| ==(x::AcbFieldElem, y::ZZRingElem) |
| ==(x::ZZRingElem, y::AcbFieldElem) |
| ==(x::ArbFieldElem, y::ZZRingElem) |
| ==(x::ZZRingElem, y::ArbFieldElem) |
| ==(x::AcbFieldElem, y::Float64) |
| ==(x::Float64, y::AcbFieldElem) |

```
julia> y = CC("3")
3.000000000000000000

julia> z = CC("4")
4.000000000000000000

julia> isequal(x, deepcopy(x))
true

julia> x == 3
false

julia> ZZ(3) == z
false

julia> x != 1.23
true
```

### Absolute value

#### Examples

```
julia> CC = AcbField(64)
Complex Field with 64 bits of precision and error bounds

julia> x = CC("-1 +/- 0.001")
[-1.00 +/- 1.01e-3]

julia> a = abs(x)
[1.00 +/- 1.01e-3]
```

### Shifting

#### Examples

```
julia> CC = AcbField(64)
Complex Field with 64 bits of precision and error bounds

julia> x = CC("-3 +/- 0.001")
```

```
[-3.00 +/- 1.01e-3]

julia> a = ldexp(x, 23)
[-2.52e+7 +/- 4.26e+4]

julia> b = ldexp(x, -ZZ(15))
[-9.16e-5 +/- 7.78e-8]
```

### Miscellaneous operations

Nemo.trim – Method.

```
trim(x::AcbFieldElem)
```

Return an AcbFieldElem box containing $x$ but which may be more economical, by rounding off insignificant bits from midpoints.

source

Nemo.unique_integer – Method.

```
unique_integer(x::AcbFieldElem)
```

Return a pair where the first value is a boolean and the second is an ZZRingElem integer. The boolean indicates whether the box $x$ contains a unique integer. If this is the case, the second return value is set to this unique integer.

source

### Examples

```
julia> CC = AcbField(64)
Complex Field with 64 bits of precision and error bounds

julia> x = CC("-3 +/- 0.001", "0.1")
[-3.00 +/- 1.01e-3] + [0.10000000000000000000 +/- 2.72e-21]*im

julia> a = trim(x)
[-3.00 +/- 1.01e-3] + [0.10000000000000000000 +/- 2.72e-21]*im

julia> b, c = unique_integer(x)
(false, 0)

julia> d = conj(x)
[-3.00 +/- 1.01e-3] + [-0.10000000000000000000 +/- 2.72e-21]*im

julia> f = angle(x)
[3.1083 +/- 3.95e-5]
```

### Constants

Nemo.const_pi – Method.

```
const_pi(r::AcbField)
```

Return $\pi = 3.14159\ldots$ as an element of $r$.

source

#### Examples

```
julia> CC = AcbField(200)
Complex Field with 200 bits of precision and error bounds

julia> a = const_pi(CC)
[3.14159265358979323846264338327950288419716939937510582097494 +/- 5.73e-60]
```

### Mathematical and special functions

Nemo.rsqrt – Method.

```
rsqrt(x::AcbFieldElem)
```

Return the reciprocal of the square root of $x$, i.e. $1/\sqrt{x}$.

source

Base.cispi – Method.

```
cispi(x::AcbFieldElem)
```

Return the exponential of $\pi i x$.

source

Nemo.root_of_unity – Method.

```
root_of_unity(C::AcbField, k::Int)
```

Return $\exp(2\pi i/k)$.

source

Nemo.log_sinpi – Method.

```
log_sinpi(x::AcbFieldElem)
```

Return $\log \sin(\pi x)$, constructed without branch cuts off the real line.

source

Nemo.gamma – Method.

```
gamma(x::AcbFieldElem)
```

Return the Gamma function evaluated at $x$.

source

Nemo.lgamma – Method.

```
lgamma(x::AcbFieldElem)
```

Return the logarithm of the Gamma function evaluated at $x$.

source

Nemo.rgamma – Method.

```
rgamma(x::AcbFieldElem)
```

Return the reciprocal of the Gamma function evaluated at $x$.

source

Nemo.digamma – Method.

```
digamma(x::AcbFieldElem)
```

Return the logarithmic derivative of the gamma function evaluated at $x$, i.e. $\psi(x)$.

source

Nemo.zeta – Method.

```
zeta(x::AcbFieldElem)
```

Return the Riemann zeta function evaluated at $x$.

source

Nemo.barnes_g – Method.

```
barnes_g(x::AcbFieldElem)
```

Return the Barnes $G$-function, evaluated at $x$.

source

Nemo.log_barnes_g – Method.

```
log_barnes_g(x::AcbFieldElem)
```

Return the logarithm of the Barnes $G$-function, evaluated at $x$.

source

Nemo.erf – Method.

```
erf(x::AcbFieldElem)
```

Return the error function evaluated at $x$.

source

Nemo.erfi – Method.

```
erfi(x::AcbFieldElem)
```

Return the imaginary error function evaluated at $x$.

source

Nemo.exp_integral_ei – Method.

```
exp_integral_ei(x::AcbFieldElem)
```

Return the exponential integral evaluated at $x$.

source

Nemo.sin_integral – Method.

```
sin_integral(x::AcbFieldElem)
```

Return the sine integral evaluated at $x$.

source

Nemo.cos_integral – Method.

```
cos_integral(x::AcbFieldElem)
```

Return the exponential cosine integral evaluated at $x$.

source

Nemo.sinh_integral – Method.

```
sinh_integral(x::AcbFieldElem)
```

Return the hyperbolic sine integral evaluated at $x$.

source

Nemo.cosh_integral – Method.

```
cosh_integral(x::AcbFieldElem)
```

Return the hyperbolic cosine integral evaluated at $x$.

source

Nemo.dedekind_eta – Method.

```
dedekind_eta(x::AcbFieldElem)
```

Return the Dedekind eta function $\eta(\tau)$ at $\tau = x$.

source

Nemo.modular_weber_f – Method.

```
modular_weber_f(x::AcbFieldElem)
```

Return the modular Weber function $\mathfrak{f}(\tau) = \frac{\eta^2(\tau)}{\eta(\tau/2)\eta(2\tau)}$, at $x$ in the complex upper half plane.

source

Nemo.modular_weber_f1 – Method.

```
modular_weber_f1(x::AcbFieldElem)
```

Return the modular Weber function $\mathfrak{f}_1(\tau) = \frac{\eta(\tau/2)}{\eta(\tau)}$, at $x$ in the complex upper half plane.

source

Nemo.modular_weber_f2 – Method.

```
modular_weber_f2(x::AcbFieldElem)
```

Return the modular Weber function $\mathfrak{f}_2(\tau) = \frac{\sqrt{2}\eta(2\tau)}{\eta(\tau)}$ at $x$ in the complex upper half plane.

source

Nemo.j_invariant – Method.

```
j_invariant(x::AcbFieldElem)
```

Return the $j$-invariant $j(\tau)$ at $\tau = x$.

source

Nemo.modular_lambda – Method.

```
modular_lambda(x::AcbFieldElem)
```

Return the modular lambda function $\lambda(\tau)$ at $\tau = x$.

source

Nemo.modular_delta – Method.

```
modular_delta(x::AcbFieldElem)
```

Return the modular delta function $\Delta(\tau)$ at $\tau = x$.

source

Nemo.eisenstein_g – Method.

```
eisenstein_g(k::Int, x::AcbFieldElem)
```

Return the non-normalized Eisenstein series $G_k(\tau)$ of $\mathrm{SL}_2(\mathbb{Z})$. Also defined for $\tau = i\infty$.

source

Nemo.elliptic_k – Method.

```
elliptic_k(x::AcbFieldElem)
```

Return the complete elliptic integral $K(x)$.

source

Nemo.elliptic_e – Method.

```
elliptic_e(x::AcbFieldElem)
```

Return the complete elliptic integral $E(x)$.

source

Nemo.agm – Method.

```
agm(x::AcbFieldElem)
```

Return the arithmetic-geometric mean of $1$ and $x$.

source

Nemo.agm – Method.

```
agm(x::AcbFieldElem, y::AcbFieldElem)
```

Return the arithmetic-geometric mean of $x$ and $y$.

source

Nemo.polygamma – Method.

```
polygamma(s::AcbFieldElem, a::AcbFieldElem)
```

Return the generalised polygamma function $\psi(s, z)$.

source

Nemo.zeta – Method.

```
zeta(s::AcbFieldElem, a::AcbFieldElem)
```

Return the Hurwitz zeta function $\zeta(s, a)$.

source

AbstractAlgebra.Generic.rising_factorial – Method.

```
rising_factorial(x::AcbFieldElem, n::Int)
```

Return the rising factorial $x(x + 1) \ldots (x + n - 1)$ as an Acb.

source

AbstractAlgebra.Generic.rising_factorial2 – Method.

```
rising_factorial2(x::AcbFieldElem, n::Int)
```

Return a tuple containing the rising factorial $x(x + 1) \ldots (x + n - 1)$ and its derivative.

source

Nemo.polylog – Method.

```
polylog(s::Union{AcbFieldElem,Int}, a::AcbFieldElem)
```

Return the polylogarithm $\mathrm{Li}_s(a)$.

source

Nemo.log_integral – Method.

```
log_integral(x::AcbFieldElem)
```

Return the logarithmic integral, evaluated at $x$.

source

Nemo.log_integral_offset – Method.

```
log_integral_offset(x::AcbFieldElem)
```

Return the offset logarithmic integral, evaluated at $x$.

source

Nemo.exp_integral_e – Method.

```
exp_integral_e(s::AcbFieldElem, x::AcbFieldElem)
```

Return the generalised exponential integral $E_s(x)$.

source

Nemo.gamma – Method.

```
gamma(s::AcbFieldElem, x::AcbFieldElem)
```

Return the upper incomplete gamma function $\Gamma(s, x)$.

source

Nemo.gamma_regularized – Method.

```
gamma_regularized(s::AcbFieldElem, x::AcbFieldElem)
```

Return the regularized upper incomplete gamma function $\Gamma(s, x)/\Gamma(s)$.

source

Nemo.gamma_lower – Method.

```
gamma_lower(s::AcbFieldElem, x::AcbFieldElem)
```

Return the lower incomplete gamma function $\gamma(s, x)/\Gamma(s)$.

source

Nemo.gamma_lower_regularized – Method.

```
gamma_lower_regularized(s::AcbFieldElem, x::AcbFieldElem)
```

Return the regularized lower incomplete gamma function $\gamma(s, x)/\Gamma(s)$.

source

Nemo.airy_ai – Method.

```
airy_ai(x::AcbFieldElem)
```

Return the Airy function $\mathrm{Ai}(x)$.

source

Nemo.airy_ai_prime – Method.

```
airy_ai_prime(x::AcbFieldElem)
```

Return the derivative of the Airy function $\mathrm{Ai}'(x)$.

source

Nemo.airy_bi – Method.

```
airy_bi(x::AcbFieldElem)
```

Return the Airy function $\mathrm{Bi}(x)$.

source

Nemo.airy_bi_prime – Method.

```
airy_bi_prime(x::AcbFieldElem)
```

Return the derivative of the Airy function $\mathrm{Bi}'(x)$.

source

Nemo.bessel_j – Method.

```
bessel_j(nu::AcbFieldElem, x::AcbFieldElem)
```

Return the Bessel function $J_\nu(x)$.

source

Nemo.bessel_y – Method.

```
bessel_y(nu::AcbFieldElem, x::AcbFieldElem)
```

Return the Bessel function $Y_\nu(x)$.

source

Nemo.bessel_i – Method.

```
bessel_i(nu::AcbFieldElem, x::AcbFieldElem)
```

Return the Bessel function $I_\nu(x)$.

source

Nemo.bessel_k – Method.

```
bessel_k(nu::AcbFieldElem, x::AcbFieldElem)
```

Return the Bessel function $K_\nu(x)$.

source

Nemo.hypergeometric_1f1 – Method.

```
hypergeometric_1f1(a::AcbFieldElem, b::AcbFieldElem, x::AcbFieldElem)
```

Return the confluent hypergeometric function ${}_1F_1(a, b, x)$.

source

Nemo.hypergeometric_1f1_regularized – Method.

```
hypergeometric_1f1_regularized(a::AcbFieldElem, b::AcbFieldElem, x::AcbFieldElem)
```

Return the regularized confluent hypergeometric function ${}_1F_1(a, b, x)/\Gamma(b)$.

source

Nemo.hypergeometric_u – Method.

```
hypergeometric_u(a::AcbFieldElem, b::AcbFieldElem, x::AcbFieldElem)
```

Return the confluent hypergeometric function $U(a, b, x)$.

source

Nemo.hypergeometric_2f1 – Method.

```
hypergeometric_2f1(a::AcbFieldElem, b::AcbFieldElem, c::AcbFieldElem, x::AcbFieldElem; flags=0)
```

Return the Gauss hypergeometric function ${}_2F_1(a, b, c, x)$.

source

Nemo.jacobi_theta – Method.

```
jacobi_theta(z::AcbFieldElem, tau::AcbFieldElem)
```

Return a tuple of four elements containing the Jacobi theta function values $\theta_1, \theta_2, \theta_3, \theta_4$ evaluated at $z, \tau$.

source

Nemo.weierstrass_p – Method.

```
weierstrass_p(z::AcbFieldElem, tau::AcbFieldElem)
```

Return the Weierstrass elliptic function $\wp(z, \tau)$.

source

**Examples**

```
julia> CC = AcbField(64)
Complex Field with 64 bits of precision and error bounds

julia> s = CC(1, 2)
1.0000000000000000000 + 2.0000000000000000000*im

julia> z = CC("1.23", "3.45")
[1.2300000000000000000 +/- 3.49e-20] + [3.4500000000000000000 +/- 8.70e-20]*im

julia> a = sin(z)^2 + cos(z)^2
[1.00000000000000 +/- 2.76e-16] + [+/- 2.11e-16]*im

julia> b = zeta(z)
[0.685803329024164062 +/- 2.05e-19] + [-0.038574782404586856 +/- 2.71e-19]*im

julia> c = bessel_j(s, z)
[0.63189634741402481 +/- 4.04e-18] + [0.00970090757446076 +/- 3.79e-18]*im

julia> d = hypergeometric_1f1(s, s+1, z)
[-1.3355297330012291 +/- 4.42e-17] + [-0.1715020340928697 +/- 3.61e-17]*im
```

### Linear dependence

Nemo.lindep – Method.

```
lindep(A::Vector{AcbFieldElem}, bits::Int)
```

Find a small linear combination of the entries of the array $A$ that is small (using LLL). The entries are first scaled by the given number of bits before truncating the real and imaginary parts to integers for use in LLL. This function can be used to find linear dependence between a list of complex numbers. The algorithm is heuristic only and returns an array of Nemo integers representing the linear combination.

source

Nemo.lindep – Method.

```
lindep(A::Matrix{AcbFieldElem}, bits::Int)
```

Find a (common) small linear combination of the entries in each row of the array $A$, that is small (using LLL). It is assumed that the complex numbers in each row of the array share the same linear combination. The entries are first scaled by the given number of bits before truncating the real and imaginary parts to integers for use in LLL. This function can be used to find a common linear dependence shared across a number of lists of complex numbers. The algorithm is heuristic only and returns an array of Nemo integers representing the common linear combination.

source

#### Examples

```
julia> CC = AcbField(128)
Complex Field with 128 bits of precision and error bounds

julia> # These are two of the roots of x^5 + 3x + 1

julia> a = CC(1.005066947858862242879105188836477525 3, -0.937259156692891826979035858687615135 85)
[1.005066947858862302922489107004366815 09 +/- 1.80e-40] -
↪  [0.937259156692891837181491609953809529543 +/- 7.71e-41]*im

julia> b = CC(-0.331989029584509316202500694922316523 19)
-[0.331989029584509320880414406929048709571 +/- 3.62e-40]

julia> V1 = [CC(1), a, a^2, a^3, a^4, a^5]; # We recover the polynomial from one root....

julia> W = lindep(V1, 20)
6-element Vector{ZZRingElem}:
 1
 3
 0
 0
 0
 1

julia> V2 = [CC(1), b, b^2, b^3, b^4, b^5]; # ...or from two
```

```
julia> Vs = [transpose(V1); transpose(V2)];

julia> X = lindep(Vs, 20)
6-element Vector{ZZRingElem}:
 1
 3
 0
 0
 0
 1
```

# Chapter 26

# Galois fields

Nemo allows the creation of Galois fields of the form $\mathbb{Z}/p\mathbb{Z}$ for a prime $p$. Note that these are not the same as finite fields of degree 1, as Conway polynomials are not used and no generator is given.

For convenience, the following constructors are provided.

```
GF(n::UInt)
GF(n::Int)
GF(n::ZZRingElem)
```

For example, one can create the Galois field of characteristic $7$ as follows.

```
julia> R = GF(7)
Prime field of characteristic 7
```

Elements of the field are then created in the usual way.

```
julia> a = R(3)
3
```

Elements of Galois fields have type `fpFieldElem` when $p$ is given to the constructor as an `Int` or `UInt`, and of type `FpFieldElem` if $p$ is given as an `ZZRingElem`, and the type of the parent objects is `fpField` or `FpField` respectively.

The modulus $p$ of an element of a Galois field is stored in its parent object.

The `fpFieldElem` and `FpFieldElem` types belong to the abstract type `FinFieldElem` and the `fpField` and `FpField` parent object types belong to the abstract type `FinField`.

## 26.1 Galois field functionality

Galois fields in Nemo provide all the residue ring functionality of AbstractAlgebra.jl:

https://nemocas.github.io/AbstractAlgebra.jl/stable/residue

In addition, all the functionality for rings is available:

https://nemocas.github.io/AbstractAlgebra.jl/stable/ring

Below we describe the functionality that is provided in addition to these.

## 26.2 Basic manipulation

### Examples

```
julia> F = GF(3)
Prime field of characteristic 3

julia> a = characteristic(F)
3

julia> b = order(F)
3
```

# Chapter 27

# Finite fields

A finite field $K$ is represented as simple extension $K = k(\alpha) = k[x]/(f)$, where $k$ can be

- a prime field $\mathbf{F}_p$ ($K$ is then an *absolute finite field*), or
- an arbitrary finite field $k$ ($K$ is then a *relative finite field*).

In both cases, we call $k$ the *base field* of $K$, $\alpha$ a *generator* and $f$ the *defining polynomial* of $K$.

Note that all field theoretic properties (like basis, degree or trace) are defined with respect to the base field. Methods with prefix `absolute_` return

## 27.1 Finite field functionality

Finite fields in Nemo provide all the field functionality described in AbstractAlgebra:

https://nemocas.github.io/AbstractAlgebra.jl/stable/field

Below we describe the functionality that is provided in addition to this.

## 27.2 Constructors

`Nemo.finite_field` – Function.

```
finite_field(p::IntegerUnion, d::Int, s::VarName = :o; cached::Bool = true, check::Bool = true)
finite_field(q::IntegerUnion, s::VarName = :o; cached::Bool = true, check::Bool = true)
finite_field(f::FqPolyRingElem, s::VarName = :o; cached::Bool = true, check::Bool = true)
```

Return a tuple $(K, x)$ of a finite field $K$ of order $q = p^d$, where $p$ is a prime, and a generator $x$ of $K$ (see `gen` for a definition). The identifier $s$ is used to designate how the finite field generator will be printed.

If a polynomial $f \in k[X]$ over a finite field $k$ is specified, the finite field $K = k[X]/(f)$ will be constructed as a finite field with base field $k$.

See also `GF` which only returns $K$.

**Examples**

```
julia> K, a = finite_field(3, 2, "a")
(Finite field of degree 2 and characteristic 3, a)

julia> K, a = finite_field(9, "a")
(Finite field of degree 2 and characteristic 3, a)

julia> Kx, x = K["x"];

julia> L, b = finite_field(x^3 + x^2 + x + 2, "b")
(Finite field of degree 3 over GF(3, 2), b)
```

source

Nemo.GF – Function.

```
GF(p::IntegerUnion, d::Int, s::VarName = :o; cached::Bool = true, check::Bool = true)
GF(q::IntegerUnion, s::VarName = :o; cached::Bool = true, check::Bool = true)
GF(f::FqPolyRingElem, s::VarName = :o; cached::Bool = true, check::Bool = true)
```

Return a finite field $K$ of order $q = p^d$, where $p$ is a prime. The identifier $s$ is used to designate how the finite field generator will be printed.

If a polynomial $f \in k[X]$ over a finite field $k$ is specified, the finite field $K = k[X]/(f)$ will be constructed as a finite field with base field $k$.

See also finite_field which additionally returns a finite field generator of $K$.

**Examples**

```
julia> K = GF(3, 2, "a")
Finite field of degree 2 and characteristic 3

julia> K = GF(9, "a")
Finite field of degree 2 and characteristic 3

julia> Kx, x = K["x"];

julia> L = GF(x^3 + x^2 + x + 2, "b")
Finite field of degree 3 over GF(3, 2)
```

source

## 27.3 Field functionality

AbstractAlgebra.Generic.base_field – Method.

```
base_field(F::FqField)
```

Return the base field of F.

source

Nemo.prime_field – Method.

```
prime_field(F::FqField)
```

Return the prime field of F.

source

AbstractAlgebra.degree – Method.

```
degree(K::FqField) -> Int
```

Return the degree of the given finite field over the base field.

**Examples**

```
julia> K, a = finite_field(3, 2, "a");

julia> degree(K)
2

julia> Kx, x = K["x"];

julia> L, b = finite_field(x^3 + x^2 + x + 2, "b");

julia> degree(L)
3
```

source

Nemo.absolute_degree – Method.

```
absolute_degree(a::FqField)
```

Return the degree of the given finite field over the prime field.

source

Nemo.is_absolute – Method.

```
is_absolute(F::FqField)
```

Return whether the base field of $F$ is a prime field.

source

AbstractAlgebra.Generic.defining_polynomial – Method.

```
defining_polynomial([R::FqPolyRing], K::FqField)
```

Return the defining polynomial of K as a polynomial over the base field of K.

If the polynomial ring R is specified, the polynomial will be an element of R.

**Examples**

```
julia> K, a = finite_field(9, "a");

julia> defining_polynomial(K)
x^2 + 2*x + 2

julia> Ky, y = K["y"];

julia> L, b = finite_field(y^3 + y^2 + y + 2, "b");

julia> defining_polynomial(L)
y^3 + y^2 + y + 2
```

source

## 27.4 Element functionality

`AbstractAlgebra.gen` – Method.

```
gen(L::FqField)
```

Return a $K$-algebra generator a of the finite field $L$, where $K$ is the base field of $L$. The element a satisfies `defining_polyomial(a) == 0`.

Note that this is in general not a multiplicative generator and can be zero, if $L/K$ is an extension of degree one.

source

`AbstractAlgebra.is_gen` – Method.

```
is_gen(a::FqFieldElem)
```

Return `true` if the given finite field element is the generator of the finite field over its base field, otherwise return `false`.

source

`LinearAlgebra.tr` – Method.

```
tr(x::FqFieldElem)
```

Return the trace of $x$. This is an element of the base field.

source

`Nemo.absolute_tr` – Method.

```
absolute_tr(x::FqFieldElem)
```

Return the absolute trace of $x$. This is an element of the prime field.

source

`LinearAlgebra.norm` – Method.

```
norm(x::FqFieldElem)
```

Return the norm of $x$. This is an element of the base field.

source

`Nemo.absolute_norm` – Method.

```
absolute_norm(x::FqFieldElem)
```

Return the absolute norm of $x$. This is an element of the prime field.

source

`AbstractAlgebra.lift` – Method.

```
lift(R::FqPolyRing, a::FqFieldElem) -> FqPolyRingElem
```

Given a polynomial ring over the base field of the parent of a, return a lift such that `parent(a)(lift(R, a)) == a` is `true`.

source

`AbstractAlgebra.lift` – Method.

```
lift(::ZZRing, x::FqFieldElem) -> ZZRingElem
```

Given an element $x$ of a prime field $\mathbf{F}_p$, return a preimage under the canonical map $\mathbf{Z} \to \mathbf{F}_p$.

**Examples**

```
julia> K = GF(19);

julia> lift(ZZ, K(3))
3
```

source

# Chapter 28

# Finite field embeddings

## 28.1 Introduction

Nemo allows the construction of finite field embeddings making use of the algorithm of Bosma, Cannon and Steel behind the scenes to ensure compatibility. Critical routines (e.g. polynomial factorization, matrix computations) are provided by the C library FLINT, whereas high level tasks are written directly in Nemo.

## 28.2 Embedding functionality

It is possible to explicitly call the embedding `embed` function to create an embedding, but it is also possible to directly ask for the conversion of a finite field element `x` in some other finite field `k` via calling `k(x)`. The resulting embedding is of type `FinFieldMorphism`. It is also possible to compute the preimage map of an embedding via the `preimage_map` function, applied to an embedding or directly to the finite fields (this actually first computes the embedding), or via conversion. An error is thrown if the element you want to compute the preimage of is not in the image of the embedding.

### Computing an embedding

`Nemo.embed` – Method.

```
embed(k::T, K::T) where T <: FinField
```

Embed $k$ in $K$, with some additional computations in order to satisfy compatibility conditions with previous and future embeddings.

source

**Examples**

```
julia> k2, x2 = finite_field(19, 2, "x2")
(Finite field of degree 2 and characteristic 19, x2)

julia> k4, x4 = finite_field(19, 4, "x4")
(Finite field of degree 4 and characteristic 19, x4)

julia> f = embed(k2, k4)
Morphism of finite fields
  from finite field of degree 2 and characteristic 19
```

```
  to finite field of degree 4 and characteristic 19

julia> y = f(x2)
13*x4^3 + 14*x4^2 + 10*x4 + 3

julia> z = k4(x2)
13*x4^3 + 14*x4^2 + 10*x4 + 3
```

### Computing the preimage of an embedding

`AbstractAlgebra.Generic.preimage_map` – Method.

```
preimage_map(k::T, k::T) where T <: FinField
```

Computes the preimage map corresponding to the embedding of $k$ into $K$.

source

`AbstractAlgebra.Generic.preimage_map` – Method.

```
preimage_map(f::FinFieldMorphism)
```

Compute the preimage map corresponding to the embedding $f$.

source

```
preimage_map(f::FinFieldPreimage)
```

Compute the preimage map corresponding to the preimage of the embedding $f$, i.e. return the embedding $f$.

source

#### Examples

```
julia> k7, x7 = finite_field(13, 7, "x7")
(Finite field of degree 7 and characteristic 13, x7)

julia> k21, x21 = finite_field(13, 21, "x21")
(Finite field of degree 21 and characteristic 13, x21)

julia> s = preimage_map(k7, k21)
Preimage of a morphism
  from finite field of degree 7 and characteristic 13
  to finite field of degree 21 and characteristic 13

julia> y = k21(x7);

julia> z = s(y)
x7
```

```
julia> t = k7(y)
x7
```

# Chapter 29

# Number field arithmetic

Number fields are provided in Nemo by FLINT. This allows construction of absolute number fields and basic arithmetic computations therein.

Number fields are constructed using the `number_field` function.

The types of number field elements in Nemo are given in the following table, along with the libraries that provide them and the associated types of the parent objects.

| Library | Field | Element type | Parent type |
|---|---|---|---|
| FLINT | $\mathbb{Q}[x]/(f)$ | `AbsSimpleNumFieldElem` | `AbsSimpleNumField` |

All the number field types belong to the `Field` abstract type and the number field element types belong to the `FieldElem` abstract type.

The Hecke.jl library radically expands on number field functionality, providing ideals, orders, class groups, relative extensions, class field theory, etc.

The basic number field type used in Hecke is the Nemo `AbsSimpleNumField` type, making the two libraries tightly integrated.

https://thofma.github.io/Hecke.jl/stable/

## 29.1 Number field functionality

The number fields in Nemo provide all of the AbstractAlgebra field functionality:

https://nemocas.github.io/AbstractAlgebra.jl/stable/field

Below, we document the additional functionality provided for number field elements.

### Constructors

In order to construct number field elements in Nemo, one must first construct the number field itself. This is accomplished with one of the following constructors.

`Nemo.number_field` – Method.

```
number_field(f::QQPolyRingElem, s::VarName;
             cached::Bool = true, check::Bool = true)
```

Return a tuple $R, x$ consisting of the parent object $R$ and generator $x$ of the number field $\mathbb{Q}[x]/(f)$ where $f$ is the supplied polynomial. The supplied string s specifies how the generator of the number field should be printed. If s is not specified, it defaults to _a.

**Examples**

```
julia> R, x = polynomial_ring(QQ, "x");

julia> K, a = number_field(x^3 + 3x + 1, "a")
(Number field of degree 3 over QQ, a)

julia> K
Number field with defining polynomial x^3 + 3*x + 1
  over rational field
```

source

Nemo.cyclotomic_field – Method.

```
cyclotomic_field(n::Int, s::VarName = "z_$n", t = "_\$"; cached::Bool = true)
```

Return a tuple $R, x$ consisting of the parent object $R$ and generator $x$ of the $n$-th cyclotomic field, $\mathbb{Q}(\zeta_n)$. The supplied string s specifies how the generator of the number field should be printed. If provided, the string t specifies how the generator of the polynomial ring from which the number field is constructed, should be printed. If it is not supplied, a default dollar sign will be used to represent the variable.

source

Nemo.cyclotomic_real_subfield – Method.

```
cyclotomic_real_subfield(n::Int, s::VarName = "(z_$n + 1/z_$n)", t = "\$"; cached = true)
```

Return a tuple $R, x$ consisting of the parent object $R$ and generator $x$ of the totally real subfield of the $n$-th cyclotomic field, $\mathbb{Q}(\zeta_n)$. The supplied string s specifies how the generator of the number field should be printed. If provided, the string t specifies how the generator of the polynomial ring from which the number field is constructed, should be printed. If it is not supplied, a default dollar sign will be used to represent the variable.

source

Here are some examples of creating number fields and making use of the resulting parent objects to coerce various elements into those fields.

**Examples**

```
julia> R, x = polynomial_ring(QQ, "x")
(Univariate polynomial ring in x over QQ, x)

julia> K, a = number_field(x^3 + 3x + 1, "a")
(Number field of degree 3 over QQ, a)

julia> L, b = cyclotomic_field(5, "b")
```

```
(Cyclotomic field of order 5, b)

julia> M, c = cyclotomic_real_subfield(5, "c")
(Maximal real subfield of cyclotomic field of order 5, c)

julia> d = K(3)
3

julia> f = L(b)
b

julia> g = L(ZZ(11))
11

julia> h = L(ZZ(11)//3)
11//3

julia> k = M(5)
5
```

### Number field element constructors

`AbstractAlgebra.gen` – Method.

```
gen(a::AbsSimpleNumField)
```

Return the generator of the given number field, i.e., a symbolic root of the defining polynomial.

source

The easiest way of constructing number field elements is to use element arithmetic with the generator, to construct the desired element by its representation as a polynomial. See the following examples for how to do this.

#### Examples

```
julia> R, x = polynomial_ring(QQ, "x")
(Univariate polynomial ring in x over QQ, x)

julia> K, a = number_field(x^3 + 3x + 1, "a")
(Number field of degree 3 over QQ, a)

julia> d = gen(K)
a

julia> f = a^2 + 2a - 7
a^2 + 2*a - 7
```

### Basic functionality

`AbstractAlgebra.mul_red!` – Method.

```
mul_red!(z::AbsSimpleNumFieldElem, x::AbsSimpleNumFieldElem, y::AbsSimpleNumFieldElem,
↪  red::Bool)
```

Multiply $x$ by $y$ and set the existing number field element $z$ to the result. Reduction modulo the defining polynomial is only performed if `red` is set to `true`. Note that $x$ and $y$ must be reduced. This function is provided for performance reasons as it saves allocating a new object for the result and eliminates associated garbage collection.

source

`AbstractAlgebra.reduce!` – Method.

```
reduce!(x::AbsSimpleNumFieldElem)
```

Reduce the given number field element by the defining polynomial, in-place. This only needs to be done after accumulating values computed by `mul_red!` where reduction has not been performed. All standard Nemo number field functions automatically reduce their outputs.

source

The following coercion function is provided for a number field $R$.

```
R(f::QQPolyRingElem)
```

Coerce the given rational polynomial into the number field $R$, i.e. consider the polynomial to be the representation of a number field element and return it.

Conversely, if $R$ is the polynomial ring to which the generating polynomial of a number field belongs, then we can coerce number field elements into the ring $R$ using the following function.

```
R(b::AbsSimpleNumFieldElem)
```

Coerce the given number field element into the polynomial ring $R$ of which the number field is a quotient.

**Examples**

```
julia> R, x = polynomial_ring(QQ, "x")
(Univariate polynomial ring in x over QQ, x)

julia> K, a = number_field(x^3 + 3x + 1, "a")
(Number field of degree 3 over QQ, a)

julia> f = R(a^2 + 2a + 3)
x^2 + 2*x + 3

julia> g = K(x^2 + 2x + 1)
a^2 + 2*a + 1
```

### Basic manipulation

`AbstractAlgebra.var` – Method.

```
var(a::AbsSimpleNumField)
```

Returns the identifier (as a symbol, not a string), that is used for printing the generator of the given number field.

source

`AbstractAlgebra.is_gen` – Method.

```
is_gen(a::AbsSimpleNumFieldElem)
```

Return `true` if the given number field element is the generator of the number field, otherwise return `false`.

source

`AbstractAlgebra.coeff` – Method.

```
coeff(x::AbsSimpleNumFieldElem, n::Int)
```

Return the $n$-th coefficient of the polynomial representation of the given number field element. Coefficients are numbered from $0$, starting with the constant coefficient.

source

`Base.denominator` – Method.

```
denominator(a::AbsSimpleNumFieldElem)
```

Return the denominator of the polynomial representation of the given number field element.

source

`AbstractAlgebra.degree` – Method.

```
degree(a::AbsSimpleNumField)
```

Return the degree of the given number field, i.e. the degree of its defining polynomial.

source

#### Examples

```
julia> R, x = polynomial_ring(QQ, "x")
(Univariate polynomial ring in x over QQ, x)

julia> K, a = number_field(x^3 + 3x + 1, "a")
(Number field of degree 3 over QQ, a)

julia> d = a^2 + 2a - 7
a^2 + 2*a - 7

julia> m = gen(K)
a

julia> c = coeff(d, 1)
2

julia> is_gen(m)
true

julia> q = degree(K)
3
```

### Norm and trace

LinearAlgebra.norm – Method.

```
norm(a::AbsSimpleNumFieldElem)
```

Return the absolute norm of $a$. The result will be a rational number.

source

LinearAlgebra.tr – Method.

```
tr(a::AbsSimpleNumFieldElem)
```

Return the absolute trace of $a$. The result will be a rational number.

source

### Examples

```
julia> R, x = polynomial_ring(QQ, "x")
(Univariate polynomial ring in x over QQ, x)

julia> K, a = number_field(x^3 + 3x + 1, "a")
(Number field of degree 3 over QQ, a)

julia> c = 3a^2 - a + 1
3*a^2 - a + 1

julia> d = norm(c)
```

```
113

julia> f = tr(c)
-15
```

# Chapter 30

# Padics

P-adic fields are provided in Nemo by FLINT. This allows construction of $p$-adic fields for any prime $p$.

P-adic fields are constructed using the `padic_field` function.

The types of $p$-adic fields in Nemo are given in the following table, along with the libraries that provide them and the associated types of the parent objects.

| Library | Field | Element type | Parent type |
|---|---|---|---|
| FLINT | $\mathbb{Q}_p$ | `PadicFieldElem` | `PadicField` |

All the $p$-adic field types belong to the `Field` abstract type and the $p$-adic field element types belong to the `FieldElem` abstract type.

## 30.1 P-adic functionality

P-adic fields in Nemo implement all the AbstractAlgebra field functionality:.

https://nemocas.github.io/AbstractAlgebra.jl/stable/field

Below, we document all the additional function that is provide by Nemo for p-adic fields.

### Constructors

In order to construct $p$-adic field elements in Nemo, one must first construct the $p$-adic field itself. This is accomplished with one of the following constructors.

`Nemo.padic_field` – Function.

```
padic_field(p::Integer; precision::Int=64, cached::Bool=true, check::Bool=true)
padic_field(p::ZZRingElem; precision::Int=64, cached::Bool=true, check::Bool=true)
```

Return the $p$-adic field for the given prime $p$. The default absolute precision of elements of the field may be set with `precision`.

source

Here are some examples of creating $p$-adic fields and making use of the resulting parent objects to coerce various elements into those fields.

### Examples

```
julia> R = padic_field(7, precision = 30)
Field of 7-adic numbers

julia> S = padic_field(ZZ(65537), precision = 30)
Field of 65537-adic numbers

julia> a = R()
O(7^30)

julia> b = S(1)
65537^0 + O(65537^30)

julia> c = S(ZZ(123))
123*65537^0 + O(65537^30)

julia> d = R(ZZ(1)//7^2)
7^-2 + O(7^28)
```

### Big-oh notation

Elements of p-adic fields can be constructed using the big-oh notation. For this purpose we define the following functions.

`AbstractAlgebra.O` – Method.

```
O(R::PadicField, m::Integer)
```

Construct the value $0 + O(p^n)$ given $m = p^n$. An exception results if $m$ is not found to be a power of `p = prime(R)`.

source

`AbstractAlgebra.O` – Method.

```
O(R::PadicField, m::ZZRingElem)
```

Construct the value $0 + O(p^n)$ given $m = p^n$. An exception results if $m$ is not found to be a power of `p = prime(R)`.

source

`AbstractAlgebra.O` – Method.

```
O(R::PadicField, m::QQFieldElem)
```

Construct the value $0 + O(p^n)$ given $m = p^n$. An exception results if $m$ is not found to be a power of `p = prime(R)`.

source

The $O(p^n)$ construction can be used to construct $p$-adic values of precision $n$ by adding it to integer values representing the $p$-adic value modulo $p^n$ as in the examples.

**Examples**

```
julia> R = padic_field(7, precision = 30)
Field of 7-adic numbers

julia> S = padic_field(ZZ(65537), precision = 30)
Field of 65537-adic numbers

julia> c = 1 + 2*7 + 4*7^2 + O(R, 7^3)
7^0 + 2*7^1 + 4*7^2 + O(7^3)

julia> d = 13 + 357*ZZ(65537) + O(S, ZZ(65537)^12)
13*65537^0 + 357*65537^1 + O(65537^12)

julia> f = ZZ(1)//7^2 + ZZ(2)//7 + 3 + 4*7 + O(R, 7^2)
7^-2 + 2*7^-1 + 3*7^0 + 4*7^1 + O(7^2)
```

Beware that the expression `1 + 2*p + 3*p^2 + O(R, p^n)` is actually computed as a normal Julia expression. Therefore if `Int` values are used instead of `ZZRingElems` or Julia `BigInts`, overflow may result in evaluating the value.

### Basic manipulation

`AbstractAlgebra.Generic.prime` – Method.

```
prime(R::PadicField)
```

Return the prime $p$ for the given $p$-adic field.

source

`Base.precision` – Method.

```
precision(a::PadicFieldElem)
```

Return the precision of the given $p$-adic field element, i.e. if the element is known to $O(p^n)$ this function will return $n$.

source

`AbstractAlgebra.valuation` – Method.

```
valuation(a::PadicFieldElem)
```

Return the valuation of the given $p$-adic field element, i.e. if the given element is divisible by $p^n$ but not a higher power of $p$ then the function will return $n$.

source

AbstractAlgebra.lift – Method.

```
lift(R::ZZRing, a::PadicFieldElem)
```

Return a lift of the given $p$-adic field element to $\mathbb{Z}$.

source

AbstractAlgebra.lift – Method.

```
lift(R::QQField, a::PadicFieldElem)
```

Return a lift of the given $p$-adic field element to $\mathbb{Q}$.

source

**Examples**

```
julia> R = padic_field(7, precision = 30)
Field of 7-adic numbers

julia> a = 1 + 2*7 + 4*7^2 + O(R, 7^3)
7^0 + 2*7^1 + 4*7^2 + O(7^3)

julia> b = 7^2 + 3*7^3 + O(R, 7^5)
7^2 + 3*7^3 + O(7^5)

julia> c = R(2)
2*7^0 + O(7^30)

julia> k = precision(a)
3

julia> m = prime(R)
7

julia> n = valuation(b)
2

julia> p = lift(ZZ, a)
211

julia> q = lift(QQ, divexact(a, b))
337//49
```

### Square root

AbstractAlgebra.is_square – Method.

```
is_square(a::NCRingElement)
```

Return `true` iff a is the square of a value in its own ring. See also `is_square(A::MatElem)` which tests whether a matrix has square shape.

source

**Examples**

```
julia> R = padic_field(7, precision = 30)
Field of 7-adic numbers

julia> a = 1 + 7 + 2*7^2 + O(R, 7^3)
7^0 + 7^1 + 2*7^2 + O(7^3)

julia> b = 2 + 3*7 + O(R, 7^5)
2*7^0 + 3*7^1 + O(7^5)

julia> c = 7^2 + 2*7^3 + O(R, 7^4)
7^2 + 2*7^3 + O(7^4)

julia> d = sqrt(a)
7^0 + 4*7^1 + 3*7^2 + O(7^3)

julia> f = sqrt(b)
3*7^0 + 5*7^1 + 7^2 + 7^3 + O(7^5)

julia> f = sqrt(c)
7^1 + 7^2 + O(7^3)

julia> g = sqrt(R(121))
3*7^0 + 5*7^1 + 6*7^2 + 6*7^3 + 6*7^4 + 6*7^5 + 6*7^6 + 6*7^7 + 6*7^8 + 6*7^9 + 6*7^10 + 6*7^11 +
↪  6*7^12 + 6*7^13 + 6*7^14 + 6*7^15 + 6*7^16 + 6*7^17 + 6*7^18 + 6*7^19 + 6*7^20 + 6*7^21 +
↪  6*7^22 + 6*7^23 + 6*7^24 + 6*7^25 + 6*7^26 + 6*7^27 + 6*7^28 + 6*7^29 + O(7^30)

julia> g^2 == R(121)
true
```

### Special functions

Base.exp – Method.

```
exp(a::PadicFieldElem)
```

Return the $p$-adic exponential of $a$, assuming the $p$-adic exponential function converges at $a$.

source

Base.log – Method.

```
log(a::PadicFieldElem)
```

Return the $p$-adic logarithm of $a$, assuming the $p$-adic logarithm converges at $a$.

source

Nemo.teichmuller – Method.

```
teichmuller(a::PadicFieldElem)
```

Return the Teichmuller lift of the $p$-adic value $a$. We require the valuation of $a$ to be non-negative. The precision of the output will be the same as the precision of the input. For convenience, if $a$ is congruent to zero modulo $p$ we return zero. If the input is not valid an exception is thrown.

source

**Examples**

```
julia> R = padic_field(7, precision = 30)
Field of 7-adic numbers

julia> a = 1 + 7 + 2*7^2 + O(R, 7^3)
7^0 + 7^1 + 2*7^2 + O(7^3)

julia> b = 2 + 5*7 + 3*7^2 + O(R, 7^3)
2*7^0 + 5*7^1 + 3*7^2 + O(7^3)

julia> c = 3*7 + 2*7^2 + O(R, 7^5)
3*7^1 + 2*7^2 + O(7^5)

julia> c = exp(c)
7^0 + 3*7^1 + 3*7^2 + 4*7^3 + 4*7^4 + O(7^5)

julia> d = log(a)
7^1 + 5*7^2 + O(7^3)

julia> c = exp(R(0))
7^0 + O(7^30)

julia> d = log(R(1))
O(7^30)

julia> f = teichmuller(b)
2*7^0 + 4*7^1 + 6*7^2 + O(7^3)
```

# Chapter 31

# Qadics

Q-adic fields, that is, unramified extensions of p-adic fields, are provided in Nemo by FLINT. This allows construction of $q$-adic fields for any prime power $q$.

Q-adic fields are constructed using the `qadic_field` function.

The types of $q$-adic fields in Nemo are given in the following table, along with the libraries that provide them and the associated types of the parent objects.

| Library | Field | Element type | Parent type |
|---|---|---|---|
| FLINT | $\mathbb{Q}_q$ | `QadicFieldElem` | `QadicField` |

All the $q$-adic field types belong to the `Field` abstract type and the $q$-adic field element types belong to the `FieldElem` abstract type.

## 31.1 P-adic functionality

Q-adic fields in Nemo provide all the functionality described in AbstractAlgebra for fields:.

https://nemocas.github.io/AbstractAlgebra.jl/stable/field

Below, we document all the additional function that is provide by Nemo for q-adic fields.

### Constructors

In order to construct $q$-adic field elements in Nemo, one must first construct the $q$-adic field itself. This is accomplished with one of the following constructors.

`Nemo.qadic_field` – Function.

```
qadic_field(p::Integer, d::Int, var::String = "a"; precision::Int=64, cached::Bool=true,
↪  check::Bool=true)
qadic_field(p::ZZRingElem, d::Int, var::String = "a"; precision::Int=64, cached::Bool=true,
↪  check::Bool=true)
qadic_field(f::ZZModPolyRingElem, var::String = "a"; precision::Int=64, cached::Bool=true,
↪  check::Bool=true)
qadic_field(f::FpPolyRingElem, var::String = "a"; precision::Int=64, cached::Bool=true,
↪  check::Bool=true)
qadic_field(f::zzModPolyRingElem, var::String = "a"; precision::Int=64, cached::Bool=true,
↪  check::Bool=true)
```

```
qadic_field(f::fpPolyRingElem, var::String = "a"; precision::Int=64, cached::Bool=true,
↪  check::Bool=true)
qadic_field(f::FqPolyRingElem, var::String = "a"; precision::Int=64, cached::Bool=true,
↪  check::Bool=true)
```

Return an unramified extension $K$ of degree $d$ of a $p$-adic field for the given prime $p$. The generator of $K$ is printed as `var`.

If a polynomial $f \in \mathbf{F}_p[X]$ is given instead, the unramified extension $K = \mathbf{Q}_p[X]/(f)$ defined by $f$ is returned. The polynomial $f$ must be irreducible.

The default absolute precision of elements of $K$ may be set with `precision`.

See also `unramified_extension`.

source

`Nemo.unramified_extension` – Function.

```
unramified_extension(Qp::PadicField, d::Int, var::String = "a"; precision::Int=64,
↪  cached::Bool=true)
unramified_extension(Qp::PadicField, f::ZZModPolyRingElem, var::String = "a";
↪  precision::Int=precision(Qp), cached::Bool=true, check::Bool=true)
unramified_extension(Qp::PadicField, f::FpPolyRingElem, var::String = "a";
↪  precision::Int=precision(Qp), cached::Bool=true, check::Bool=true)
unramified_extension(Qp::PadicField, f::zzModPolyRingElem, var::String = "a";
↪  precision::Int=precision(Qp), cached::Bool=true, check::Bool=true)
unramified_extension(Qp::PadicField, f::fpPolyRingElem, var::String = "a";
↪  precision::Int=precision(Qp), cached::Bool=true, check::Bool=true)
unramified_extension(Qp::PadicField, f::FqPolyRingElem, var::String = "a";
↪  precision::Int=precision(Qp), cached::Bool=true, check::Bool=true)
```

Return an unramified extension $K$ of degree $d$ of the given $p$-adic field `Qp`. The generator of $K$ is printed as `var`.

If a polynomial $f$ over the residue field of `Qp` is provided, the unramified extension defined by $f$ is returned; $f$ must be irreducible.

The default absolute precision of elements of $K$ may be set with `precision`.

source

Here are some examples of creating $q$-adic fields and making use of the resulting parent objects to coerce various elements into those fields.

**Examples**

```
julia> R, p = qadic_field(7, 1, precision = 30);

julia> S, _ = qadic_field(ZZ(65537), 1, precision = 30);

julia> a = R()
0

julia> b = S(1)
```

```
65537^0 + O(65537^30)

julia> c = S(ZZ(123))
123*65537^0 + O(65537^30)

julia> d = R(ZZ(1)//7^2)
7^-2 + O(7^28)
```

### Big-oh notation

Elements of p-adic fields can be constructed using the big-oh notation. For this purpose we define the following functions.

`AbstractAlgebra.O` – Method.

```
O(R::QadicField, m::Integer)
```

Construct the value $0 + O(p^n)$ given $m = p^n$. An exception results if $m$ is not found to be a power of `p = prime(R)`.

source

`AbstractAlgebra.O` – Method.

```
O(R::QadicField, m::ZZRingElem)
```

Construct the value $0 + O(p^n)$ given $m = p^n$. An exception results if $m$ is not found to be a power of `p = prime(R)`.

source

`AbstractAlgebra.O` – Method.

```
O(R::QadicField, m::QQFieldElem)
```

Construct the value $0 + O(p^n)$ given $m = p^n$. An exception results if $m$ is not found to be a power of `p = prime(R)`.

source

The $O(p^n)$ construction can be used to construct $q$-adic values of precision $n$ by adding it to integer values representing the $q$-adic value modulo $p^n$ as in the examples.

#### Examples

```
julia> R, _ = qadic_field(7, 1, precision = 30);

julia> S, _ = qadic_field(ZZ(65537), 1, precision = 30);
```

```
julia> c = 1 + 2*7 + 4*7^2 + O(R, 7^3)
7^0 + 2*7^1 + 4*7^2 + O(7^3)

julia> d = 13 + 357*ZZ(65537) + O(S, ZZ(65537)^12)
13*65537^0 + 357*65537^1 + O(65537^12)

julia> f = ZZ(1)//7^2 + ZZ(2)//7 + 3 + 4*7 + O(R, 7^2)
7^-2 + 2*7^-1 + 3*7^0 + 4*7^1 + O(7^2)
```

Beware that the expression `1 + 2*p + 3*p^2 + O(R, p^n)` is actually computed as a normal Julia expression. Therefore if `{Int}` values are used instead of FLINT integers or Julia bignums, overflow may result in evaluating the value.

### Basic manipulation

`AbstractAlgebra.Generic.prime` – Method.

```
prime(R::QadicField)
```

Return the prime $p$ for the given $q$-adic field.

source

`Base.precision` – Method.

```
precision(a::QadicFieldElem)
```

Return the precision of the given $q$-adic field element, i.e. if the element is known to $O(p^n)$ this function will return $n$.

source

`AbstractAlgebra.valuation` – Method.

```
valuation(a::QadicFieldElem)
```

Return the valuation of the given $q$-adic field element, i.e. if the given element is divisible by $p^n$ but not a higher power of $q$ then the function will return $n$.

source

`AbstractAlgebra.lift` – Method.

```
lift(R::QQPolyRing, a::QadicFieldElem)
```

Return a lift of the given $q$-adic field element to $\mathbb{Q}[x]$.

source

AbstractAlgebra.lift – Method.

```
lift(R::ZZPolyRing, a::QadicFieldElem)
```

Return a lift of the given $q$-adic field element to $\mathbb{Z}[x]$ if possible.

source

**Examples**

```
julia> R, _ = qadic_field(7, 1, precision = 30);

julia> a = 1 + 2*7 + 4*7^2 + O(R, 7^3)
7^0 + 2*7^1 + 4*7^2 + O(7^3)

julia> b = 7^2 + 3*7^3 + O(R, 7^5)
7^2 + 3*7^3 + O(7^5)

julia> c = R(2)
2*7^0 + O(7^30)

julia> k = precision(a)
3

julia> m = prime(R)
7

julia> n = valuation(b)
2

julia> Zy, y = ZZ["y"]
(Univariate polynomial ring in y over ZZ, y)

julia> p = lift(Zy, a)
211

julia> Qx, x = QQ["x"]
(Univariate polynomial ring in x over QQ, x)

julia> q = lift(Qx, divexact(a, b))
337//49
```

### Square root

AbstractAlgebra.is_square – Method.

```
is_square(a::NCRingElement)
```

Return `true` iff a is the square of a value in its own ring. See also `is_square(A::MatElem)` which tests whether a matrix has square shape.

source

### Examples

```
julia> R, _ = qadic_field(7, 1, precision = 30);

julia> a = 1 + 7 + 2*7^2 + O(R, 7^3)
7^0 + 7^1 + 2*7^2 + O(7^3)

julia> b = 2 + 3*7 + O(R, 7^5)
2*7^0 + 3*7^1 + O(7^5)

julia> c = 7^2 + 2*7^3 + O(R, 7^4)
7^2 + 2*7^3 + O(7^4)

julia> d = sqrt(a)
7^0 + 4*7^1 + 3*7^2 + O(7^3)

julia> f = sqrt(b)
4*7^0 + 7^1 + 5*7^2 + 5*7^3 + 6*7^4 + O(7^5)

julia> f = sqrt(c)
7^1 + 7^2 + O(7^3)

julia> g = sqrt(R(121))
4*7^0 + 7^1 + O(7^30)
```

### Special functions

`Base.exp` – Method.

```
exp(a::QadicFieldElem)
```

Return the $p$-adic exponential of $a$, assuming the $p$-adic exponential function converges at $a$.

source

`Base.log` – Method.

```
log(a::QadicFieldElem)
```

Return the $p$-adic logarithm of $a$, assuming the $p$-adic logarithm converges at $a$.

source

`Nemo.teichmuller` – Method.

```
teichmuller(a::QadicFieldElem)
```

Return the Teichmuller lift of the $q$-adic value $a$. We require the valuation of $a$ to be non-negative. The precision of the output will be the same as the precision of the input. For convenience, if $a$ is congruent to zero modulo $q$ we return zero. If the input is not valid an exception is thrown.

source

Nemo.frobenius – Method.

```
frobenius(a::QadicFieldElem, e::Int = 1)
```

Return the image of the $e$-th power of Frobenius on the $q$-adic value $a$. The precision of the output will be the same as the precision of the input.

source

**Examples**

```
julia> R, _ = qadic_field(7, 1, precision = 30);

julia> a = 1 + 7 + 2*7^2 + O(R, 7^3)
7^0 + 7^1 + 2*7^2 + O(7^3)

julia> b = 2 + 5*7 + 3*7^2 + O(R, 7^3)
2*7^0 + 5*7^1 + 3*7^2 + O(7^3)

julia> c = 3*7 + 2*7^2 + O(R, 7^5)
3*7^1 + 2*7^2 + O(7^5)

julia> c = exp(c)
7^0 + 3*7^1 + 3*7^2 + 4*7^3 + 4*7^4 + O(7^5)

julia> d = log(a)
7^1 + 5*7^2 + O(7^3)

julia> c = exp(R(0))
7^0 + O(7^30)

julia> d = log(R(1))
0

julia> f = teichmuller(b)
2*7^0 + 4*7^1 + 6*7^2 + O(7^3)

julia> g = frobenius(a, 2)
7^0 + 7^1 + 2*7^2 + O(7^3)
```

# Part VII

# Nemo matrices

Dense matrices can be created over any computable ring R. There are two different kinds of implementation: a generic one for the case where no specific implementation exists (provided by AbstractAlgebra.jl), and efficient implementations of matrices over numerous specific rings provided by FLINT.

The following table shows the available matrix types, together with their base ring R and the corresponding Julia types (the type information is mainly of concern to developers).

| Base ring | Library | Element type | Parent type |
|---:|---:|---:|---:|
| Generic ring $R$ | AbstractAlgebra.jl | `Generic.Mat{T}` | `Generic.MatSpace{T}` |
| $\mathbb{Z}$ | FLINT | `ZZMatrix` | `ZZMatrixSpace` |
| $\mathbb{Z}/n\mathbb{Z}$ (small $n$) | FLINT | `zzModMatrix` | `zzModMatrixSpace` |
| $\mathbb{Z}/n\mathbb{Z}$ (large $n$) | FLINT | `ZZModMatrix` | `ZZModMatrixSpace` |
| $\mathbb{Q}$ | FLINT | `QQMatrix` | `QQMatrixSpace` |
| $\mathbb{Z}/p\mathbb{Z}$ (small $p$) | FLINT | `fpMatrix` | `fpMatrixSpace` |
| $\mathbb{F}_{p^n}$ (small $p$) | FLINT | `fqPolyRepMatrix` | `fqPolyRepMatrixSpace` |
| $\mathbb{F}_{p^n}$ (large $p$) | FLINT | `FqPolyRepMatrix` | `FqPolyRepMatrixSpace` |
| $\mathbb{R}$ (arbitrary precision) | FLINT | `RealMatrix` | `RealMatrixSpace` |
| $\mathbb{C}$ (arbitrary precision) | FLINT | `ComplexMatrix` | `ComplexMatrixSpace` |
| $\mathbb{R}$ (fixed precision) | FLINT | `ArbMatrix` | `ArbMatrixSpace` |
| $\mathbb{C}$ (fixed precision) | FLINT | `AcbMatrix` | `AcbMatrixSpace` |

The dimensions and base ring $R$ of a generic matrix are stored in its parent object.

All matrix element types belong to the abstract type `MatElem` and all of the matrix space types belong to the abstract type `MatSpace`. This enables one to write generic functions that can accept any matrix type described on this page.

Note that the preferred way to create matrices is not to use the type constructors but to use the `matrix` function, see also the Matrix element constructors section of the AbstractAlgebra manual.

# Chapter 32

# Matrix functionality

All matrix spaces described on this page provide the generic matrix functionality of AbstractAlgebra:

https://nemocas.github.io/AbstractAlgebra.jl/stable/matrix

Some of this functionality is implemented by underlying C libraries such as FLINT.

The following sections describe functionality provided in addition to the generic matrix functionality for specific rings.

## 32.1 Comparison operators

`Nemo.overlaps` – Method.

```
overlaps(x::RealMatrix, y::RealMatrix)
```

Returns `true` if all entries of $x$ overlap with the corresponding entry of $y$, otherwise return `false`.

source

`Nemo.overlaps` – Method.

```
overlaps(x::ComplexMatrix, y::ComplexMatrix)
```

Returns `true` if all entries of $x$ overlap with the corresponding entry of $y$, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::RealMatrix, y::RealMatrix)
```

Returns `true` if all entries of $x$ contain the corresponding entry of $y$, otherwise return `false`.

source

`Base.contains` – Method.

```
contains(x::ComplexMatrix, y::ComplexMatrix)
```

Returns `true` if all entries of $x$ contain the corresponding entry of $y$, otherwise return `false`.

source

In addition we have the following ad hoc comparison operators.

**Examples**

```
julia> C = RR[1 2; 3 4]
[1.0000000000000000000   2.0000000000000000000]
[3.0000000000000000000   4.0000000000000000000]

julia> D = RR["1 +/- 0.1" "2 +/- 0.1"; "3 +/- 0.1" "4 +/- 0.1"]
[[1e+0 +/- 0.101]   [2e+0 +/- 0.101]]
[[3e+0 +/- 0.101]   [4e+0 +/- 0.101]]

julia> overlaps(C, D)
true

julia> contains(D, C)
true
```

## 32.2 Scaling

Base.:<< – Method.

```
<<(x::ZZMatrix, y::Int)
```

Return $2^y x$.

source

Base.:>> – Method.

```
>>(x::ZZMatrix, y::Int)
```

Return $x/2^y$ where rounding is towards zero.

source

**Examples**

```
julia> A = ZZ[2 3 5; 1 4 7; 9 6 3]
[2   3   5]
[1   4   7]
[9   6   3]
```

```
julia> B = A<<5
[ 64    96   160]
[ 32   128   224]
[288   192    96]

julia> C = B>>2
[16   24   40]
[ 8   32   56]
[72   48   24]
```

## 32.3 Determinant

Nemo.det_divisor – Method.

```
det_divisor(x::ZZMatrix)
```

Return some positive divisor of the determinant of $x$, if the determinant is nonzero, otherwise return zero.

source

Nemo.det_given_divisor – Method.

```
det_given_divisor(x::ZZMatrix, d::Integer, proved=true)
```

Return the determinant of $x$ given a positive divisor of its determinant. If `proved == true` (the default), the output is guaranteed to be correct, otherwise a heuristic algorithm is used.

source

Nemo.det_given_divisor – Method.

```
det_given_divisor(x::ZZMatrix, d::ZZRingElem, proved=true)
```

Return the determinant of $x$ given a positive divisor of its determinant. If `proved == true` (the default), the output is guaranteed to be correct, otherwise a heuristic algorithm is used.

source

**Examples**

```
julia> A = ZZ[2 3 5; 1 4 7; 9 6 3]
[2   3   5]
[1   4   7]
[9   6   3]

julia> c = det_divisor(A)
3

julia> d = det_given_divisor(A, c)
-30
```

## 32.4 Pseudo inverse

AbstractAlgebra.pseudo_inv – Method.

```
pseudo_inv(x::ZZMatrix)
```

Return a tuple $(z, d)$ consisting of a matrix $z$ and denominator $d$ such that $z/d$ is the inverse of $x$.

**Examples**

```
julia> A = ZZ[1 0 1; 2 3 1; 5 6 7]
[1   0   1]
[2   3   1]
[5   6   7]

julia> B, d = pseudo_inv(A)
([15 6 -3; -9 2 1; -3 -6 3], 12)
```

source

## 32.5 Nullspace

Nemo.nullspace_right_rational – Method.

```
nullspace_right_rational(x::ZZMatrix)
```

Return a tuple $(r, U)$ consisting of a matrix $U$ such that the first $r$ columns form the right rational nullspace of $x$, i.e. a set of vectors over $\mathbb{Z}$ giving a $\mathbb{Q}$-basis for the nullspace of $x$ considered as a matrix over $\mathbb{Q}$.

source

## 32.6 Modular reduction

Nemo.reduce_mod – Method.

```
reduce_mod(x::ZZMatrix, y::Integer)
```

Reduce the entries of $x$ modulo $y$ and return the result.

source

Nemo.reduce_mod – Method.

```
reduce_mod(x::ZZMatrix, y::ZZRingElem)
```

Reduce the entries of $x$ modulo $y$ and return the result.

source

**Examples**

```
julia> A = ZZ[2 3 5; 1 4 7; 9 2 2]
[2   3   5]
[1   4   7]
[9   2   2]

julia> reduce_mod(A, ZZ(5))
[2   3   0]
[1   4   2]
[4   2   2]

julia> reduce_mod(A, 2)
[0   1   1]
[1   0   1]
[1   0   0]
```

## 32.7 Lifting

`AbstractAlgebra.lift` – Method.

```
lift(a::T) where {T <: Zmodn_mat}
```

Return a lift of the matrix $a$ to a matrix over $\mathbb{Z}$, i.e. where the entries of the returned matrix are those of $a$ lifted to $\mathbb{Z}$.

source

`AbstractAlgebra.lift` – Method.

```
lift(a::T) where {T <: Zmodn_mat}
```

Return a lift of the matrix $a$ to a matrix over $\mathbb{Z}$, i.e. where the entries of the returned matrix are those of $a$ lifted to $\mathbb{Z}$.

source

**Examples**

```
julia> R, = residue_ring(ZZ, 7)
(Integers modulo 7, Map: ZZ -> ZZ/(7))

julia> a = R[4 5 6; 7 3 2; 1 4 5]
[4   5   6]
[0   3   2]
[1   4   5]

julia> b = lift(a)
[-3   -2   -1]
[ 0    3    2]
[ 1   -3   -2]
```

## 32.8 Special matrices

Nemo.hadamard – Method.

```
hadamard(R::ZZMatrixSpace)
```

Return the Hadamard matrix for the given matrix space. The number of rows and columns must be equal.

source

Nemo.is_hadamard – Method.

```
is_hadamard(x::ZZMatrix)
```

Return `true` if the given matrix is Hadamard, otherwise return `false`.

source

Nemo.hilbert – Method.

```
hilbert(R::QQMatrixSpace)
```

Return the Hilbert matrix in the given matrix space. This is the matrix with entries $H_{i,j} = 1/(i + j - 1)$.

source

**Examples**

```
julia> hadamard(matrix_space(ZZ, 3, 3))
ERROR: Unable to create Hadamard matrix
[...]

julia> A = hadamard(matrix_space(ZZ, 4, 4))
[1    1    1    1]
[1   -1    1   -1]
[1    1   -1   -1]
[1   -1   -1    1]

julia> is_hadamard(A)
true

julia> B = hilbert(matrix_space(QQ, 3, 3))
[   1   1//2   1//3]
[1//2   1//3   1//4]
[1//3   1//4   1//5]
```

## 32.9 Hermite Normal Form

AbstractAlgebra.hnf – Method.

```
hnf(x::ZZMatrix)
```

Return the Hermite Normal Form of $x$.

source

AbstractAlgebra.hnf_with_transform – Method.

```
hnf_with_transform(x::ZZMatrix)
```

Compute a tuple $(H, T)$ where $H$ is the Hermite normal form of $x$ and $T$ is a transformation matrix so that $H = Tx$.

source

Nemo.hnf_modular – Method.

```
hnf_modular(x::ZZMatrix, d::ZZRingElem)
```

Compute the Hermite normal form of $x$ given that $d$ is a multiple of the determinant of the nonzero rows of $x$.

source

Nemo.hnf_modular_eldiv – Method.

```
hnf_modular_eldiv(x::ZZMatrix, d::ZZRingElem)
```

Compute the Hermite normal form of $x$ given that $d$ is a multiple of the largest elementary divisor of $x$. The matrix $x$ must have full rank.

source

AbstractAlgebra.is_hnf – Method.

```
is_hnf(x::ZZMatrix)
```

Return `true` if the given matrix is in Hermite Normal Form, otherwise return `false`.

source

**Examples**

```
julia> A = ZZ[2 3 5; 1 4 7; 19 3 7]
[ 2   3   5]
[ 1   4   7]
[19   3   7]
```

```
julia> B = hnf(A)
[1   0   16]
[0   1   18]
[0   0   27]

julia> H, T = hnf_with_transform(A)
([1 0 16; 0 1 18; 0 0 27], [-43 30 3; -44 31 3; -73 51 5])

julia> M = hnf_modular(A, ZZ(27))
[1   0   16]
[0   1   18]
[0   0   27]

julia> N = hnf_modular_eldiv(A, ZZ(27))
[1   0   16]
[0   1   18]
[0   0   27]

julia> is_hnf(M)
true
```

## 32.10 Lattice basis reduction

Nemo provides LLL lattice basis reduction. Optionally one can specify the setup using a context object created by the following function.

```
LLLContext(delta::Float64, eta::Float64, rep=:zbasis, gram=:approx)
```

Return a LLL context object specifying LLL parameters $\delta$ and $\eta$ and specifying the representation as either `:zbasis` or `:gram` and the Gram type as either `:approx` or `:exact`.

`Nemo.lll` – Method.

```
lll(x::ZZMatrix, ctx::LLLContext = LLLContext(0.99, 0.51))
```

Return a matrix $L$ whose rows form an LLL-reduced basis of the $\mathbb{Z}$-lattice generated by the rows of $x$. $L$ may contain additional zero rows.

By default, the LLL is performed with reduction parameters $\delta = 0.99$ and $\eta = 0.51$. These defaults can be overridden by specifying an optional context object.

See `lll_gram` for a function taking the Gram matrix as input.

source

`Nemo.lll_with_transform` – Method.

```
lll_with_transform(x::ZZMatrix, ctx::LLLContext = LLLContext(0.99, 0.51))
```

Return a tuple $(L, T)$ where the rows of $L$ form an LLL-reduced basis of the $\mathbb{Z}$-lattice generated by the rows of $x$ and $T$ is a transformation matrix so that $L = Tx$. $L$ may contain additional zero rows. See `lll` for the used default parameters which can be overridden by supplying an optional context object.

See `lll_gram_with_transform` for a function taking the Gram matrix as input.

source

`Nemo.lll_gram` – Method.

```
lll_gram(x::ZZMatrix, ctx::LLLContext = LLLContext(0.99, 0.51, :gram))
```

Return the Gram matrix $L$ of an LLL-reduced basis of the lattice given by the Gram matrix $x$. The matrix $x$ must be symmetric and semidefinite. If an indefinite matrix is provided, the output is undefined.

By default, the LLL is performed with reduction parameters $\delta = 0.99$ and $\eta = 0.51$. These defaults can be overridden by specifying an optional context object.

source

`Nemo.lll_gram_with_transform` – Method.

```
lll_gram_with_transform(x::ZZMatrix, ctx::LLLContext = LLLContext(0.99, 0.51, :gram))
```

Return a tuple $(L, T)$ where $L$ is the Gram matrix of an LLL-reduced basis of the lattice given by the Gram matrix $x$ and $T$ is a transformation matrix with $L = T^\top xT$. The matrix $x$ must be symmetric and non-singular.

See `lll_gram` for the used default parameters which can be overridden by supplying an optional context object.

source

`Nemo.lll_with_removal` – Method.

```
lll_with_removal(x::ZZMatrix, b::ZZRingElem, ctx::LLLContext = LLLContext(0.99, 0.51))
```

Compute the LLL reduction of $x$ and throw away rows whose norm exceeds the given bound $b$. Return a tuple $(r, L)$ where the first $r$ rows of $L$ are the rows remaining after removal.

source

`Nemo.lll_with_removal_transform` – Method.

```
lll_with_removal_transform(x::ZZMatrix, b::ZZRingElem, ctx::LLLContext = LLLContext(0.99, 0.51))
```

Compute a tuple $(r, L, T)$ where the first $r$ rows of $L$ are those remaining from the LLL reduction after removal of vectors with norm exceeding the bound $b$ and $T$ is a transformation matrix so that $L = Tx$.

source

`Nemo.lll!` – Method.

```
lll!(x::ZZMatrix, ctx::LLLContext = LLLContext(0.99, 0.51))
```

Compute an LLL-reduced basis of the $\mathbb{Z}$-lattice generated by the rows of $x$ inplace.

By default, the LLL is performed with reduction parameters $\delta = 0.99$ and $\eta = 0.51$. These defaults can be overridden by specifying an optional context object.

See `lll_gram!` for a function taking the Gram matrix as input.

source

Nemo.lll_gram! – Method.

```
lll_gram!(x::ZZMatrix, ctx::LLLContext = LLLContext(0.99, 0.51, :gram))
```

Compute the Gram matrix of an LLL-reduced basis of the lattice given by the Gram matrix $x$ inplace. The matrix $x$ must be symmetric and semidefinite.

By default, the LLL is performed with reduction parameters $\delta = 0.99$ and $\eta = 0.51$. These defaults can be overridden by specifying an optional context object.

source

**Examples**

```
julia> A = ZZ[2 3 5; 1 4 7; 19 3 7]
[ 2   3   5]
[ 1   4   7]
[19   3   7]

julia> L = lll(A, LLLContext(0.95, 0.55, :zbasis, :approx))
[-1    1   2]
[-1   -2   2]
[ 4    1   1]

julia> L, T = lll_with_transform(A)
([-1 1 2; -1 -2 2; 4 1 1], [-1 1 0; -15 10 1; 3 -2 0])

julia> G = lll_gram(gram(A))
[ 6    3   -1]
[ 3    9   -4]
[-1   -4   18]

julia> G, T = lll_gram_with_transform(gram(A))
([6 3 -1; 3 9 -4; -1 -4 18], [-1 1 0; -15 10 1; 3 -2 0])

julia> r, L = lll_with_removal(A, ZZ(100))
(3, [-1 1 2; -1 -2 2; 4 1 1])

julia> r, L, T = lll_with_removal_transform(A, ZZ(100))
(3, [-1 1 2; -1 -2 2; 4 1 1], [-1 1 0; -15 10 1; 3 -2 0])
```

## 32.11 Smith Normal Form

`AbstractAlgebra.snf` – Method.

```
snf(x::ZZMatrix)
```

Compute the Smith normal form of $x$.

source

`Nemo.snf_diagonal` – Method.

```
snf_diagonal(x::ZZMatrix)
```

Given a diagonal matrix $x$ compute the Smith normal form of $x$.

source

`AbstractAlgebra.is_snf` – Method.

```
is_snf(x::ZZMatrix)
```

Return `true` if $x$ is in Smith normal form, otherwise return `false`.

source

**Examples**

```
julia> A = ZZ[2 3 5; 1 4 7; 19 3 7]
[ 2   3   5]
[ 1   4   7]
[19   3   7]

julia> B = snf(A)
[1   0    0]
[0   1    0]
[0   0   27]

julia> is_snf(B) == true
true

julia> B = ZZ[2 0 0; 0 4 0; 0 0 7]
[2   0   0]
[0   4   0]
[0   0   7]

julia> C = snf_diagonal(B)
[1   0    0]
[0   2    0]
[0   0   28]
```

## 32.12 Strong Echelon Form

Nemo.strong_echelon_form – Method.

```
strong_echelon_form(a::zzModMatrix)
```

Return the strong echelon form of $a$. The matrix $a$ must have at least as many rows as columns.

source

Nemo.strong_echelon_form – Method.

```
strong_echelon_form(a::fpMatrix)
```

Return the strong echeleon form of $a$. The matrix $a$ must have at least as many rows as columns.

source

**Examples**

```
julia> R, = residue_ring(ZZ, 12);

julia> A = R[4 1 0; 0 0 5; 0 0 0 ]
[4   1   0]
[0   0   5]
[0   0   0]

julia> B = strong_echelon_form(A)
[4   1   0]
[0   3   0]
[0   0   1]
```

## 32.13 Howell Form

AbstractAlgebra.howell_form – Method.

```
howell_form(a::zzModMatrix)
```

Return the Howell normal form of $a$. The matrix $a$ must have at least as many rows as columns.

source

AbstractAlgebra.howell_form – Method.

```
howell_form(a::fpMatrix)
```

Return the Howell normal form of $a$. The matrix $a$ must have at least as many rows as columns.

source

**Examples**

```
julia> R, = residue_ring(ZZ, 12);

julia> A = R[4 1 0; 0 0 5; 0 0 0 ]
[4   1   0]
[0   0   5]
[0   0   0]

julia> B = howell_form(A)
[4   1   0]
[0   3   0]
[0   0   1]
```

## 32.14 Gram-Schmidt Orthogonalisation

Nemo.gram_schmidt_orthogonalisation – Method.

```
gram_schmidt_orthogonalisation(x::QQMatrix)
```

Takes the columns of $x$ as the generators of a subset of $\mathbb{Q}^m$ and returns a matrix whose columns are an orthogonal generating set for the same subspace.

**Examples**

```
julia> S = matrix_space(QQ, 3, 3);

julia> A = S([4 7 3; 2 9 1; 0 5 3])
[4   7   3]
[2   9   1]
[0   5   3]

julia> B = gram_schmidt_orthogonalisation(A)
[4   -11//5     95//123]
[2    22//5   -190//123]
[0        5    209//123]
```

source

## 32.15 Exponential

**Examples**

```
julia> A = RR[2 0 0; 0 3 0; 0 0 1]
[2.0000000000000000000                      0                      0]
[                   0   3.0000000000000000000                      0]
[                   0                      0   1.0000000000000000000]

julia> B = exp(A)
[[7.389056098930650227 +/- 4.72e-19]                                0
↪  0]
```

```
[                                0   [20.08553692318766774 +/- 1.94e-18]
↪  0]
[                                0                                    0   [2.718281828459045235
↪  +/- 4.30e-19]]
```

## 32.16 Norm

Nemo.bound_inf_norm – Method.

```
bound_inf_norm(x::RealMatrix)
```

Returns a non-negative element $z$ of type RealFieldElem, such that $z$ is an upper bound for the infinity norm for every matrix in $x$

source

Nemo.bound_inf_norm – Method.

```
bound_inf_norm(x::ComplexMatrix)
```

Returns a non-negative element $z$ of type ComplexFieldElem, such that $z$ is an upper bound for the infinity norm for every matrix in $x$

source

**Examples**

```
julia> A = RR[1 2 3; 4 5 6; 7 8 9]
[1.0000000000000000000   2.0000000000000000000   3.0000000000000000000]
[4.0000000000000000000   5.0000000000000000000   6.0000000000000000000]
[7.0000000000000000000   8.0000000000000000000   9.0000000000000000000]

julia> d = bound_inf_norm(A)
[24.000000059604644775 +/- 3.91e-19]
```

## 32.17 Shifting

**Examples**

```
julia> A = RR[1 2 3; 4 5 6; 7 8 9]
[1.0000000000000000000   2.0000000000000000000   3.0000000000000000000]
[4.0000000000000000000   5.0000000000000000000   6.0000000000000000000]
[7.0000000000000000000   8.0000000000000000000   9.0000000000000000000]

julia> B = ldexp(A, 4)
[16.000000000000000000   32.000000000000000000   48.000000000000000000]
[64.000000000000000000   80.000000000000000000   96.000000000000000000]
[112.00000000000000000   128.00000000000000000   144.00000000000000000]
```

```
julia> overlaps(16*A, B)
true
```

## 32.18 Predicates

**Examples**

```
julia> A = CC[1 2 3; 4 5 6; 7 8 9]
[1.0000000000000000000   2.0000000000000000000   3.0000000000000000000]
[4.0000000000000000000   5.0000000000000000000   6.0000000000000000000]
[7.0000000000000000000   8.0000000000000000000   9.0000000000000000000]

julia> isreal(A)
true

julia> isreal(onei(CC)*A)
false
```

## 32.19 Conversion to Julia matrices

Julia matrices use a different data structure than Nemo matrices. Conversion to Julia matrices is usually only required for interfacing with other packages. It isn't necessary to convert Nemo matrices to Julia matrices in order to manipulate them.

This conversion can be performed with standard Julia syntax, such as the following, where `A` is an `ZZMatrix`:

```
julia> A = ZZ[1 2; 3 4]
[1   2]
[3   4]

julia> Matrix{Int}(A)
2×2 Matrix{Int64}:
 1  2
 3  4

julia> Matrix{BigInt}(A)
2×2 Matrix{BigInt}:
 1  2
 3  4
```

In case the matrix cannot be converted without loss, an `InexactError` is thrown: in this case, cast to a matrix of `BigInts` rather than `Ints`.

## 32.20 Eigenvalues and Eigenvectors (experimental)

`Nemo.eigenvalues` – Method.

```
eigenvalues(A::ComplexMatrix)
```

Return the eigenvalues of A.

This function is experimental.

source

Nemo.eigenvalues_with_multiplicities – Method.

```
eigenvalues_with_multiplicities(A::ComplexMatrix)
```

Return the eigenvalues of A with their algebraic multiplicities as a vector of tuples (ComplexFieldElem, Int). Each tuple (z, k) corresponds to a cluster of k eigenvalues of $A$.

This function is experimental.

source

Nemo.eigenvalues_simple – Method.

```
eigenvalues_simple(A::ComplexMatrix, algorithm::Symbol = :default)
```

Returns the eigenvalues of A as a vector of ComplexFieldElem. It is assumed that A has only simple eigenvalues.

The algorithm used can be changed by setting the algorithm keyword to :vdhoeven_mourrain or :rump.

This function is experimental.

source

```
julia> A = CC[1 2 3; 0 4 5; 0 0 6]
[1.0000000000000000000   2.0000000000000000000   3.0000000000000000000]
[                    0   4.0000000000000000000   5.0000000000000000000]
[                    0                       0   6.0000000000000000000]

julia> eigenvalues_simple(A)
3-element Vector{ComplexFieldElem}:
 1.0000000000000000000
 4.0000000000000000000
 6.0000000000000000000

julia> A = CC[2 2 3; 0 2 5; 0 0 2]
[2.0000000000000000000   2.0000000000000000000   3.0000000000000000000]
[                    0   2.0000000000000000000   5.0000000000000000000]
[                    0                       0   2.0000000000000000000]

julia> eigenvalues(A)
1-element Vector{ComplexFieldElem}:
 2.0000000000000000000
```

```
julia> eigenvalues_with_multiplicities(A)
1-element Vector{Tuple{ComplexFieldElem, Int64}}:
 (2.0000000000000000000, 3)
```

# Part VIII

# Factorisation

Nemo provides a unified interface to handle factorisations using the Fact objects. These can only be constructed using the factor function for the respective ring elements. This is best illustrated by an example.

```
julia> fac = factor(ZZ(-6000361807272228723606))
-1 * 2 * 3 * 229^3 * 43669^3

julia> unit(fac)
-1

julia> -6000361807272228723606 == unit(fac) * prod([ p^e for (p, e) in fac])
true

julia> for (p, e) in fac; println("$p $e"); end
2 1
3 1
229 3
43669 3

julia> 229 in fac
true

julia> fac[229]
3
```

## 32.21 Basic functionality

Objects of type Fac are iterable, that is, if a is an object of type Fac, then `for (p, e) in a` will iterate through all pairs `(p, e)`, where p is a factor and e the corresponding exponent.

Base.in – Method.

```
in(a, b::Fac)
```

Test whether $a$ is a factor of $b$.

source

Base.getindex – Method.

```
getindex(a::Fac, b) -> Int
```

If $b$ is a factor of $a$, the corresponding exponent is returned. Otherwise an error is thrown.

source

Base.length – Method.

```
length(a::Fac) -> Int
```

Return the number of factors of $a$, not including the unit.

source

`AbstractAlgebra.unit` – Method.

```
unit(a::Fac{T}) -> T
```

Return the unit of the factorization.

source

# Part IX

# Miscellaneous

# Chapter 33

# Global variables and precompilation

Due to limitations of the precompilation of modules in julia, global variables referring to certain Nemo types require special attention when used inside modules. As a simple example, the following code for a module called A will not work as expected:

```
module A

using Nemo
Qx, x = QQ["x"]
f(n) = x^n
end
```

When running julia and loading the module via `using/import A`, calling `f` will lead to segmentation faults. The preferred workaround is to put the definitions of the global variables into the `__init__()` function of the module as follows:

```
module A

using Nemo

function __init__()
  global (Qx, x) = QQ["x"]
end

f(n) = x^n
end
```

Alternatively, one can disable precompilation by adding `__precompile__(false)` inside A. Note that this might have other unwanted side effects.

# Part X

# Developer

# Chapter 34

# Introduction to Nemo development

## 34.1 Relationship to AbstractAlgebra.jl

Some time in the past, Nemo was split into two packages called Nemo.jl and AbstractAlgebra.jl. The purpose was to provide a Julia only package which did some subset of what Nemo could do, albeit slower. This was requested by people in the Julia community.

Unfortunately, this hasn't been terribly successful. Most Julia developers expect that AbstractAlgebra and Nemo functionality will work for Julia matrices over AbstractAlgebra/Nemo rings. This would be possible for functions that do not conflict with `Base` or `LinearAlgebra`, at least when working with non-empty matrices. However, for reasons that we explain in both the Appendix to the AbstractAlgebra package and in the parent object section of the developer documentation, this is not possible, even in theory, for functions that would conflict with Julia's standard library or for empty matrices (except in a limited number of special cases).

Unfortunately, the Julia standard library functions do not work with matrices of Nemo objects and there is little we can do about this. Moreover, some Julia functionality isn't supported by the underlying C libraries in Nemo and would be difficult or impossible to provide on the C side.

Nowadays we see AbstractAlgebra to provide three things to Nemo:

- An abstract type hierarchy
- Generic ring constructions, e.g. generic polynomials and matrices
- Generic implementations that should work for any ring implementing the required interfaces. These interfaces are documented in the AbstractAlgebra documentation.

Nemo itself is more or less just a wrapper of the C library FLINT, which provides highly optimised implementations of commonly used rings.

Each ring implemented in FLINT is wrapped in such a way as to implement the interfaces described by AbstractAlgebra.

Most of the time an AbstractAlgebra implementation will work just as well as using Nemo, but the latter will usually be faster, due to the extremely performant C code (around half a million lines of it).

## 34.2 Layout of files

Before FLINT v3, the content of FLINT was split into a number of separate libraries:

- FLINT : polynomials and matrices over Z, Q, Z/nZ, Qp, Fq

- Arb : polynomials, matrices and special functions over balls over R and C
- Antic : algebraic number field element arithmetic
- Calcium : exact real and complex numbers, including algebraic numbers

This historical fact is the reason why Nemo's `src` directory contains subdirectories `flint`, `arb`, `antic` and `calcium`. Each of these directories contains the wrappers for functionality that used to be in the C library its name indicates. The `test` directory is similarly organised.

Within each of these directories is a set of files, one per module within the C libraries, e.g. the `fmpz.jl` file wraps the FLINT `fmpz` module for multiple precision integers. The `fmpz_poly.jl` file wraps the FLINT univariate polynomials over `fmpz` integers, and so on.

The `fmpq` prefix is for FLINT rationals, `fq` for FLINT finite fields with multiprecision characteristic, `fq_nmod` is the same but for single word characteristic. The `padic` prefix is for the field of p-adic numbers for a given `p`. The `nmod` prefix is for Z/nZ for a given `n`. The `gfp` prefix is the same as Z/nZ but where `n` is prime, so that we are dealing with a field.

The `FlintTypes.jl` file contains the implementation of all the original FLINT types.

In the `antic` directory, `nf_elem` is for elements of a number field.

The `AnticTypes.jl` file contains the former Antic types.

In the `arb` directory the `arb` prefix is for arbitrary precision ball arithmetic over the reals. The `acb` prefix is similar but for complex numbers.

The `ArbTypes.jl` file contains the former Arb types.

In the `calcium` directory the `ca` prefix is for exact real and complex numbers. There is also a `qqbar` file for the field of algebraic numbers.

## 34.3 Git, GitHub and project workflows

The official repositories for AbstractAlgebra and Nemo are:

https://github.com/Nemocas/AbstractAlgebra.jl

https://github.com/Nemocas/Nemo.jl

If you wish to contribute to these projects, the first step is to fork them on GitHub. The button for this is in the upper right of the main project page. You will need to sign up for a free GitHub account to do this.

Once you have your own GitHub copy of our repository you can push changes to it from your local machine and this will make them visible to the world.

Before sinking a huge amount of time into a contribution, please open a ticket on the official project page on GitHub explaining what you intend to do and discussing it with the other developers.

The easiest way to get going with development on your local machine is to dev AbstractAlgebra and/or Nemo. To do this, press the `]` key in Julia to enter the special package mode and type:

```
dev Nemo
```

Now you will find a local copy on your machine of the Nemo repository in

```
.julia/dev/Nemo
```

However, this will be set up to push to the official repository instead of your own, so you will need to change this. For example, if your GitHub account name is `myname`, edit the `.git/config` file in your local `Nemo` directory to say:

```
        url = https://github.com/Nemocas/Nemo.jl.git
        pushurl = https://github.com/myname/Nemo.jl
```

instead of just the first line which will already be there.

It is highly recommended that you do not work in the master branch, but create a new branch for each thing you want to contribute to Nemo.

```
git checkout -b mynewbranch
```

If your contribution is small and does not take a long time to implement, everything will likely be fine if you simply commit the changes locally, then push them to your GitHub account online:

```
git commit -a
git push --all
```

However, if you are working on a much larger project it is *highly* recommended that you frequently pull from the official master branch and rebase your new branch on top of any changes that have been made there:

```
git checkout master
git pull
git checkout mynewbranch
git rebase master
```

Note that rebasing will try to rewrite each of your commits over the top of the branch you are rebasing on (master in this case). This process will have many steps if there are many commits and lots of conflicts. Simply follow the instructions until the process is finished.

The longer you leave it before rebasing on master the longer the rebase process will take. It can eventually become overwhelming as it is not replaying the latest state of your repository over master, but each commit that you made in order. You may have completely forgotten what those older commits were about, so this can become very difficult if not done regularly.

Once you have pushed your changes to your GitHub account, go to the official project GitHub page and you should see your branch mentioned near the top of the page. Open a pull request.

Someone will review your code and suggest changes they'd like made. Simply add more commits to your branch and push again. They will automatically get added to your pull request.

Note that we don't accept code without tests and documentation. We use Documenter.jl for our documentation, in Markdown format. See our existing code for examples of docstrings above functions in the source code and look in the `docs/src` directory to see how these docstrings are merged into our online documentation.

## 34.4 Development list

All developers of AbstractAlgebra and Nemo are welcome to write to our development list to ask questions and discuss development:

https://groups.google.com/g/nemo-devel

## 34.5 Reporting bugs

Bugs should be reported by opening an issue (ticket) on the official GitHub page for the relevant project. Please state the Julia version being used, the machine you are using and the version of AbstractAlgebra/Nemo you are using. The version can be found in the Project.toml file at the top level of the source tree.

## 34.6 Development roadmap

AbstractAlgebra has a special roadmap ticket which lists the most important tickets that have been opened. If you want to contribute something high value this is the place to start:

https://github.com/Nemocas/AbstractAlgebra.jl/issues/492

This ticket is updated every so often.

## 34.7 Binaries

Binaries of C libraries for Nemo are currently made in a separate repository:

https://github.com/JuliaPackaging/Yggdrasil

If code is added to any of the C libraries used by Nemo, this jll package must be updated first and the version updated in Nemo.jl before the new functionality can be used. Ask the core developers for help with this as various other tasks must be completed at the same time.

## 34.8 Relationship to Oscar

Nemo and AbstractAlgebra are heavily used by the Oscar computer algebra system being developed in Germany by a number of universities involved in a large project known as TRR 195, funded by the DFG.

Oscar is the number one customer for Nemo. Many bugs in Nemo are found and fixed by Oscar developers and most of the key Nemo developers are part of the Oscar project.

See the Oscar website for further details:

https://www.oscar-system.org/

# Chapter 35

# Conventions

AbstractAlgebra and Nemo have adopted a number of conventions to help maintain a uniform codebase.

## 35.1 Code conventions

### Function and type names

Names of types in Julia follow the convention of CamelCase where the first letter of each word is capitalised, e.g. `Int64` and `AbstractString`.

Function/method names in Julia use all lowercase with underscores between the words, e.g. `zip` and `jacobi_symbol`.

We follow these conventions in Nemo with some exceptions:

- When interfacing C libraries the types use the same spelling and capitalisation in Nemo as they do in C, e.g. the FLINT library's `ZZPolyRingElem` remains uncapitalised in Nemo.
- Types such as `fpPolyRingElem` which don't exist under that name on the C side also use the lowercase convention as they wrap an actual C type which must be split into more than one type on the Julia side. For example `zzModPolyRingElem` and `fpPolyRingElem` on the Julia side both represent FLINT `zzModPolyRingElem`'s on the C side.
- Types of rings and fields, modules, maps, etc. are capitalised whether they correspond to a C type or not, e.g. `fqPolyRepField` for the type of an object representing the field that `fqPolyRepFieldElem`'s belong to.

.

- We omit an underscore if the first word of a method is "is" or "has", e.g. `iseven`.
- Underscores are omitted if the method name is already well established without an underscore in Julia itself, e.g. `setindex`.
- Constructors with the same name as a type use the same spelling and capitalisation as that type, e.g. `ZZRingElem(1)`.
- Functions for creating rings, fields, modules, maps, etc. (rather than the elements thereof) use CamelCase, e.g. `polynomial_ring`. We refer to these functions as parent constructors. Note that we do not follow the Julia convention here, e.g. `polynomial_ring` is a function and not a type constructor (in fact we often return a tuple consisting of a parent object and other objects such as generators with this type of function) yet we capitalise it.

- We prefer words to not be abbreviated, e.g. `denominator` instead of `den`.
- Exceptions always exist where the result would be offensive in any major spoken language (example omitted).

It is easy to find counterexamples to virtually all these rules. However we have been making efforts to remove the most egregious cases from our codebase over time. As perfect consistency is not possible, work on this has to at times take a back seat.

### Use of ASCII characters

All code and printed output in Nemo should use ASCII characters only. This is because we have developers who are using versions of the WSL that cannot correctly display non-ASCII characters.

This extends to function and operator names, which saves people having to learn how to enter them to use the system.

### Spacing and tabs

All function bodies and control blocks should be indented using spaces.

A survey of existing code shows 2, 3 or 4 space indenting commonly used in our files. Values outside this range should not be used.

When contributing to an existing file, follow the majority convention in that file. Consistency within a file is valued highly.

If you are new to Nemo development and do not already have a very strong preference, new files should be started with 3 space indenting. This maximises the likelihood that copy and paste between files will be straightforward, though modern editors ease this to some degree.

Function signatures in docstrings should have four spaces before them.

Where possible, line lengths should not exceed 80 characters.

We use a term/factor convention for spacing. This means that all (additive) terms have spaces before and after them, (multiplicative) factors usually do not.

In practice this means that `+`, `-`, `=`, `==`, `!=`, `<`, `>`, `<=`, `>=` all have spaces before and after them. The operators `*`, `/`, `^` and unary minus do not.

As per English, commas are followed by a single space in expressions. This applies for example to function arguments and tuples.

We do not put spaces immediately inside or before parentheses.

Colons used for ranges do not have spaces before or after them.

Logical operators, `&`, `|`, `&&`, etc. usually have spaces before and after them.

### Comments

Despite appearances to the contrary, we now prefer code comments explaining the algorithm as it proceeds.

The hash when used for a comment should always be followed by a space. Full sentences are preferred.

We do not generally use comments in Nemo for questions, complaints or proposals for future improvement. These are better off in a ticket on GitHub with a discussion that will be brought to the attention of all relevant parties.

Any (necessary) limitations of the implementation should be noted in docstrings.

### Layout of files

In Nemo, all types are places in special files with the word "Types" in their name, e.g. `FlintTypes.jl`. This is because Julia must be aware of all types before they are used. Separation of types from implementations makes it easy to ensure this happens.

Most implementation files present functions in a particular order, which is as follows:

- A header stating what the file is for, and if needed, any copyright notices
- Functions applying to any "types" used in the file, e.g. `parent_type`, `elem_type`, `base_ring`, `parent`, `check_parent`.
- Basic manipulation, including hashes, predicates, getters/setters, functions for creating special values (e.g. `one`, `zero` and the like), `deepcopy_internal`. These are usually fairly short functions, often a single line.
- Indexing (`getindex`, `setindex`), iteration, views.
- String I/O (`expressify` and file access, etc.)
- Arithmetic operations, usually in multiple sections, such as unary operations, binary operations, ad hoc binary operations (e.g. multiplication of a complex object by a scalar), comparisons, ad hoc comparisons, division, etc.
- More complex functionality separated into sections based on functionality provided, e.g. gcd, interpolation, special functions, solving, etc.
- Functions for mapping between different types, coercion, changing base ring, etc.
- Unsafe operators, e.g. `mul!`, `add!`, etc.
- Random generation
- Promotion rules
- Parent object call overload (e.g. for implementing `R(2)` where `R` is an object representing a ring or field, etc.)
- Additional constructors, e.g. `matrix`, which might be used instead of a parent object to construct elements.
- Parent object constructors, e.g. `polynomial_ring`, etc.

The exact order within the file is less important than generally following something like the above. This aids in finding functions in a file since all files are more or less set out the same way.

For an example to follow, see the `src/Poly.jl` and `src/generic/Poly.jl` files in AbstractAlgebra which form the oldest and most canonical example.

Headings for sections should be 80 characters wide and formed of hashes in the style that can be seen in each Nemo file.

# Chapter 36

# The type system

## 36.1 Use of Julia types in Nemo

### Concrete and abstract types

Julia does not provide a traditional class/inheritance approach to programming. Instead, the basic unit of its object oriented approach is the type definition (`struct` and `mutable struct`) and inheritance exists only on the function side of the language rather than data side. Julia provides a rich system of abstract types and unions on the data side and multimethods on the function side to effect this.

For example Julia's `Number` type is an abstract type containing all concrete types that behave like numbers, e.g. `Int64`, `Float64`, and so on.

Abstract types can also belong to other abstract types, forming a tree of abstract types.

In Nemo the most important abstract types are `Ring` and `Field`, with the latter belonging to the former so that all fields are rings, and the abstract types `RingElem` and `FieldElem` for the objects that represent elements of rings and fields, again with the latter abstract type belonging to the former.

Because this hierarchy of abstract types must form a tree, Julia is strictly speaking single inheritance, as each concrete and abstract type can belong to at most one other abstract type. For example, one could not have a diamond of abstract types with `ExactField` belonging to both `Field` and `ExactRing`.

### Recovering aspects of multiple inheritance in Nemo

Various possibilities exist to get around the limitation that abstract types must form a 'tree' in Nemo and AbstractAlgebra.

One such possibility is union types. If a function should accept one of a number of concrete or abstract types that can't all be made to belong to a single abstract type due to this limitation then one can use a union type.

For example, Nemo defines `RingElement` to be a union of `RingElem` and all the Julia standard types which behave like ring elements, e.g. all `Integer` types and types of rationals with `Integer` components.

Other union types are defined in `src/AbstractAlgebra.jl` in AbstractAlgebra.

A second feature we make use of in Nemo is parameterised types. Each concrete and abstract type can take one or more parameters. These parameter can be any other type, either concrete or abstract. For example, in Julia `Rational{T}` is for rationals with numerator and denominator of type T.

A great deal of control over parameterised types is possible, e.g. one can restrict the type parameter T using a `where` clause, e.g. to write a function that accepts all rational types with integer components of the same type one can use the type `Rational{T} where T <: Integer`.

Nemo makes use of such parameterised types for generic ring constructions such as generic polynomial rings and matrices over a given base ring. The type of the elements of the base ring is substituted for the parameter T in any concrete instantiation of the types `Poly{T}` and `Mat{T}`, which are defined in AbstractAlgebra in `src/generic/GenericTypes.jl`.

The totality of all univariate polynomial types, including those of generic `Poly{T}` types and those coming from C libraries (such as `ZZPolyRingElem`), is represented by the abstract type `PolyRingElem{T}` which in turn belongs to `RingElem`, both defined in AbstractAlgebra in `src/AbstractTypes.jl`.

Similarly, the totality of all matrix types, including explicit C types like `ZZMatrix` and the generic `Mat{T}` types is given by the abstract type `MatElem{T}`, again defined in AbstractAlgebra in `src/AbstractTypes.jl`.

This hierarchy of types allows one to write functions at any level, e.g. for all univariate polynomial types, just those with a given base type T, or for a specific concrete type corresponding to just one kind of univariate polynomial.

A third possibility to get around the single inheritance limitation of Julia is type traits. There is currently no explicit compiler/language support for traits, however various implementations exist that make use of type parameters in tricky ways. This allows one to add 'traits' to types, so long as those traits can be expressed as types. In this way, types can have multiple 'properties' at the same time, instead of belonging to just a single abstract type.

Nemo does not currently use type traits, though the map types in Nemo do make use of a custom analogue of this.

Note that unlike class based systems that dispatch on the type of a (sometimes implicit) `this` or `self` parameter, Julia methods dispatch on the type of all arguments. This is a natural fit for mathematics where all sorts of ad hoc left and right operations may be required.

### Encapsulation, maps and runtime flags

One limitation of the Julia approach is that the type of an object cannot be changed at runtime. For example one might like to insist that a given ring is in fact a field. There are three standard ways to handle this in Julia.

The first approach is to encapsulate the object in another object which does have the desired type. The second approach is to map the object to a different one of the required type (e.g. by applying a morphism). The third approach is to introduce data fields in the original type which can be changed at runtime, unlike its type. All three approaches come with downsides.

Encapsulation can be time consuming for the developer as methods which applied to the original object do not automatically apply to the encapsulated object. One can write methods which do, but this is not automatic.

Application of a map may come with a performance penalty and may be difficult for the user to navigate. Moreover, mutation of the resulting object does not result in mutation of the original object.

The third option of adding runtime data fields essentially takes one back to writing a (possibly bug ridden) interpreter. It relies on the developer implementing outer methods that make use of hand written control statements to determine which of a range of inner methods should be applied to the object. This misses the benefits of one of the main defining features of Julia, namely its multimethod system and can also make introspection more difficult.

Nemo does not apply any of these three approaches widely at present, though information which can only be known at runtime such as whether a ring is Euclidean will eventually have to be encoded using one of these three methods.

### Nemo's custom map types

It makes sense that map types in Nemo should be parameterised by the element types of both the domain and codomain of the map, and of course all maps in the system should somehow belong to an abstract type `Map`.

This leads one to consider a two parameter system of types `Map{D, C}` where `D` and `C` are the domain and codomain types respectively.

One may also wish to implement various types of map, e.g. linear maps (where the map contains a matrix representing the map) or functional maps (where the map is implemented by a Julia function) and so on. Notionally one imagines doing this with a hierarchy of two parameter abstract types all ultimately belonging to `Map{D, C}` as the root of the tree.

This approach begins to break down when constructions from homological algebra begin to be applied to maps. In such cases, the maps themselves are the object of study and functions may be applied to maps to produce other maps.

The simplest such function is composition. In a system where composition of maps always results in a map of the same type, no problem arises with the straightforward approach outlined above.

However, for various reasons (including performance) it may not be desirable or even possible to construct a composition of two given maps using the same representation as the original maps. This means that the result of composing two maps of the same type may be a map of a different type, e.g. in the worst case a general composition type.

This problem makes many homological and category theoretic operations on maps difficult or impossible to implement.

Other operations which may be desirable to implement are caching of maps (e.g. where the map is extremely time consuming to compute, such as discrete logarithms) and attaching category theoretic information to maps. Such operations can be effected by encapsulating existing maps in objects containing the extra information, e.g. a cache or a category. However all the methods that applied to the original map objects now no longer apply to the encapsulated objects.

To work around these limitations Nemo implements a four parameter `Map` type, `Map{D, C, T, U}`.

The first two parameters are the domain and codomain types as discussed above.

The parameter `T` is a "map class" which is itself an abstract type existing in a hierarchy of abstract types. This parameter is best thought of as a trait, independent of the hierarchy of abstract types belonging to `Map`, giving additional flexibility to the map types in the system.

For example, `T` may be set to `LinearMap` or `MapFromFunc`. This may be useful if one wishes to distinguish maps in other ways, e.g. whether they are homomorphisms, isomorphisms, maps with section or retraction etc. As usual, offering traits partially gets around the single inheritance problem.

The final parameter `U` is used to allow maps of a given type `U` to be composed and still result in a map of type `U`, even though the concrete type of the composition is different to that of the original maps. Methods can be written for all maps of type `U` by matching this parameter, rather than matching on the concrete type `U` of the original maps.

For example, two maps with concrete type `MyRingHomomorphism` would belong to `Map{D, C, T, MyRingHomomorphism}` as would any composition of such maps, even if the concrete type of the composition was not a `MyRingHomomorphism`.

Naturally four parameter types are rather unwieldy and so various helper functions are provided to compute four parameter map types. In the first instance one still has the type `Map{D, C}` which will give the union of all map types whose first two parameters are `D` and `C`, and where the remaining two parameters are arbitrary.

However one can also pass a map class or a concrete type `U` to a `Map` function to compute the class of all maps of the given map class or type.

For example, to write a function which accepts all maps of "type" `MyRingHomomorphism`, including all compositions of such maps, one inserts `Map(MyRingHomomorphism)` in place of the type, e.g.

```
function myfun(f::Map(MyRingHomomorphism))
```

Note the parentheses here, rather than curly braces; it's a function to compute a type! Now the function `myfun` will accept any map type whose fourth parameter `U` is set to `MyRingHomomorphism`.

This four parameter system is flexible, but may need to be expanded in the future. For example it may be useful to have more than one trait `T`. This could be achieved either by making `T` a tuple of traits or by introducing a parameterised `MapTrait` type which can be placed at that location. Naturally the `Map` functions for computing the four parameter types will have to be similarly expanded to make it easier for the user.

The map type system is currently considered experimental and our observation so far is that it is not intuitive for developers.

## 36.2 Type hierarchy diagram

The most important abstract types in the system are the element types. Their hierarchy is shown in the following diagram.

Most of the element types have a corresponding parent abstract type. These are shown in the following diagram.

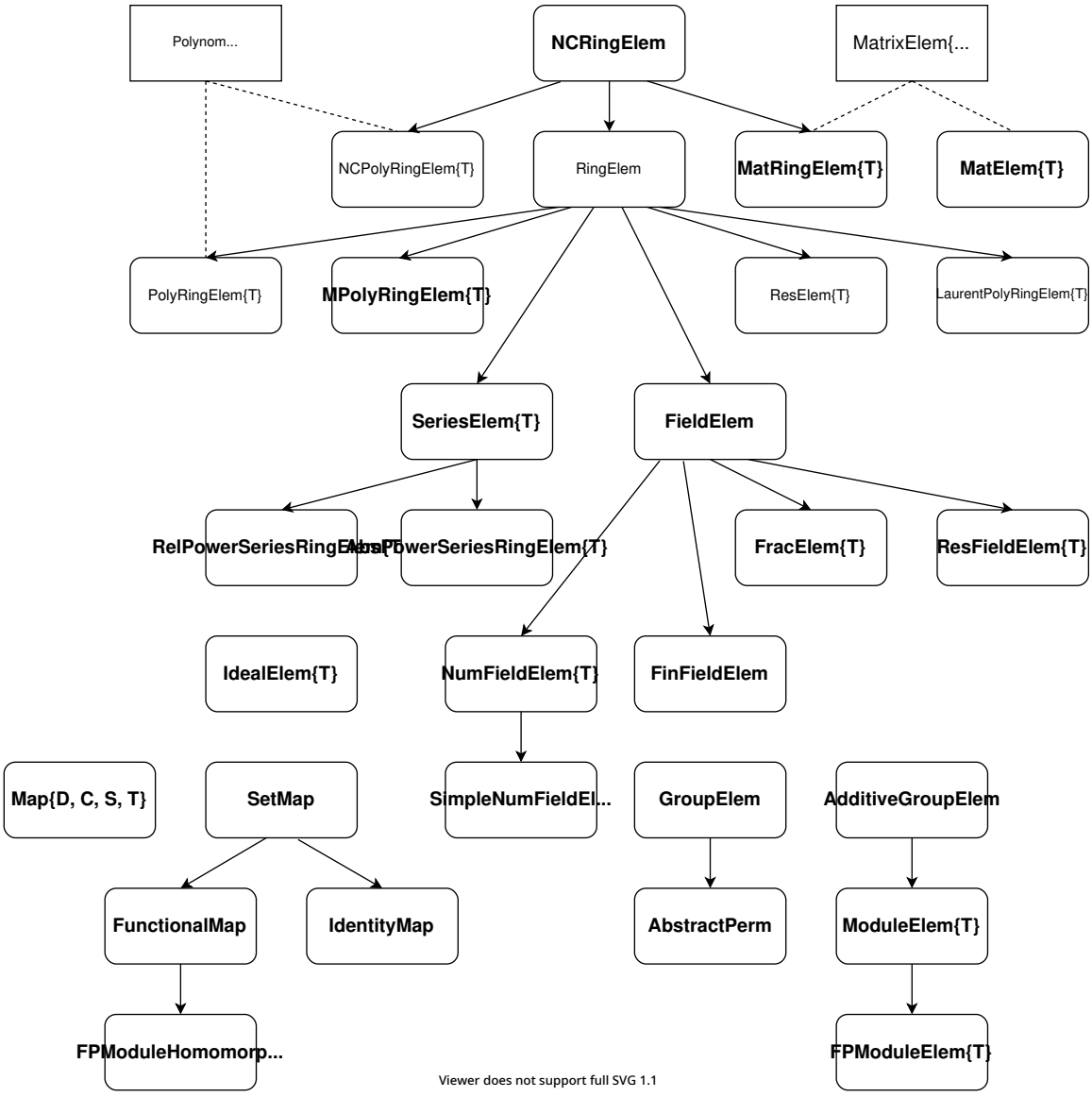



Figure 36.1: Diagram of element types

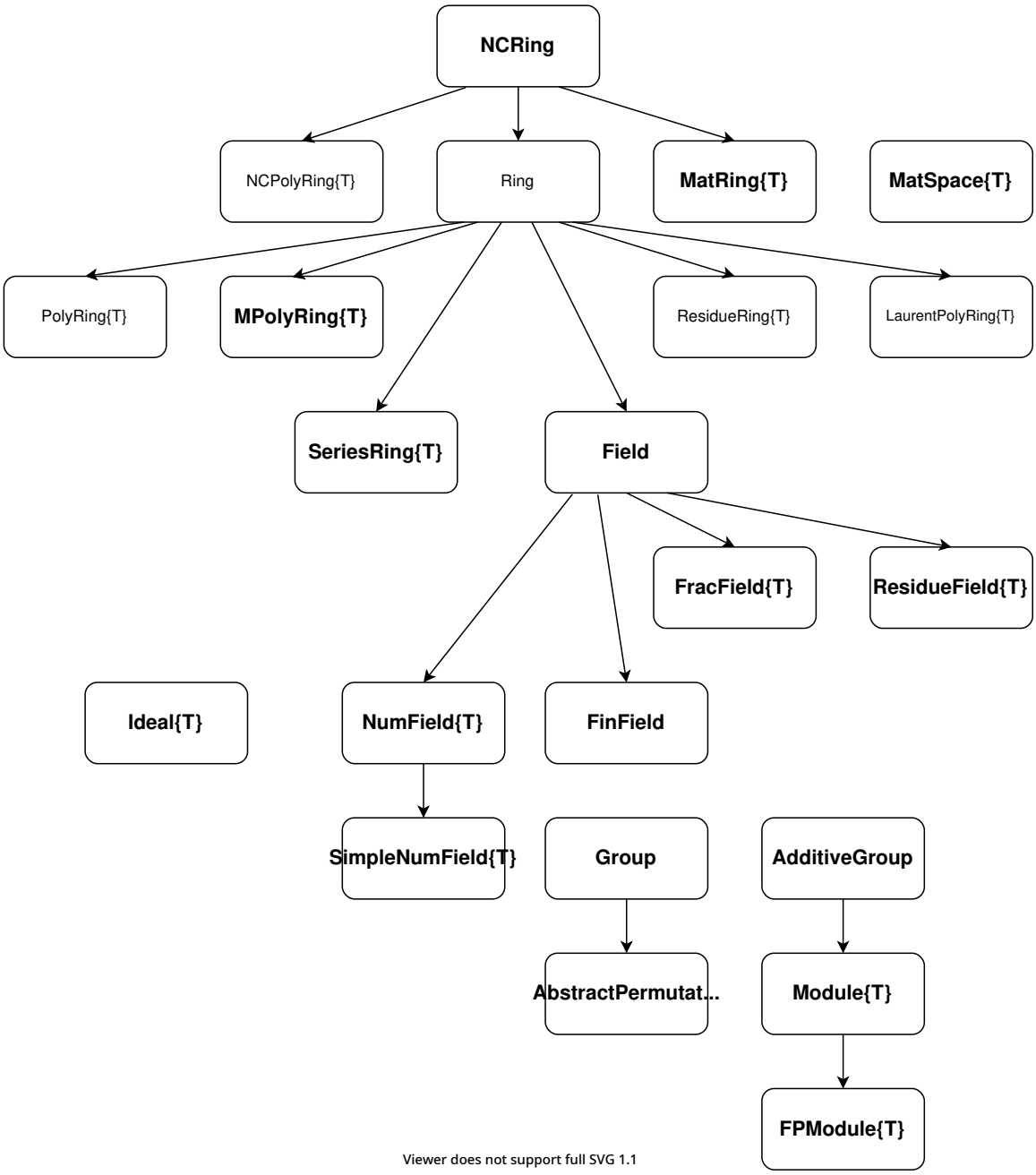



Figure 36.2: Diagram of parent types

# Chapter 37

# Parent objects

## 37.1 The use of parent objects in Nemo

### The parent/element model

As for other major computer algebra projects such as Sage and Magma, Nemo uses the parent/element model to manage its mathematical objects.

As explained in the appendix to the AbstractAlgebra documentation, the standard type/object model used in most programming languages is insufficient for much of mathematics which often requires mathematical structures parameterised by other objects.

For example a quotient ring by an ideal would be parameterised by the ideal. The ideal is an object in the system and not a type and so parameterised types are not sufficient to represent such quotient rings.

This means that each mathematical "domain" in the system (set, group, ring, field, module, etc.) must be represented by an object in the system, rather than a type. Such objects are called parent objects.

Just as one would write `typeof(a)` to get the type of an object `a` in an object/type system of a standard programming language, we write `parent(a)` to return the parent of the object `a`.

When talked about with reference to a parent in this way, the object `a` is referred to as an `element` of the parent. Thus the system is divided into elements and parents. For example a polynomial would be an element of a polynomial ring, the latter being the parent of the former.

Naturally the parent/element system leads to some issues in a programming language not built around this model. We discuss some of these issues below.

### Types in the parent/element model

As all elements and parents in Nemo are objects, those objects have types which we refer to as the element type and parent type respectively.

For example, FLINT integers have type `ZZRingElem` and the parent object they all belong to, `ZZ` has type `ZZRing`.

More complex parents and elements are parameterised. For example, generic univariate polynomials over a base ring `R` are parameterised by `R`. The base ring of a ring `S` can be obtained by the call `base_ring(S)`.

We have found it extremely useful to parameterise the type of both the parent and element objects of such a ring by the type of the elements of the base ring. Thus for example, a generic polynomial with FLINT integer coefficients would have type `Poly{ZZRingElem}`.

In practice FLINT already implements univariate polynomials over FLINT integers, and these have type `ZZPolyRingElem`. But both `ZZPolyRingElem` and the generic polynomials `Poly{ZZRingElem}` belong to the abstract type `PolyRingElem{ZZRingElem}` making it possible to write functions for all univariate polynomials over FLINT integers.

Given a specific element type or parent type it is possible to compute one from the other with the functions `elem_type` and `parent_type`. For example `parent_type(ZZPolyRingElem)` returns `ZZPolyRing` and `elem_type(ZZPolyRing)` returns `ZZPolyRingElem`. Similarly `parent_type(Generic.Poly{ZZRingElem})` returns `Generic.PolyRing{ZZRingElem}` and so on.

These functions are especially useful when writing type assertions or constructing arrays of elements insides function where only the parent object was passed.

### Other functions for computing types

Sometimes one needs to know the type of a polynomial or matrix one would obtain if it were constructed over a given ring or with coefficients/entries of a given element type.

This is especially important in generic code where it may not even be known which Julia package is being used. The user may be expecting an AbstractAlgebra object, a Nemo object or even some other kind of object to be constructed, depending on which package they are using.

The function for returning the correct type for a dense matrix is `dense_matrix_type` to which one can pass either a base ring or an element type. For example, if AbstractAlgebra is being used, `dense_matrix_type(ZZ)` will return `Mat{BigInt}` whereas if Nemo is being used it will return `ZZMatrix`.

We also have `poly_type` for univariate polynomials, `abs_series_type` for absolute series and `rel_series_type` for relative series.

In theory such functions should exist for all major object types, however they have in most cases not been implemented yet.

### Functions for creating objects of a similar type

A slightly more consistent interface for creating objects of a type that is suitable for the package currently in use is the `similar` interface.

For example, given a matrix `M` one can create one with the same dimensions but over a different ring `R` by calling `similar(M, R)`. Likewise one can create one over the same ring with different dimensions `r x c` by calling `similar(M, r, c)`.

The `similar` system is sophisticated enough to know that there is no native type provided by FLINT for matrices and polynomials over a number field. The system knows that in such cases it must create a generic matrix or polynomial over the given number field.

A great deal of thought went into the design of the `similar` system so that developers would not be required to implement similar for every pair of types in the package.

Again this interface should exist for all major Nemo domains, but the functionality is still being implemented in some cases.

### Changing base rings and map

Given a polynomial, matrix or other composite object over a base ring, it is often convenient to create a similar object but with all the entries or coefficients coerced into a different ring.

For this purpose the function `change_base_ring` is provided.

Similarly it may be useful to create the matrix or polynomial that results by applying a given map/function/lambda to each of the entries or coefficients.

For this purpose Julia's `map` function is overloaded. There are also functions specific to polynomials and matrices called `map_coefficients` and `map_entries` respectively, which essentially do the same thing.

Note that the implementation of such functions must make use of the functions discussed above to ensure that a matrix/polynomial of the right type is output.

### Parent checking

When applying binary operations to a pair of elements of a given ring, it is useful to check that they are in fact elements of the same ring. This is not possible by checking the types alone. For example elements of $Z/7Z$ and $Z/3Z$ would have the same type but different parents (one parameterised by the integer 7, the other by the integer 3).

In order to perform such a check in a function one uses `check_parent(a, b)` where `a` and `b` are the objects one wishes to assert must have the same parent. If not, an exception is raised by `check_parent`.

### Parent object constructors

Various functions are provided for constructing parent objects. For example a polynomial ring is constructed by calling a `polynomial_ring` function. Such functions are called parent object constructors.

In general parent object constructors are intended for the user and should only be used with great care in library code. There are a number of reasons for this.

One issue is that parent objects are allowed to be arbitrarily large, and they are frequently cached by the system. They can also perform arbitrary precomputations for the ring/field/module etc. that is being constructed. This can lead to excessive memory usage. It can also have odd effects when a behavior affecting change is applied to a cached parent and thus has effects in places where one might not expect it.

Secondly, those parent caches are global objects. This is problematic for any future attempts to parallelise library code. And if the caches are not regularly emptied, in the worst case memory usage can balloon excessively.

To deal with this, most parent object constructors take a `cached` keyword which specifies whether the parent object should be cached which library code should generally set to `false`. That said it is better overall to simply eschew the use of parent object constructors in library code and instead let parents be passed in via arguments (directly or indirectly, e.g. the parent or base ring of some argument might be a suitable parent for some return values or intermediate objects).

One may also use, where applicable, functions such as `similar`, `zero`, `zero_matrix`, `identity_matrix`, `change_base_ring`, `map`, etc. for constructing polynomials and matrices directly without a parent object.

However even when using these functions in library code, it is important to remember to pass `cached=false` so that the cache is not filled up by calls to the library code. This creates an additional problem, namely that if one uses `polynomial` say, to construct two polynomials over the same base ring, they will not be compatible in the sense that they will have different parents.

Forcing the creation of full parent objects into as few bottlenecks as possible will make it much easier for developers to remove problems associated with such calls when they arise in future.

# Chapter 38

# Interfaces

## 38.1 Functionality for Generic and Abstract Types

As previously mentioned, Nemo provides various generic types, e.g. `Poly{T}` for generic univariate polynomials and `Mat{T}` for generic matrices over a base ring. These and other polynomial and matrix types belong in turn to abstract types or unions thereof, e.g. `PolyRingElem{T}` is an abstract type representing all univariate polynomial types and `MatrixElem{T}` is a union of all Nemo matrix types.

When implementing generic functionality, one should usually implement it for the abstract types and unions thereof, since the new functionality will then work for all types of the specified kind, instead of just the generic types.

In order for this to work in practice, such implementations can only use functions in the relevant official interface. These are the functions required to be implemented by all types of that kind. For example, matrix implementations make heavy use of `add!` and `mul!` to accumulate entries, but they cannot make use of functions such as `subeq!` as it is not part of the official interface.

In addition to implementations for abstract types and their unions, one may also like to provide specialised implementations for the generic types e.g. `Poly{T}` and `Mat{T}` as one would for other specialised types. The generic types are based on Julia arrays internally, and so it makes perfect sense to implement lower level functionality for these types specifically, as this may lead to performance gains. Such specialised implementations can make use of any functions provided for the generic types, whether in the interface or not.

For convenience we list the most important abstract types and their unions for which one should usually prefer to write generic implementations.

- `PolyRingElem{T}` : all univariate polynomial types
- `MPolyRingElem{T}` : all multivariate polynomial types (see note below)
- `MatrixElem{T}` : union of all matrix types including matrix algebras
- `MatElem{T}` : all matrix types not including matrix algebras
- `AbsPowerSeriesRingElem{T}` : all abstract series types
- `RelPowerSeriesRingElem{T}` : all relative series types
- `LaurentSeriesElem{T}` : union of all Laurent series over rings and fields
- `PuiseuxSeriesElem{T}` : union of all Puiseux series over rings and fields
- `FPModule{T}` : all finitely presented modules over a Euclidean domain

- `FPModuleElem{T}` : all elems of fin. presented modules over a Euc. domain
- `FracElem{T}` : all fractions
- `ResElem{T}` : all elements of a residue ring
- `ResFieldElem{T}` : all elements of a residue field
- `Map{D, C}` : all maps (see Maps developer docs for a description)

N.B: inside the `Generic` submodule of AbstractAlgebra some abstract types `Blah` are only accessible by writing `AbstractAlgebra.Blah`. The unions are directly accessible. There may be generic types and abstract types with the same name, so this is more than just a convention.

Note that multivariate polynomials tend to require very specialised implementations depending heavily on implementation details of the specific multivariate type. Therefore it is rare to write implementations for the abstract type `MPolyRingElem{T}`. Instead, implementations tend to be done for each concrete multivariate type separately.

## 38.2 Generic interfaces

As mentioned above, the generic implementations in Nemo depend on carefully written interfaces for each of the abstract types provided by the system.

These interfaces are spelled out in the AbstractAlgebra documentation. Note that a generic implementation may depend on functions in both the required and optional interfaces as the optional functions are all implemented with generic fallbacks in terms of the required functions.

For convenience we provide here a list of interfaces that can be relied on in generic implementations, along with a description.

- Ring : all commutative rings in the system
- Field : all fields in the system
- NCRing : all rings in the system (not necessarily commutative)
- Euclidean Ring : Euclidean rings (see notes below)
- Univariate Polynomial Ring : all dense univariate polynomials
- Multivariate Polynomial Ring : all sparse distributed multivariate polys.
- Series Ring : all series, relative and absolute
- Residue Ring : all quotients of gcd domains with gcdx by a principal ideal
- Fraction Field : all fractions over a gcd domain with gcdx
- Module : all finitely presented modules over a Euclidean domain
- Matrix : all matrices over a commutative ring
- Map : all (set) maps in the system

Although we allow Z/nZ in our definition of Euclidean ring, much of the functionality in Nemo can be expected to misbehave (impossible inverses, etc.) when working with Euclidean rings that are not domains. In some cases the algorithms just don't exist, and in other cases we simply haven't implemented the required functionality to support all Euclidean rings for which computations can be done.

Whether a ring is a Euclidean domain or not cannot be encoded in the type. Thus there is no abstract type for Euclidean domains or their elements. Instead, generic functions rely on the existence of certain functions such as `gcdx` to implement functionality for Euclidean domains.

There is also currently no way to define a Euclidean function for a given ring (which is known to be Euclidean) and have the system recognise the ring as such. This kind of Euclidean interface may be provided in a future version of Nemo.

## 38.3 Julia interfaces we support

Many Julia interfaces rely on being able to create zero and one elements given the type only. As we use the parent/element model (see developer notes on this topic) we cannot support all Julia interfaces fully.

We do however partially implement some Julia interfaces.

- Iteration : iterators are currently provided for multivariate polynomials to iterate over the coefficients, terms and monomials. Nemo matrices can also be iterated over. Iteration proceeds down each column in turn. One can also iterate over all permutations and partitions. Finally, all finite field types can be iterated over.
- Views : because C libraries cannot be expected to implement the full range of Julia view types, views of matrices in Nemo can only be constructed for submatrices consisting of contiguous blocks in the original matrix.
- `map` and `similar` : we implement the map and similar interfaces with the caveat that we generally use parent objects where Julia would use types. See the specific documentation for the module of interest to see details.
- `zero` and `one` : these are implemented for parent types, which is not what Julia typically expects. Exceptions include the FLINT `ZZRingElem` and `QQFieldElem` types, as their parents are not parameterised, which makes it possible to implement these functions for the types as well as the parents.
- `rand` : we have a Nemo specific `rand` interface, which passes the tail of a given `rand` invocation to the `rand` function for the base ring, e.g. to create random matrix elements or polynomial coefficients and so on. In addition to this custom `rand` interface, we also support much of the Julia `rand` interface, with the usual caveat that we use parent objects instead of types where necessary.
- serialisation : unfortunately this is currently NOT implemented by Nemo, but we would certainly like to see that done in the future. It's not automatic because of the C objects that underly many of our constructions.
- `Number` : Nemo number types do NOT belong to Julia's `Number` hierarchy, as we must make all our ring element types belong to our `RingElem` abstract type. To make some Julia `Number` types cooperate with Nemo, we define the unions `RingElement` and `FieldElement` which include some Julia types, such as `BigInt` and `Rational{BigInt}`, etc. Note that fixed precision integer types cannot be expected to be well-behaved when they overflow. We recommend using Nemo integer types if one wants good performance for small machine word sized integers, but no overflow when the integer becomes large (Nemo integers are based on FLINT's multiprecision `ZZRingElem` type).
- `hash` : we implement hash functions for all major element types in Nemo.

- `getindex/setindex!/typed_hvcat` : we implement these to access elements of Nemo matrices, however see the note below on row major representation. In addition, we allow creation of matrices using the notation `R[a b; c d]` etc. This is done by overloading `typed_hvcat` for the parent object `R` instead of a type as Julia would normally expect. This produces a Nemo matrix rather than a Julia one. Note that when passed a type, Julia's `typed_hvcat` can only construct Julia matrices for Nemo types such as `ZZRingElem` and `QQFieldElem` where elements can be constructed from types alone.

Many other Julia interfaces are either not yet implemented or only very partially implemented.

## 38.4 Column major vs row major matrices

Whereas Julia uses column major representation for its matrices, Nemo follows the convention of the C libraries it wraps and uses row major representation. Although Julia 2-D arrays are used internally in Nemo's generic matrix type, the interface from the perspective of the user is still the Nemo row major convention, not the Julia column major convention.

In row major representation, some row operations may be able to be performed more cheaply than similar column operations. In column major representation the converse is true. This may mean that some Julia matrix implementations may perform more slowly if naively ported to Nemo matrices, unless suitably modified.

# Chapter 39

# Specific topics

## 39.1 Julia arithmetic

At the console, Julia arithmetic is often defined in a way that a numerical person would expect. For example, `3/1` returns a floating point number `3.0`, `sqrt(4)` returns the floating point number `2.0` and `exp(0)` returns the floating point number `1.0`.

In each case the ring is changed from the input to the output of the function. Whilst this is often what one expects to happen in a computer algebra system, these are not the definitions one would want for algebraic operations.

In this section we describe the alternatives we have implemented to allow algebraic computations, particularly for rings and fields.

### `divexact` and `divides`

Nemo implements numerous kinds of division:

- floating point division using the `/` operator as per Julia
- exact division in a ring using `divexact` and `divides`
- quotient field element construction using `//` as per Julia
- Euclidean division using `div`, `rem`, `divrem`, `mod` and `%`

The expression `divexact(a, b)` for `a` and `b` in a ring `R` returns a value `c` in `R` such that `a = bc`. If such an element of `R` does not exist, an exception is raised.

To instead test whether such an element exists, `divides(a, b)` returns a tuple `(flag, q)` where `flag` is a boolean saying whether such an exact quotient exists in the ring and if so `q` is such a quotient.

### Euclidean division

Nemo must provide Euclidean division, i.e. given `a` and `b` in a Euclidean ring `R` it must be able to find `q` and `r` such that `a = bq + r` with `r` smaller than `a` with respect to some fixed Euclidean function on `R`. There are some restrictions imposed by Julia however.

Firstly, `%` is a constant alias of `rem` in Julia, so these are not actually two independent functions but the same function.

Julia defines `div`, `rem` and `divrem` for integers as a triple of functions that return Euclidean quotient and remainder, where the remainder has the same sign as the dividend, e.g. `rem(1, 3) == 1` but `rem(-2, 3) == -2`. In other words, this triple of functions gives Euclidean division, but without a consistent set of representatives.

When using Nemo at the console (or indeed inside any other package without importing the internal Nemo definitions) `div`, `rem` and `divrem` return the same values as Julia and these functions follows the Julia convention of making the sign of the remainder the same as the dividend over ZZ, e.g. `rem(ZZ(1), ZZ(3)) == 1` but `rem(ZZ(-2), ZZ(3)) == -2`.

Internally to Nemo however, this is not convenient. For example, Hermite normal form over ZZ will only return a unique result if there is a consistent choice of representatives for the Euclidean division. This applies to the generic HNF code in AbstractAlgebra, but similar problems exist for the generic finitely presented module code in AbstractAlgebra, even when used over Nemo integers. Thus the Julia definition of `rem` will not suffice.

Furthermore, as Nemo wraps FLINT, it is convenient that Euclidean division inside Nemo should operate the way FLINT operates. This is critical if for example one wants the result of a Hermite normal form coming from FLINT to be reduced using the same definition of Euclidean remainder as used elsewhere throughout the Nemo module and to return the same answers as the generic HNF code in AbstractAlgebra for example.

In particular, FLINT defines Euclidean remainder over the integers in line with the Julia function `mod`, namely by returning the smallest remainder with the same sign as the *divisor*, i.e. `mod(1, 3) == 1` but `mod(1, -3) == -2`.

Therefore internally, Nemo chooses `div`, `mod` and `divrem` to be a consistent triple of functions for Euclidean division, with `mod` defined as per Julia. Thus in particular, `div` and `divrem` behave differently to Julia inside of Nemo itself, viz. `Nemo.divrem(-1, 3) == (-1, 2)`.

The same definitions for `div`, `mod` and `divrem` are used internally to AbstractAlgebra as well, even for Julia integers, so that AbstractAlgebra and Nemo are both consistent internally. However, both AbstractAlgebra and Nemo export definitions in line with Julia so that behaviour at the console is consistent.

The Nemo developers have given considerable thought to this compromise and the current situation has evolved over many iterations to the current state. We do not consider this to be a situation that needs 'fixing', though we are acutely aware that many tickets will be opened complaining about some inconsistency.

When reflecting on the choice we have made, one must consider the following:

- Nemo must internally behave as FLINT does for consistency
- There are also functions such as `powmod`, `invmod` that reduce as per mod
- HNF requires a consistent set of representatives for uniqueness over ZZ

Also note that Julia's `rem` does not provide symmetric mod, a misconception that often arises. The issues here are independent of the decision to use positive remainder (for positive modulus) in FLINT, rather than symmetric mod.

We are aware that the conventions we have chosen have inconsistencies with Julia and do not have the nice property that `div`, `rem` and `divrem` are a triple of Euclidean functions inside Nemo. However, we are sure that the convention we have chosen is one of only two sensible possibilities, and switching to the other convention (apart from being a huge amount of effort) would only succeed in replacing one kind of inconsistency with another.

As a consequence of these choices, `div`, `mod` and `divrem` are a triple of functions for *all* Euclidean division across Nemo, not just for the integers. As generic code must use a consistent set of functions, we ask that developers respect this choice by using these three functions in all generic code. The functions `rem` and `%` should only be used for Julia integers, and only when one specifically wants the Julia definition.

### sqrt, inv and exp

As mentioned above, Julia does not perform computations within a given ring, but often returns a numerical result when given an exact input.

Whilst this is often what a user expects, it makes operations such as power series square root, inversion or exponentiation more tricky over an exact ring.

Therefore, AbstractAlgebra defines `sqrt`, `inv` and `exp` internally in a strictly algebraic way, returning a result only if it exists in the ring of the input and otherwise raising an exception.

For example, `AbstractAlgebra.sqrt(4) == 2`, `AbstractAlgebra.inv(-1) == -1` and `AbstractAlgebra.exp(0) == 1`.

Naturally these definitions are not so terribly useful to a user and are only needed for internal consistency. Therefore, of course these definitions are not exported by `AbstractAlgebra` so that the behaviour at the console is not affected by these definitions.

There is currently some inconsistency in that Nemo follows the Julia numerical definitions internally rather than following the algebraic definitions provided internally in AbstractAlgebra. This may or may not change in future.

It is worth recalling that Julia provides `isqrt` for integer square root. This is not sufficient to solve our problem as we require square root for all rings, not just integers. We don't feel that developers will want to type `isqrt` rather than `sqrt` internally for all rings.

A number of changes are expected to be made with regard to the behaviour of root taking and division functions, including the ability to specify high performance alternatives that do not check the exactness of the computation. These changes are being discussed on the Nemo ticket https://github.com/Nemocas/Nemo.jl/issues/862 In particular, the table given there by thofma represents the current consensus on the changes that will be made in the future.

Note that many of the above issues with exact computations in rings exist for all the Julia transcendental functions, `sin`, `cos`, `log`, etc., of which there are many. If we ever add some kind of generic power series functions for these, we may extend the internal definitions to include exact algebraic versions of all these functions. At least for now this is not a pressing issue.

The way that AbstractAlgebra deals with functions which must have a different definition inside the module than what it exports is as follows. Firstly, we do not import the functions from Base or export the functions at all. Internally we make our definitions as we want them, but then we overload the Base version explicitly to do what the console version of the function should do. This is done by explicitly defining `Base.sqrt(::ZZRingElem)` for example without explicitly importing `sqrt` from Base, etc.

In the Generic module discussed below, we import the definitions from AbstractAlgebra rather than Base.

### Determinant

Another function which Nemo handles differently to Julia is `det` for determinant of matrices. If the input is an integer matrix, Nemo outputs an integer rather than a floating point number for the determinant.

However, this is not such an acute problem as Julia's `det` has now been placed in `LinearAlgebra` rather than `Base`. Moreover, Nemo has its own matrices and so does not conflict with the definition of `det` for Julia matrices.

It is important for developers to understand this difference however. It is not generally wise to use the Julia linear algebra functionality on the Julia matrices underlying generic Nemo matrices for this reason.

## 39.2 The Generic submodule

In AbstractAlgebra we define a submodule called Generic. The purpose of this module is to allow generic constructions over a given base ring. For example in Nemo, `R, x = Generic.polynomial_ring(ZZ, "x")` will construct a generic polynomial ring over Nemo integers instead of constructing a FLINT polynomial ring.

In other words x will have the type `Generic.Poly{ZZRingElem}` instead of the usual `ZZPolyRingElem`.

The ability to construct generic polynomials and matrices and the like is useful for test code and for tracking down bugs in basic arithmetic. It is also useful for performance comparison of arithmetic defined for generic ring constructions vs the specialised implementations provided by C libraries like FLINT.

Whilst most developers will not need to use the Generic module specifically, unless they have such needs, all Nemo developers need to understand how to define new generic ring constructions and functions for them. They also need to understand some subtleties that arise because of this mechanism.

Firstly, a generic construction like `polynomial_ring` must be defined inside the Generic submodule of AbstractAlgebra. All files inside the `src/generic` directory of AbstractAlgebra exist for this purpose. However, exporting from that submodule will not export the functionality to the Nemo user.

To do this, one must add a function `polynomial_ring` for example, in `src/Poly.jl`, say, which calls `Generic.polynomial_ring`. Then one needs to export `polynomial_ring` from AbstractAlgebra (also in that file).

Similarly, all functions provided for generic polynomial rings are not automatically available, even when exported from the Generic submodule. Two additional things are required, namely an import from Generic into AbstractAlgebra and then an export from AbstractAlgebra to the user.

An exception to this is if there is a function with the same name in AbstractAlgebra (i.e. in the top level `src` directory). In this case it is sufficient to simply import that function into `Generic` in the file `src/Generic.jl`.

In the former case, two large lists exist in `src/AbstractAlgebra.jl` with these imports and exports. These are kept in alphabetical order to prevent duplicate imports/exports being added over time.

If one wishes to extend a definition provided by Base, one can simply overload `Base.blah` inside the Generic submodule directly. Exceptions to this include the `div`, `mod`, `divrem`, `sqrt`, `inv` and `exp` functions mentioned above.

For AbstractAlgebra types, one still defines these exceptions `blah` by overloading `Base.blah` directly inside Generic. However, for the versions that would conflict with the Julia definition (e.g. the definition for `Int`), we instead define `AbstractAlgebra.blah` for that specific type and a fallback `AbstractAlgebra.blah(a) = Base.blah(a)` which calls the Base version of the function for all other types. Of course we do not export `blah` from AbstractAlgebra.

In order to make the AbstractAlgebra version available in Generic (rather than the Base version), we do not import `blah` from Base inside Generic, but instead import it from AbstractAlgebra. One can see these imports for the exceptional functions `blah` in the file `src/Generic.jl`.

## 39.3 Unsafe operations and aliasing

As with most object oriented languages that overload arithmetic operators, Julia creates new objects when doing an arithmetic operation. For example, `BigInt(3) + BigInt(5)` creates a new `BigInt` object to return the value `BigInt(8)`. This can be problematic when accumulating many such operations in a single coefficient of a polynomial or entry of a matrix due to the large number of temporary objects the garbage collector must allocate and clean up.

To speed up such accumulations, Nemo provides numerous unsafe operators, which mutate the existing elements of the polynomial, matrix, etc. These include functions such as `add!`, `mul!`, `zero!` and `addmul!`.

These functions take as their first argument the object that should be modified with the return value.

Note that functions such as `sub!`, `submul!` and `subeq!` are not in the official interface and not provided consistently, thus generic code cannot rely on them existing. So far it has always been the case that when doing accumulation where subtraction is needed rather than addition, that a single negation can be performed outside the accumulation loop and then the additive versions of the functions can be called inside the loop where the performance matters.

If we encounter cases in future where this is not the case, it may be necessary to add the versions that do subtraction to the interface. However, this can only be done if all rings in Nemo support it. One cannot define a fallback which turns a subtraction into a negation and an addition, as then the old performance characteristics of a new object being created per operation will result, meaning that the developer will not be able to reason about the likely performance of unsafe operators.

The function `set!` changes the value of an element to be the value of another element of the same mutable type and returns that value. Moreover, `set!` also provides type casting, that is, the types of the inputs may differ. For example, often ring elements can be set to values of type `Int`. More information on using `set!` in the context of `ZZRingElem` and `Int` can be found here. Warning: Bad things can happen if this method is used incorrectly, including memory corruption and crashes.

### Interaction of unsafe operators and immutable types

Because not all objects in Nemo are mutable, the unsafe operators somehow have to support immutable objects. This is done by also returning the "modified" return value from the unsafe operators. Naturally, this return value is not a mutated version of the original value, as that is not possible. However, it does allow the unsafe operators to accept immutable values in their first argument. Instead of modifying this value, the old value is replaced with the return value of the unsafe operator.

In order to make this work correctly, every single call to an unsafe operator must assign the return value to the original location. This requires discipline on the part of the developer using unsafe operators.

For example, to set the existing value a to `a + b` one must write

```
a = add(a, b)
```

i.e. one must have an explicit assignment to the left of the add call and indeed all the unsafe operator calls.

In the case of a mutable type (and the existence of a specialized method), add will simply modify the original a. The modified object will be returned and assigned to the exact same variable, which has no effect.

In the case of an immutable type, add does not modify the original object a as this is impossible, but it still returns the new value and assigns it to a which is what one wants.

## 39.4 Aliasing rules and mutation

One must be incredibly careful when mutating an existing value that one owns the value. If the user passes an object to a generic function for example and it changes the object without the user knowing, this can result in incorrect results in user code due to the value of their objects changing from under them.

In the first instance, functions should never modify their inputs. But further problems can also occur if the output of an unsafe operator happens to alias one of the other inputs. Such cases need to be handled exceptionally carefully.

A second issue arises as Nemo is based on FLINT, which has its own aliasing rules which are distinct from the default expectation in Julia. This leads to some interesting corner cases.

In particularly, FLINT always allows aliasing of inputs and outputs in its polynomial functions but expects matrix functions to have output matrices that are distinct from their inputs, except in a handful of functions that are specially documented to be inplace operations.

Moreover, when assigning an element to a coefficient of a polynomial or entry of a matrix FLINT always makes a copy of the element being assigned to that location. In Julia however, if one assigns an element to some index of an array, the existing object at that location is replaced with the new object. This means that inplace modification of Julia array elements is not safe as it would modify the original object that was assigned to that location, whereas in FLINT inplace modification is highly desirable for performance reasons and is completely safe due to the fact that a copy was made when the value was assigned to that location.

We have developed over a period of many years a set of rules that maximise the performance benefit we get from our unsafe operators, whilst keeping the burden imposed on the programmer to a minimum. It has been a *very* difficult task to arrive at the set of rules we have whilst respecting correctness of our code, and it would be extremely hard to change any of them.

### Arithmetic operations return a new object

In order to make it easy for the Nemo developer to create a completely new object when one is needed, e.g. for accumulating values using unsafe operators, we developed the following rules.

Whenever an arithmetic operation is used, i.e. +, -, *, unary minus and ^, Nemo always returns a new object, in line with Julia. Naturally, deepcopy also makes a copy of an object which can be used in unsafe functions.

Note that if R is a type and an element a of that type is passed to it, e.g. `R(a)` then, the Julia convention is that the original object a will be returned rather than a copy of a. This convention ensures there is not an additional cost when coercing values that are already of the right type, e.g in generic code where coercion may or may not be needed depending on the type.

We extend this convention to parent objects R and elements a of that parent. In particular, `R(a)` cannot be used to make a copy of a for use in an unsafe function if R is the parent of a.

All other functions *may* also return the input object if they wish. In other words, the return value of all other functions is not suitable for use in an unsafe function. Only return values of arithmetic operations and deepcopy or objects freshly created using inner constructors will be suitable for such use.

This convention has been chosen to maximise performance of Nemo. Low level operations (where performance matters) make a new object, even if the result is the same arithmetically as one of the inputs. But higher level functions will not necessarily make a new object, meaning that they cannot be used with unsafe functions.

### Aliasing rules

We now summarise the aliasing rules used by Nemo and AbstractAlgebra. We are relatively confident by now that following these rules will result in correct code given the constraints mentioned above.

- matrices are viewed as containers which may contain elements that alias one another. Other objects, e.g. polynomials, series, etc., are constructed from objects that do not alias one another, *even in part*
- standard unsafe operators, mul!, addmul!, zero!, add! which mutate their outputs are allow to be used iff that output is entirely under the control of the caller, i.e. it was created for the purpose of accumulation, but otherwise must not be used
- all arithmetic functions i.e. unary minus, +, -, *, ^, and deepcopy must return new objects and cannot return one of their inputs
- all other functions are allowed to return their inputs as outputs

- matrix functions with an exclamation mark should not mutate the objects that occur as entries of the output matrix, though should be allowed to arbitrarily replace/swap the entries that appear in the matrix. In other words, these functions should be interpreted as inplace operations, rather than operations that are allowed to mutate the actual entries themselves
- `R(a)` where R is the parent of a, always just returns a and not a copy
- `setcoeff!` and `setindex!` and `getcoeff` and `getindex` should not make copies. Note that this implies that setcoeff! should not be passed an element that aliases another somewhere else, even in part
- Constructors for polynomials, series and similar ring element objects (that are not matrices) that take an array as input, must ensure that the coefficients being placed into the object do not alias, even in part

## 39.5 The SparsePoly module

The SparsePoly module in AbstractAlgebra is a generic module for sparse univariate polynomials over a given base ring.

This module is used internally, e.g. in the generic multivariate gcd code, however it is not particularly suitable for general use.

Firstly, whilst the representation is sparse (recursive) the algorithms used generally are not. This is because the amount of time taken by the Jit in Julia is simply too large (upwards of 6s for the first multivariate gcd).

Secondly, the order of terms in that representation is not the one which a developer would expect for a sparse univariate format.

If the Julia Jit is ever made orders of magnitude faster, it may be worth cleaning up this module and making it generally available. But for now, it should be considered internal and heavily incomplete.

## 39.6 Parent object caching

Parent objects in Nemo must be unique given the data that is used to create them. For this purpose most parent objects are cached globally and looked up upon creation. If a parent object with that data already exists, it is returned from the cache instead of creating a new one.

There are two situations where this can be problematic however.

The first situation is if one is doing some parallel programming. Here global objects are a blight and it may be necessary to turn off caching and simply ensure that that same data is only ever used once when creating parent objects.

The second situation is when doing multimodular algorithms, where many similar parent objects with different moduli are created. The cache can become overwhelmed slowing the code down or even grinding to a halt.

In both these situations one can pass `false` as an additional argument to a parent constructor to avoid caching the parent object it creates. This parameter normally has a default value of `true` and under normal circumstances doesn't need to be supplied.

## 39.7 Throw/nothrow for `check_parent`

By default the `check_parent` functions throw an exception if parents do not match. However sometimes one would like to know if they match without throwing.

For this purpose one can pass an additional `false` argument to `check_parent`. This suppresses the exception that would be thrown if the parent objects didn't match. Instead the function simply returns `true` or `false` to indicate whether they matched or not.

## 39.8 Delayed reduction

When working in residue rings, various functions will perform an arithmetic operation followed by a reduction modulo the modulus of the residue ring.

Some accumulations, e.g. in linear algebra or polynomial arithmetic, can be dramatically sped up if one can delay the reductions that would happen after each operation in the accumulation.

Some of the Generic code in Nemo is designed to allow such delayed reduction if the ring supports it and to simply use fallbacks that do the reduction after every intermediate operation if they don't.

To support delayed reduction, a ring must support the delayed reduction interface which we describe here.

Two additional functions must be supplied for the element type. We give examples for the Nemo `AbsSimpleNumFieldElem` type:

```
mul_red!(z::AbsSimpleNumFieldElem, x::AbsSimpleNumFieldElem, y::AbsSimpleNumFieldElem, red::Bool)
```

This function behaves as per `mul!` but only performs reduction if the additional boolean argument `red` is set to `true`. This function can assume that both the inputs are reduced.

```
reduce!(x::AbsSimpleNumFieldElem)
```

This function must perform reduction on an unreduced element (mutating it). Note that it must return the mutated value as per all unsafe operators.

Finally, the `add!` and operators must be able to add nonreduced values.

If one wishes to speed up generic code for rings that provide delayed reduction, one makes use of the function `addmul_delayed_reduction!` in the accumulation loop. Here is an example for accumulation into a two dimensional matrix element in Generic in a matrix multiplication routine:

```
A[i, j] = base_ring(X)()
for k = 1:ncols(X)
    A[i, j] = addmul_delayed_reduction!(A[i, j], x[i, k], y[k, j], C)
end
A[i, j] = reduce!(A[i, j])
```

Here `C` is a temporary element of the same type as the other inputs which is used internally in `addmul_delayed_reduction!` if needed.

Notice the final call to `reduce!` to reduce the accumulated value after the accumulation loop has finished.

Note that `mul_red!` is never called directly but is called inside the generic implementation of `addmul_delayed_reduction!` for rings that support delayed reduction. That generic code falls back to a call to `addmul!` which in turn falls back to `mul!` and `add!` where delayed reduction or `addmul!` are not available.